GenAI Engineer Handbook

From Foundations to Production-Ready AI Systems — Math, ML, Deep Learning, NLP & Transformers, LLMs, Generative AI, RAG, Agentic AI, MLOps/LLMOps and Interview Preparation

How to use this book: chapters follow the training roadmap (Foundations → Core ML & DL → NLP & Transformers → GenAI → RAG → Agents → MLOps → Capstone). Each chapter goes basic → advanced with diagrams, maths, visualisations, code, and ends with interview questions. Chapter 10 is a rapid-revision SME interview bank.

Prepared for Mansavi Aggarwal — AI Engineer & BU SME interview preparation

Contents

1. Foundations: Math, Python & ML Basics **4**

1.1 Python for Machine Learning . . . 4
1.2 Statistics & Probability . . . 6
1.3 Linear Algebra . . . 8
1.4 Calculus for Deep Learning . . . 9
1.5 Machine Learning Overview . . . 10
1.6 Tools & Environment . . . 12

2. Machine Learning: Classical ML & Model Building **14**

2.1 Data Preparation . . . 14
2.2 Classical ML Algorithms . . . 17
2.3 Model Evaluation & Tuning . . . 22
2.4 Model Persistence & Serving . . . 25

3. Deep Learning: Neural Networks to Generative Models **28**

3.1 Neural Network Basics . . . 28
3.2 Convolutional Neural Networks (CNN) . . . 32
3.3 Recurrent Neural Networks (RNN) . . . 38
3.4 Generative Models: GANs & VAEs . . . 41

4. NLP & Transformers: From Text Processing to LLMs **46**

4.1 Classical NLP . . . 46
4.2 Sequence Models (recap) & the Transition to Transformers . . . 48
4.3 Transformers — Architecture Deep Dive . . . 48
4.4 BERT, GPT, T5 and Their Variants . . . 52
4.5 Large Language Models (LLMs) . . . 54
4.6 Practical NLP Projects . . . 58

5. Generative AI Beyond Text: Images, Video, Audio, Multimodal **61**

5.1 Introduction to Generative AI . . . 61
5.2 Diffusion Models . . . 62
5.3 Advanced Generation: Video, Audio, Music, Multimodal . . . 65
5.4 Creative Applications . . . 66

6. RAG: Retrieval-Augmented Generation **70**

6.1 What is RAG and Why Do We Need It? . . . 70
6.2 RAG Architecture: The Big Picture . . . 71
6.3 Components of RAG — In Depth . . . 72

6.4 Types of RAG — Explained One by One . . . 77
6.5 Hands-on: Building RAG in Python . . . 87
6.6 Evaluating RAG . . . 89
6.7 Failure Modes, Fixes & Production Concerns . . . 90
6.8 Multi-Document RAG, Metadata Filtering & Enterprise Applications . . . 91
6.9 Interview Questions & Answers . . . 93
6.10 RAG Cheat Sheet . . . 99

7. Agentic AI: Building Autonomous AI Agents 100

7.1 Agent Basics . . . 100
7.2 LangChain & LangGraph . . . 102
7.3 Building Agents . . . 103
7.4 Advanced Agent Topics . . . 107

8. MLOps & LLMOps: Deploy, Scale and Maintain AI Systems 111

8.1 MLOps Fundamentals . . . 111
8.2 Deployment & Infrastructure . . . 113
8.3 CI/CD & Monitoring . . . 116
8.4 Cloud & LLMOps . . . 118

9. Capstone Projects & Assessments 122

9.1 Capstone 1: End-to-End ML/DL Project . . . 122
9.2 Capstone 2: Integrating RAG and Agents . . . 123
9.3 Real-World Use-Case Catalogue . . . 123
9.4 System Design Walkthroughs (How to Answer) . . . 124
9.5 Mock Client / SME Interview . . . 125
9.6 Incremental Project Evaluation Rubric . . . 126

10. Rapid Revision: Formula Sheet, One-Liners & Roadmap Coverage 127

10.1 Formula Sheet . . . 127
10.2 Common Confusions (say the difference in one line) . . . 128
10.3 Sixty-Second SME Answers . . . 129
10.4 Roadmap Coverage Checklist . . . 130

CHAPTER 1

Foundations: Math, Python & ML Basics

Prerequisites • Python for ML • Data manipulation & visualisation • SQL • Statistics & probability • Linear algebra • Calculus • What ML is • Tools & environment

Every later chapter — from logistic regression to transformers and diffusion — is built from four things: **linear algebra** (data and weights are vectors/matrices), **calculus** (training = minimising a loss using gradients), **probability & statistics** (models output probabilities, and we reason about uncertainty) and **code** (Python + NumPy/Pandas). This chapter builds that base with exactly the depth an AI engineer needs.

1.1 Python for Machine Learning

Core Python you must be fluent in

Concept	Why it matters in ML	Example
Lists, dicts, sets, tuples	Configs, label maps, vocabularies, deduplication	label2id = {l: i for i, l in enumerate(labels)}
Comprehensions	Fast, readable transforms of data	[x.lower() for x in texts if x]
Functions, *args/**kwargs	Reusable pipelines, model wrappers	def train(model, **params): ...
Classes & OOP	Custom models (nn.Module), datasets, agents	class MyModel(nn.Module): ...
Generators / yield	Stream huge datasets batch by batch without loading into RAM	def batches(): yield X[i:i+32]
Decorators	@tool in LangChain, @app.get in FastAPI, timing, caching	@lru_cache
Type hints, dataclasses, Pydantic	Validated schemas for APIs and LLM structured output	class Out(BaseModel): label: str
Context managers	Files, DB sessions, torch.no_grad()	with open(f) as fh: ...
async / await	Concurrent LLM/API calls, streaming endpoints	await asyncio.gather(*calls)
Virtual envs & pip	Reproducible dependencies	python -m venv .venv; pip install -r requirements.txt

NumPy — the engine under every ML library

NumPy stores data in contiguous typed arrays (ndarray) and runs operations in compiled C, so **vectorised** code is 10–100× faster than Python loops. Key ideas: **shape**, **dtype**, **axis**, **broadcasting**, **vectorisation**.

numpy_essentials.py

```
import numpy as np
X = np.random.randn(100, 3)            # 100 samples, 3 features → shape (100, 3)
X.mean(axis=0)                          # column means → shape (3,)
Xs = (X - X.mean(0)) / X.std(0)         # standardise with BROADCASTING: (100,3) - (3,)
w = np.array([0.5, -1.0, 2.0]); b = 0.1
y_hat = X @ w + b                       # matrix-vector product → (100,)
mask = y_hat > 0; X[mask].shape         # boolean indexing
np.argmax(np.array([[.1,.7,.2]]), axis=1)   # predicted class → [1]
A = np.arange(12).reshape(3, 4); A.T.shape  # (4, 3)
```

Key idea: Broadcasting rule: compare shapes from the right; dimensions are compatible if equal or one of them is 1. (100,3) and (3,) → OK; (100,3) and (100,) → error, use (100,1).

Pandas — tabular data manipulation

pandas_essentials.py

```
import pandas as pd
df = pd.read_csv("customers.csv")
df.info(); df.describe(); df.isna().sum()             # first look: types, stats, missing
df["age"] = df["age"].fillna(df["age"].median())      # impute
df = df.drop_duplicates().query("income > 0")
df["income_per_age"] = df["income"] / df["age"]       # feature engineering
df.groupby("city")["churn"].mean().sort_values()      # aggregation
pd.merge(df, orders, on="customer_id", how="left")    # SQL-style join
pd.get_dummies(df, columns=["city"], drop_first=True) # one-hot encode
df["signup"] = pd.to_datetime(df["signup"]); df["month"] = df["signup"].dt.month
df.pivot_table(index="city", columns="plan", values="revenue", aggfunc="sum")
```

Operation	Pandas	SQL equivalent
Filter rows	df[df.age > 30] / df.query()	WHERE age > 30
Select cols	df[['a','b']]	SELECT a, b
Group & aggregate	df.groupby('c').agg({'x':'mean'})	GROUP BY c
Join	pd.merge(a, b, on='id', how='left')	LEFT JOIN
Sort	df.sort_values('x', ascending=False)	ORDER BY x DESC
Window	df.groupby('c')['x'].rank()	RANK() OVER (PARTITION BY c)

Data visualisation — Matplotlib & Seaborn

Question you ask	Plot	Code
Distribution of one variable?	Histogram / KDE / box plot	sns.histplot(df.age, kde=True)
Relationship of two numerics?	Scatter / regplot	sns.scatterplot(x='age', y='income', hue='churn', data=df)
Category vs numeric?	Box / violin / bar	sns.boxplot(x='plan', y='revenue', data=df)
Correlations among many?	Heatmap of df.corr()	sns.heatmap(df.corr(numeric_only=True), annot=True)
Trend over time?	Line plot	df.set_index('date').revenue.plot()
Pairwise overview	Pair plot	sns.pairplot(df, hue='label')
Class balance?	Count plot	sns.countplot(x='label', data=df)

SQL basics for ML engineers

sql_essentials.sql

```
-- Customers with >3 orders in 2024 and their total spend
SELECT c.customer_id, c.city, COUNT(o.order_id) AS n_orders, SUM(o.amount) AS total
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_date >= '2024-01-01'
GROUP BY c.customer_id, c.city
HAVING COUNT(o.order_id) > 3
ORDER BY total DESC
LIMIT 10;

-- Window function: rank of each order within its customer
SELECT order_id, customer_id, amount,
       RANK() OVER (PARTITION BY customer_id ORDER BY amount DESC) AS rnk
FROM orders;
```

Know: SELECT/WHERE/GROUP BY/HAVING/ORDER BY, INNER vs LEFT vs FULL joins, subqueries and CTEs (WITH), window functions, indexes (why queries are fast), and from Python: *pd.read_sql(query, engine)* or SQLAlchemy ORM models.

1.2 Statistics & Probability

Descriptive statistics

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i-\mu)^2 \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i-\bar{x})^2$$

mean, population variance, sample variance (n−1 = Bessel's correction)

- **Median** is robust to outliers; **mode** = most frequent value. Skewed income data → report median.
- **Percentiles / IQR** = Q3 − Q1; outlier rule: below Q1 − 1.5·IQR or above Q3 + 1.5·IQR.
- **Skewness** (asymmetry) and **kurtosis** (tail heaviness). Log-transform right-skewed features.
- **Z-score** z = (x − μ)/σ: how many standard deviations from the mean.

$$\mathrm{Cov}(X,Y) = \mathbb{E}[(X-\mu_X)(Y-\mu_Y)] \qquad \rho_{XY} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X\,\sigma_Y} \in [-1,1]$$

Pearson correlation measures LINEAR association only; correlation ≠ causation

Probability rules

$$P(A\cup B) = P(A) + P(B) - P(A\cap B) \qquad P(A\mid B) = \frac{P(A\cap B)}{P(B)} \qquad \text{independent}: \; P(A\cap B) = P(A)P(B)$$

Bayes' theorem — the most important formula in ML

$$P(\theta\mid D) = \frac{P(D\mid\theta)\,P(\theta)}{P(D)} \qquad \text{posterior} = \frac{\text{likelihood}\times\text{prior}}{\text{evidence}}$$

Worked example (disease test): prevalence 1%, test sensitivity 99%, false-positive rate 5%. If a person tests positive, P(disease | +) = 0.99·0.01 / (0.99·0.01 + 0.05·0.99) = 0.0099/0.0594 ≈ **16.7%**. The low prior dominates — the same reason why fraud/anomaly models with rare positives have low precision. Bayes powers Naive Bayes, Bayesian optimisation, and the probabilistic view of regularisation (prior on weights).

Random variables & distributions

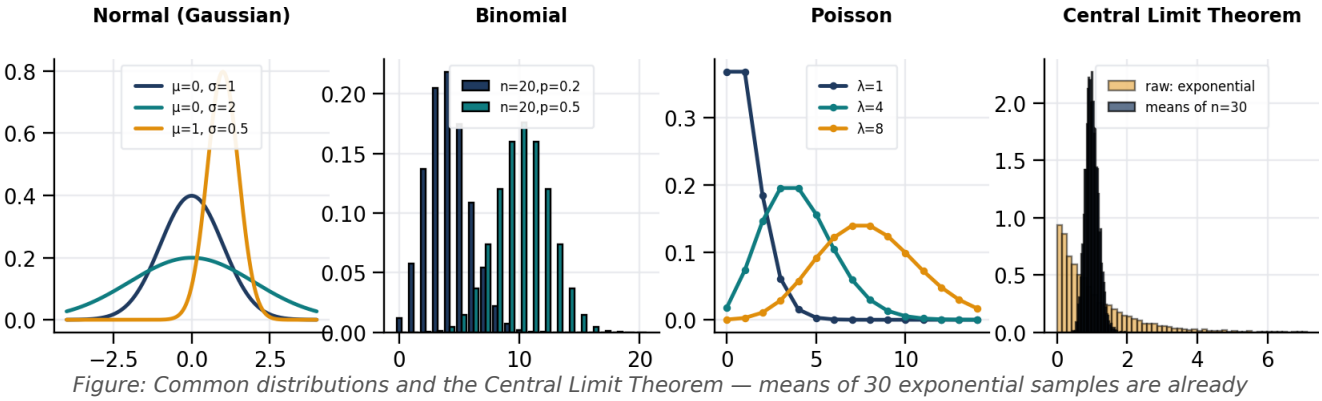


Figure: Common distributions and the Central Limit Theorem — means of 30 exponential samples are already near-Gaussian.

Distribution	Models	Mean / Var	ML usage
Bernoulli(p)	single yes/no	p / p(1−p)	binary labels → sigmoid + BCE loss
Binomial(n,p)	# successes in n trials	np / np(1−p)	A/B test conversions
Categorical	one of K classes	—	softmax output, next-token prediction
Poisson(λ)	counts per interval	λ / λ	tickets per hour, call volumes
Uniform(a,b)	equal likelihood	(a+b)/2	random init, random search
Normal(μ,σ²)	sums of many effects	μ / σ²	noise, weight init, VAE latent, diffusion noise
Exponential(λ)	time between events	1/λ	survival / wait time

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Key idea: Central Limit Theorem: the mean of n independent samples from ANY distribution with finite variance is approximately Normal(μ, σ²/n) for large n. That is why confidence intervals and A/B tests work.

Expectation, MLE and entropy

$$\mathbb{E}[X] = \sum_x x\,P(x) \qquad \hat{\theta}_{MLE} = \arg\max_\theta \prod_i P(x_i \mid \theta) = \arg\max_\theta \sum_i \log P(x_i \mid \theta)$$

Maximum Likelihood Estimation picks parameters that make the observed data most probable. Key insight: minimising **MSE** = MLE under Gaussian noise; minimising **cross-entropy** = MLE for classification. MAP (maximum a posteriori) adds a prior → L2 regularisation = Gaussian prior, L1 = Laplace prior.

$$H(p) = -\sum_x p(x)\log p(x) \qquad H(p,q) = -\sum_x p(x)\log q(x) \qquad D_{KL}(p\|q) = \sum_x p(x)\log\frac{p(x)}{q(x)}$$

Entropy = uncertainty (used in decision trees). **Cross-entropy** = loss for classifiers and LLMs. **KL divergence** = 'distance' between distributions (VAEs, RLHF penalty, knowledge distillation). Perplexity of a language model = exp(cross-entropy).

Inferential statistics & hypothesis testing

1. State H_0 (no effect) and H_1.
2. Choose significance level α (usually 0.05).
3. Compute a test statistic (t, z, χ^2, F).
4. p-value = P(data at least this extreme | H_0 true). If $p < \alpha \rightarrow$ reject H_0.

Test	Use when
t-test (two-sample)	compare means of two groups (model A vs B latency)
Paired t-test	same items measured twice (two models on same test folds)
Chi-square	independence of categorical variables; feature selection
ANOVA	means of 3+ groups
KS test	two distributions equal? (data drift)
Mann-Whitney U	non-parametric alternative to t-test

Type I error (false positive) = α; **Type II error** (false negative) = β; **power** = 1 − β. A 95% confidence interval for the mean: $\bar{x} \pm 1.96 \cdot s/\sqrt{n}$.

1.3 Linear Algebra

Data is a matrix $X \in \mathbb{R}^{n \times d}$ (n samples, d features). A neural layer is y = Wx + b. Embeddings are vectors. Attention is matrix multiplication. PCA is eigen-decomposition. Linear algebra is literally the language of ML.

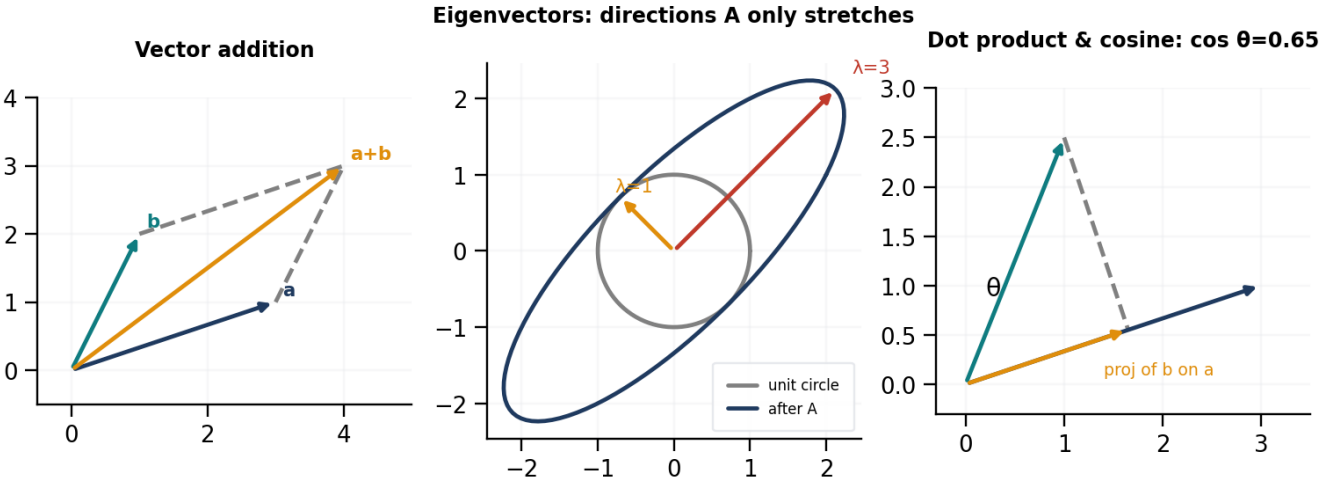


Figure: Vector addition, eigenvectors of A=[[2,1],[1,2]] (λ=1,3), and dot-product projection.

Object	Notation	ML meaning
Scalar	$a \in \mathbb{R}$	learning rate, loss value
Vector	$x \in \mathbb{R}^d$	one sample, one embedding
Matrix	$W \in \mathbb{R}^{m \times n}$	dataset, layer weights
Tensor	$T \in \mathbb{R}^{b \times c \times h \times w}$	batch of images (batch, channels, height, width)

Key operations

$$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta \qquad \cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

dot product and cosine similarity (used in embeddings, RAG, attention)

$$\|x\|_1 = \sum_i |x_i| \qquad \|x\|_2 = \sqrt{\sum_i x_i^2} \qquad (AB)_{ij} = \sum_k A_{ik} B_{kj} \quad (m \times n)(n \times p) \to (m \times p)$$

L1/L2 norms (regularisation) and matrix multiplication shape rule

- **Transpose** $(A^T)_{ij} = A_{ji}$; $(AB)^T = B^T A^T$.
- **Identity** I and **inverse** $A^{-1}A = I$ (exists only if det(A) ≠ 0, i.e. full rank).
- **Rank** = number of linearly independent rows/columns. Low-rank ideas power LoRA fine-tuning and matrix factorisation recommenders.
- **Linear regression closed form** (Normal equation): $w = (X^T X)^{-1} X^T y$.
- **Hadamard product** $A \odot B$ = element-wise (LSTM gates).

Eigen-decomposition and SVD

$$A\mathbf{v} = \lambda\mathbf{v} \qquad A = Q\Lambda Q^{-1} \qquad X = U\Sigma V^T$$

eigenvector v is only scaled (by λ) by A; SVD works for any matrix

For a covariance matrix, eigenvectors are the directions of maximum variance and eigenvalues are the variance along them — that is exactly **PCA**. **SVD** $X = U\Sigma V^T$ generalises this to any rectangular matrix; truncating to the top-k singular values gives the best rank-k approximation (compression, LSA topic models, recommender systems).

1.4 Calculus for Deep Learning

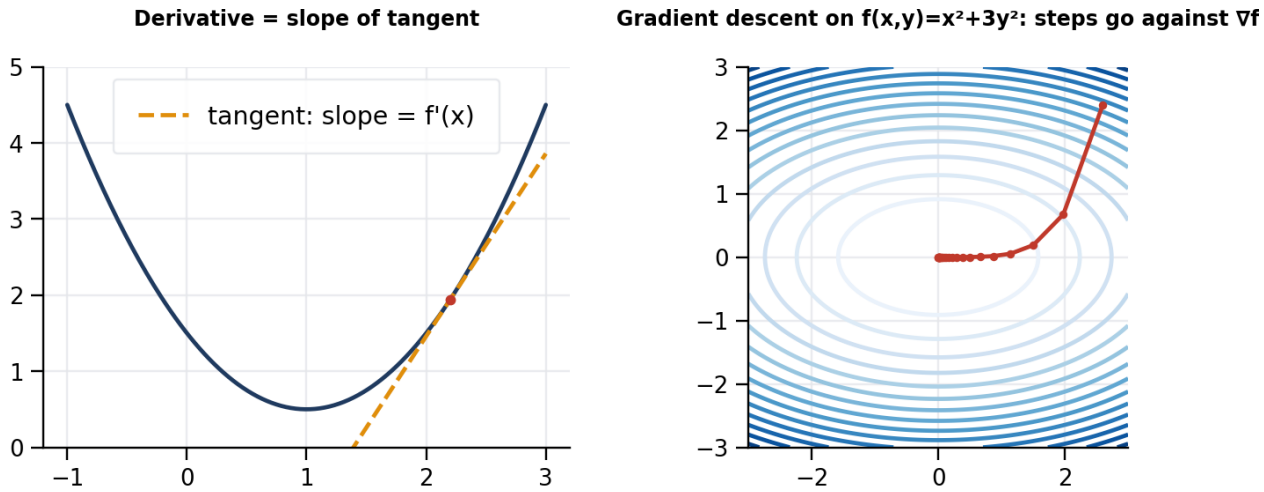


Figure: Left: derivative as slope. Right: gradient descent follows −∇f to the minimum.

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \qquad \nabla f = \left[\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n} \right]$$

Rule	Formula	Example
Power	d/dx x^n = n x^{n-1}	d/dx x^2 = 2x
Exponential/log	d/dx e^x = e^x; d/dx ln x = 1/x	used in softmax, cross-entropy
Product	(uv)' = u'v + uv'	

Rule	Formula	Example
Chain rule	d/dx f(g(x)) = f'(g(x))·g'(x)	the basis of backpropagation
Sigmoid	σ'(z) = σ(z)(1 − σ(z))	max 0.25 → vanishing gradients
Tanh	tanh'(z) = 1 − tanh²(z)	max 1

Chain rule → backpropagation

$$L = \ell(\hat{y}, y),\ \hat{y} = \sigma(z),\ z = wx + b \quad \Rightarrow \quad \frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

For sigmoid + binary cross-entropy this collapses beautifully to ∂L/∂w = (ŷ − y)·x. Gradient descent then updates every parameter:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

η = learning rate

Jacobian = matrix of all first derivatives of a vector function; **Hessian** = matrix of second derivatives (curvature). Convex functions (MSE for linear regression, log-loss for logistic regression) have one global minimum; neural network losses are non-convex but SGD still finds good minima in practice.

1.5 Machine Learning Overview

AI vs ML vs DL vs GenAI

AI
machines doing tasks needing intelligence (incl. rules)
→
ML
learn patterns from data instead of hand-written rules
→
Deep Learning
ML with multi-layer neural networks
→
Generative AI
DL models that create new content (text, image, audio)

Traditional programming: *rules + data → answers*. Machine learning: *data + answers → rules (model)*. Mitchell's definition: a program learns from experience E for task T with performance P if P on T improves with E.

Types of machine learning

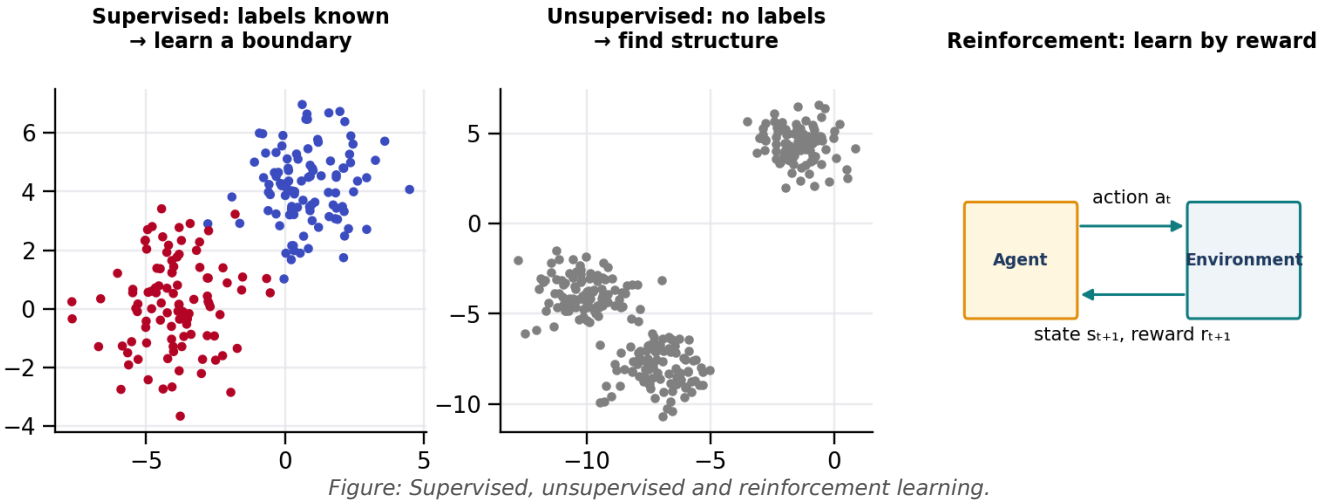
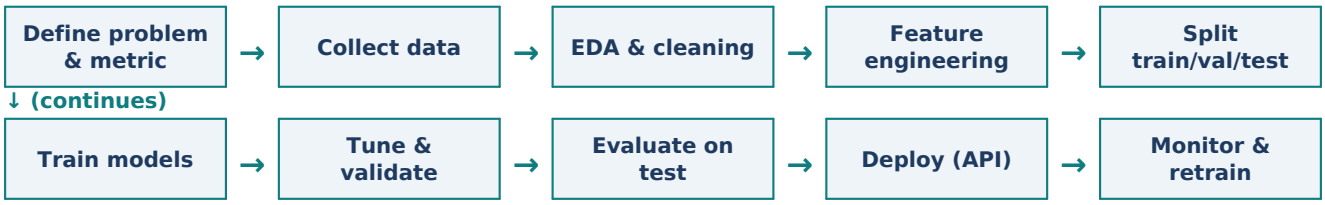


Figure: Supervised, unsupervised and reinforcement learning.

Type	Data	Goal	Algorithms	Examples
Supervised – regression	X + numeric y	predict a number	Linear reg, RF, XGBoost, NN	price, demand, ETA
Supervised – classification	X + class y	predict a label	Logistic, SVM, trees, NN	spam, churn, fraud, sentiment
Unsupervised	X only	find structure	K-means, DBSCAN, PCA, autoencoders	segmentation, anomaly detection
Semi-supervised	few labels + many unlabelled	leverage unlabelled data	pseudo-labelling, label propagation	medical imaging
Self-supervised	labels derived from data itself	learn representations	masked LM (BERT), next token (GPT), contrastive (CLIP)	foundation models
Reinforcement	rewards from environment	learn a policy	Q-learning, PPO	games, robotics, RLHF for LLMs

End-to-end ML pipeline



end_to_end_pipeline.py

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

num, cat = ["age", "income"], ["city", "plan"]
pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())]), num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")), ("oh", OneHotEncoder(handle_unknown="ignor
model = Pipeline([("pre", pre), ("clf", RandomForestClassifier(n_estimators=300, random_state=42))])

X_tr, X_te, y_tr, y_te = train_test_split(df[num + cat], df["churn"], test_size=0.2, stratify=df["churn"], random_
model.fit(X_tr, y_tr)
print(classification_report(y_te, model.predict(X_te)))
```

Interview tip: Wrap preprocessing inside a **Pipeline** so the scaler/imputer is fit only on training data — this prevents **data leakage** and lets you save one object for deployment.

ML use cases by industry

Domain	Use cases
Banking/Finance	fraud detection, credit scoring, customer propensity for sales calls, churn
Retail/E-commerce	recommendations, demand forecasting, dynamic pricing, product search
Healthcare	medical imaging, risk prediction, clinical NLP
IT operations	incident classification, ticket routing, anomaly detection, root-cause analysis

Domain	Use cases
Manufacturing	predictive maintenance, visual defect detection
Customer service	chatbots, sentiment, intent detection, RAG assistants

1.6 Tools & Environment

Tool	What it is	Must-know
Jupyter Notebook / Lab	interactive cells for EDA and experiments	kernels, magic commands (%timeit), restart & run all for reproducibility; move code to .py modules for production
Scikit-learn	classical ML library	fit / predict / transform API, Pipeline, ColumnTransformer, GridSearchCV, metrics
TensorFlow / Keras	Google's DL framework; Keras = high-level API	Sequential / Functional API, compile-fit-evaluate, callbacks, SavedModel, TF Serving
PyTorch	Meta's DL framework (research + most LLM work)	tensors, autograd, nn.Module, DataLoader, manual training loop, .to('cuda')
Hugging Face	hub of models/datasets + libraries	transformers (pipeline, AutoModel, AutoTokenizer), datasets, Trainer, PEFT, sentence-transformers
Git & GitHub	version control and collaboration	clone, branch, add, commit, push, pull request, merge conflicts, .gitignore (never commit .env/keys)
VS Code	editor/IDE	Python interpreter selection (venv), debugger & breakpoints, Jupyter extension, terminal, pytest integration

git_and_env.sh

```
git checkout -b feature/rag-retriever     # new branch
git add . && git commit -m "Add hybrid retriever"
git push -u origin feature/rag-retriever  # then open a Pull Request
python -m venv .venv && .venv\Scripts\activate   # Windows  (source .venv/bin/activate on Linux/Mac)
pip install -r requirements.txt && pip freeze > requirements.txt
```

1.7 Interview Questions — Foundations

Q1. Why is NumPy faster than Python lists?

Arrays are contiguous, fixed-type memory and operations run as vectorised C loops (SIMD), avoiding per-element Python interpreter overhead and type checks.

Q2. Explain broadcasting.

NumPy automatically stretches dimensions of size 1 so arrays of different shapes can be combined element-wise without copying, e.g. subtracting a (3,) mean vector from a (100,3) matrix.

Q3. Difference between variance and standard deviation? Why n−1?

Variance is mean squared deviation; SD is its square root in the original units. n−1 (Bessel's correction) makes the sample variance an unbiased estimator because the sample mean is itself estimated from the data.

Q4. What is the Central Limit Theorem and why does it matter?

Sample means approach a normal distribution as n grows, regardless of the original distribution. It justifies confidence intervals, z/t-tests and A/B testing on averages.

Q5. Explain Bayes' theorem with an ML example.

Posterior ∝ likelihood × prior. Naive Bayes spam filter: P(spam | words) ∝ P(words | spam)·P(spam), assuming words are independent given the class.

Q6. What is a p-value?

The probability of seeing data at least as extreme as observed if the null hypothesis were true. It is NOT the probability that the null is true.

Q7. Correlation vs causation?

Correlation measures co-movement; causation needs an intervention or causal design (randomised experiment). Ice-cream sales and drownings correlate due to the confounder summer.

Q8. What are eigenvalues/eigenvectors and where are they used?

Eigenvectors are directions a matrix only scales; eigenvalues are the scale factors. Used in PCA (covariance eigenvectors), spectral clustering, and analysing stability.

Q9. Why is the chain rule important in deep learning?

A network is a composition of functions; the chain rule lets us compute the gradient of the loss with respect to every weight layer by layer — that is backpropagation.

Q10. Cross-entropy vs KL divergence?

$H(p,q) = H(p) + KL(p\|q)$. Since $H(p)$ of the true labels is constant, minimising cross-entropy is the same as minimising KL divergence between true and predicted distributions.

Q11. Supervised vs unsupervised vs self-supervised?

Supervised uses human labels; unsupervised finds structure without labels; self-supervised creates labels from the data itself (mask a word, predict next token) and is how foundation models are pre-trained.

Q12. What is data leakage?

When information not available at prediction time leaks into training — e.g. scaling on the full dataset before splitting, or a feature derived from the target. It gives unrealistically good validation scores.

CHAPTER 2

Machine Learning: Classical ML & Model Building

Data preparation • Feature engineering • Scaling • Missing values • PCA & t-SNE • Regression • Classification • Trees & ensembles • SVM • KNN • Naive Bayes • Clustering • Evaluation • Tuning • Bias–variance • Serving

2.1 Data Preparation

Data collection & cleaning

Real data is messy. Typical cleaning steps: fix data types, remove duplicates, standardise categories ('Bangalore' vs 'Bengaluru'), handle outliers, fix invalid values (age = −3), parse dates, strip whitespace, unify units. Rule of thumb: 60–80% of ML project time is data work, and better data beats a fancier algorithm.

Handling missing values

Strategy	How	When	Risk
Drop rows	df.dropna()	few rows missing, MCAR	lose data, bias if not random
Drop column	df.drop(col)	>60–70% missing, low value	lose signal
Mean / median	SimpleImputer(strategy='median')	numeric; median for skewed	shrinks variance
Mode / 'Unknown'	most_frequent or constant	categorical	—
KNN / iterative imputer	KNNImputer, IterativeImputer (MICE)	features correlated	slower
Forward/back fill, interpolate	df.ffill(), interpolate()	time series	—
Missing indicator	add is_missing flag	missingness itself informative	more features

Missingness types: **MCAR** (completely at random), **MAR** (depends on other observed features), **MNAR** (depends on the missing value itself — e.g. high earners hide income). Tree models like XGBoost/LightGBM handle NaN natively.

Outliers

Detect with IQR rule, z-score (|z| > 3), box plots, Isolation Forest or LOF. Treat by removing (if errors), capping/winsorising, log transform, or using robust models (trees, Huber loss). Never blindly delete — in fraud detection the outliers are the signal.

Feature engineering

Technique	Description	Example
One-hot encoding	binary column per category (nominal)	city → city_Pune, city_Delhi
Label / ordinal encoding	integer per category with order	low=0, medium=1, high=2
Target / mean encoding	replace category with mean target (use CV to avoid leakage)	high-cardinality pincode
Frequency encoding	replace by category count	rare vs common merchants
Binning / discretisation	numeric → ranges	age → 18–25, 26–35...
Log / Box-Cox / Yeo-Johnson	reduce skew	log(1 + income)
Polynomial & interaction	x^2, $x_1 \cdot x_2$	area = length × width
Date-time features	hour, weekday, month, is_holiday, cyclical sin/cos	$\sin(2\pi \cdot hour/24)$
Aggregations	group stats per entity	avg transaction amount per customer last 30 days
Text features	length, TF-IDF, embeddings	ticket description → embedding
Domain ratios	expert-driven	debt-to-income ratio

Feature scaling

$$\text{Standardisation}: z = \frac{x - \mu}{\sigma} \qquad \text{Min-Max}: x' = \frac{x - x_{min}}{x_{max} - x_{min}} \qquad \text{Robust}: x' = \frac{x - \text{median}}{IQR}$$

Needs scaling	Does not need scaling
KNN, K-means, SVM (distance/margin based); linear/logistic regression with regularisation; PCA; neural networks (faster convergence)	Decision trees, Random Forest, XGBoost/LightGBM (split thresholds are scale-invariant); Naive Bayes

Watch out: Always **fit** the scaler on the training set only, then **transform** validation/test with the same parameters. Fitting on all data = leakage.

Train / validation / test split

Typical: 70/15/15 or 80/20 + cross-validation. Use **stratified** splits for imbalanced classes, **time-based** splits for time series (never shuffle the future into training), and **group** splits when the same user appears many times (GroupKFold).

Imbalanced data

- Metrics: use precision, recall, F1, PR-AUC — not accuracy (99% accuracy by always predicting 'not fraud').
- Resampling: random undersampling, oversampling, **SMOTE** (synthesise minority points by interpolating between neighbours) — apply only on training folds.
- class_weight='balanced' / scale_pos_weight in XGBoost; focal loss in DL.
- Tune the decision threshold instead of using 0.5.

Dimensionality reduction: PCA & t-SNE

High-dimensional data suffers from the **curse of dimensionality**: distances become similar, data becomes sparse, models overfit. Reduce dimensions by **feature selection** (keep a subset: filter — correlation, chi², mutual information; wrapper — RFE; embedded — L1, tree importance) or **feature extraction** (create new features: PCA, LDA, t-SNE, UMAP, autoencoders).

PCA — Principal Component Analysis (step by step)

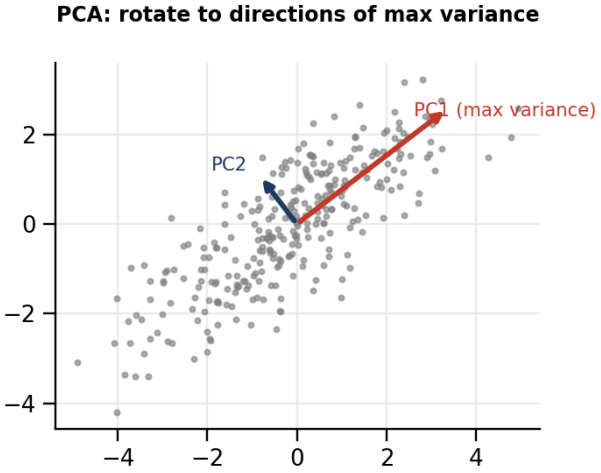


Figure: PC1 points along the direction of maximum variance.

1. Standardise the data (PCA is scale-sensitive).
2. Compute the covariance matrix C = (1/(n−1)) $X^T X$ (d × d).
3. Eigen-decompose C: eigenvectors = principal directions, eigenvalues = variance along each.
4. Sort by eigenvalue; keep the top k eigenvectors W_k.
5. Project: Z = X W_k (n × k). Explained variance ratio = λ_i / Σλ.

$$C = \frac{1}{n-1} X^T X, \qquad C\,\mathbf{w}_i = \lambda_i \mathbf{w}_i, \qquad Z = XW_k, \qquad \text{EVR}_i = \frac{\lambda_i}{\sum_j \lambda_j}$$

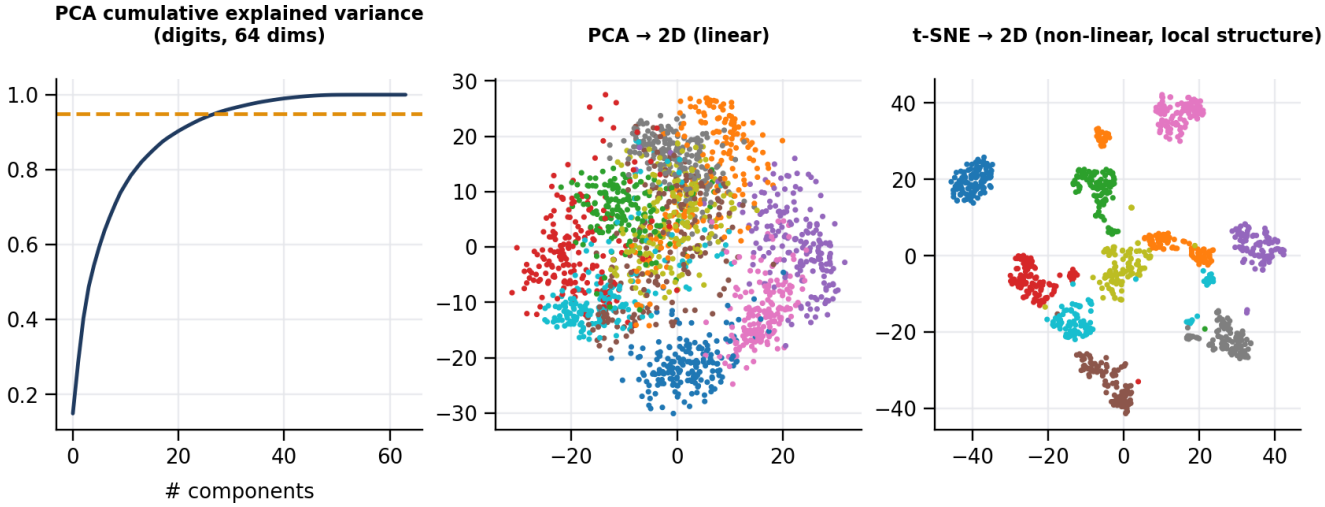


Figure: Digits dataset: ~29 of 64 components keep 95% variance; t-SNE separates clusters much more clearly but only for visualisation.

	PCA	t-SNE	UMAP
Type	linear, deterministic	non-linear, stochastic	non-linear
Preserves	global variance	local neighbourhoods	local + more global

	PCA	t-SNE	UMAP
Use	compression, denoising, pre-processing, speed	2D/3D visualisation only	visualisation, also pre-processing
New data	transform() works	no transform for new points	transform() works
Key params	n_components	perplexity (5–50), learning rate	n_neighbors, min_dist

2.2 Classical ML Algorithms

Linear Regression

$$\hat{y} = w^T x + b \qquad \text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \qquad w^* = (X^T X)^{-1} X^T y$$

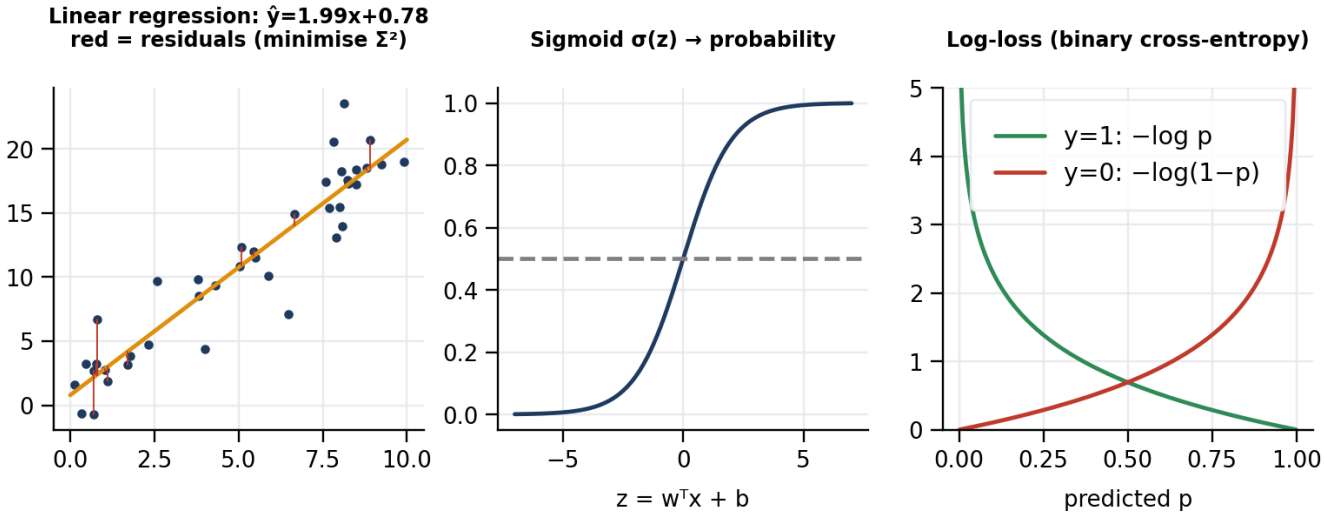


Figure: Linear regression residuals, the sigmoid, and log-loss which punishes confident wrong predictions heavily.

Assumptions (LINE): Linear relationship, Independent errors, Normal residuals, Equal variance (homoscedasticity); plus no strong multicollinearity (check VIF). Train by the closed-form normal equation (small d) or gradient descent (large data). Interpret: w_j = change in y for one-unit change in x_j, holding others constant. Metrics: MAE, MSE, RMSE, $R^2 = 1 - SS_{res}/SS_{tot}$.

Regularisation: Ridge, Lasso, Elastic Net

$$\text{Ridge}: L = \text{MSE} + \lambda\sum_j w_j^2 \qquad \text{Lasso}: L = \text{MSE} + \lambda\sum_j |w_j| \qquad \text{ElasticNet}: \lambda_1\|w\|_1 + \lambda_2\|w\|_2^2$$

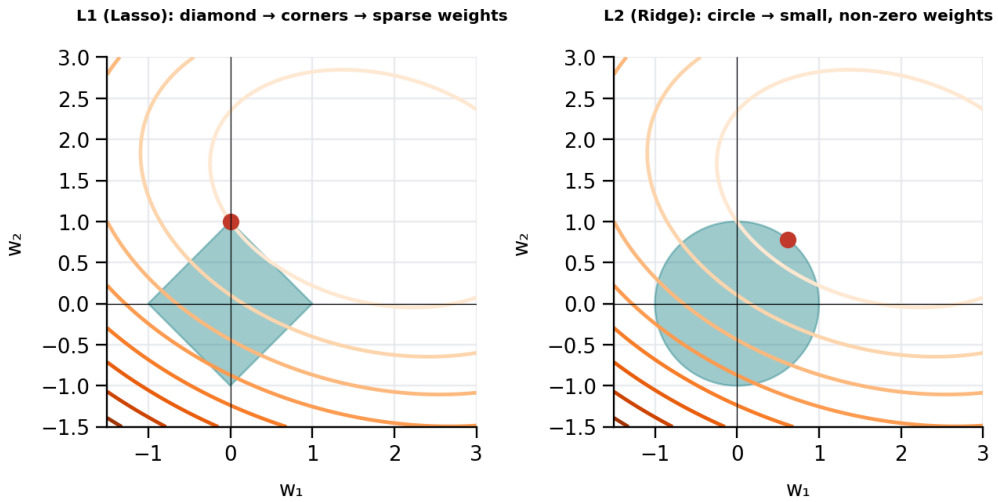


Figure: Geometric view: the loss contours first touch the L1 diamond at a corner → some weights become exactly 0 (feature selection).

Logistic Regression (classification)

$$P(y=1 \mid x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}} \qquad L = -\frac{1}{n}\sum_i [y_i \log \hat{p}_i + (1 - y_i)\log(1 - \hat{p}_i)]$$

Despite the name it is a classifier: a linear model of the **log-odds** log(p/(1−p)) = $w^T x + b$. So e^{w_j} is the odds ratio for feature j. The gradient is simply ($\hat{p} - y$)x. Multi-class: one-vs-rest or **softmax regression** with categorical cross-entropy. Decision boundary is linear (add polynomial features for curves).

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Decision Trees

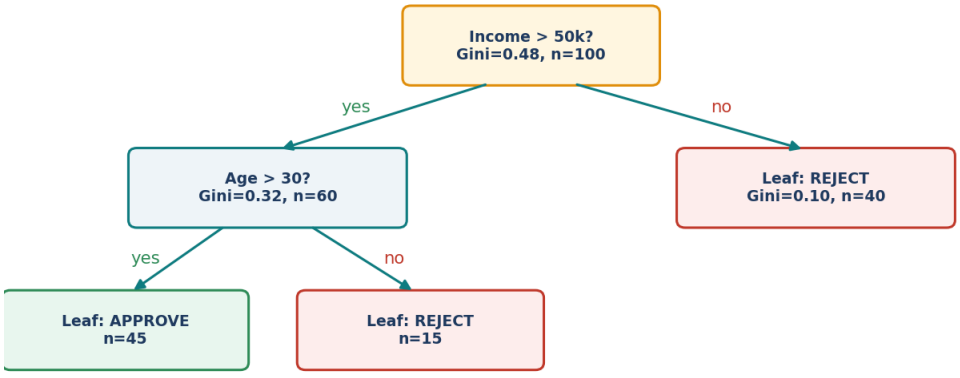


Figure: A decision tree splits on the feature/threshold that most reduces impurity.

$$\text{Gini} = 1 - \sum_k p_k^2 \qquad \text{Entropy} = -\sum_k p_k \log_2 p_k \qquad \text{IG} = H(\text{parent}) - \sum_c \frac{n_c}{n} H(c)$$

How it is built (CART, greedy): for every feature and every candidate threshold, compute the weighted impurity of the two children; pick the split with the largest impurity decrease (information gain); recurse until a stopping rule (max_depth, min_samples_split, min_samples_leaf, pure node). Regression trees use variance/MSE reduction and predict the leaf mean.

Worked Gini example: node with 6 yes / 4 no → Gini = 1 − (0.6² + 0.4²) = 0.48. Split into left (5 yes/1 no, Gini 0.278) and right (1 yes/3 no, Gini 0.375): weighted = 0.6·0.278 + 0.4·0.375 = 0.317 → decrease 0.163.

Pros	Cons
Interpretable, no scaling needed, handles non-linearity & mixed types, feature importance	Overfits easily (high variance), unstable to small data changes, axis-aligned splits; fix with pruning (ccp_alpha) or ensembles

Ensemble methods

Method	Idea	Reduces	Examples
Bagging	train many models on bootstrap samples in parallel; average/vote	variance	Bagged trees, Random Forest
Random Forest	bagging + random subset of features at each split (√d for classification) → decorrelated trees	variance	RandomForestClassifier; OOB score for free validation
Boosting	train models sequentially; each fixes previous errors	bias (and variance)	AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost
Stacking	meta-model learns to combine base models' predictions	both	LogReg on top of RF + XGB + SVM
Voting	hard (majority) or soft (avg probabilities)	variance	VotingClassifier

AdaBoost

Start with equal sample weights; fit a weak learner (stump); compute its weighted error ε; give it a say $\alpha = \frac{1}{2}\ln((1-\varepsilon)/\varepsilon)$; increase weights of misclassified samples; repeat. Final model = $\text{sign}(\Sigma\ \alpha_m\ h_m(x))$.

Gradient Boosting & XGBoost

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x), \qquad h_m \text{ fits } \; r_i = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \; \text{(for MSE: } r_i = y_i - F(x_i))$$

Each new tree fits the **residuals / negative gradients** of the current ensemble — gradient descent in function space. **XGBoost** adds: second-order (Hessian) Taylor approximation of the loss, explicit regularisation of tree complexity ($\gamma\cdot\#\text{leaves} + \frac{1}{2}\lambda\Sigma w^2$), shrinkage (learning_rate), column/row subsampling, native missing-value handling (learned default direction), and parallelised, cache-aware split finding. **LightGBM**: histogram-based, leaf-wise growth (faster, can overfit — control num_leaves). **CatBoost**: ordered target encoding for categoricals.

$$\text{Obj} = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \qquad \Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

XGBoost key params	Effect
n_estimators + learning_rate	more trees with smaller LR = better but slower; use early_stopping_rounds
max_depth / min_child_weight	tree complexity
subsample / colsample_bytree	randomness → less overfitting
reg_alpha / reg_lambda / gamma	L1, L2, min split gain
scale_pos_weight	imbalance (≈ negatives/positives)

Interview tip: Real example (bank: pick 1 lakh customers to call out of 4 lakh): train an XGBoost/LightGBM classifier on past campaign outcomes, output P(conversion), rank customers by probability and call the top 1 lakh. Evaluate with lift/gain charts and precision@top-K, not accuracy.

Support Vector Machine (SVM)

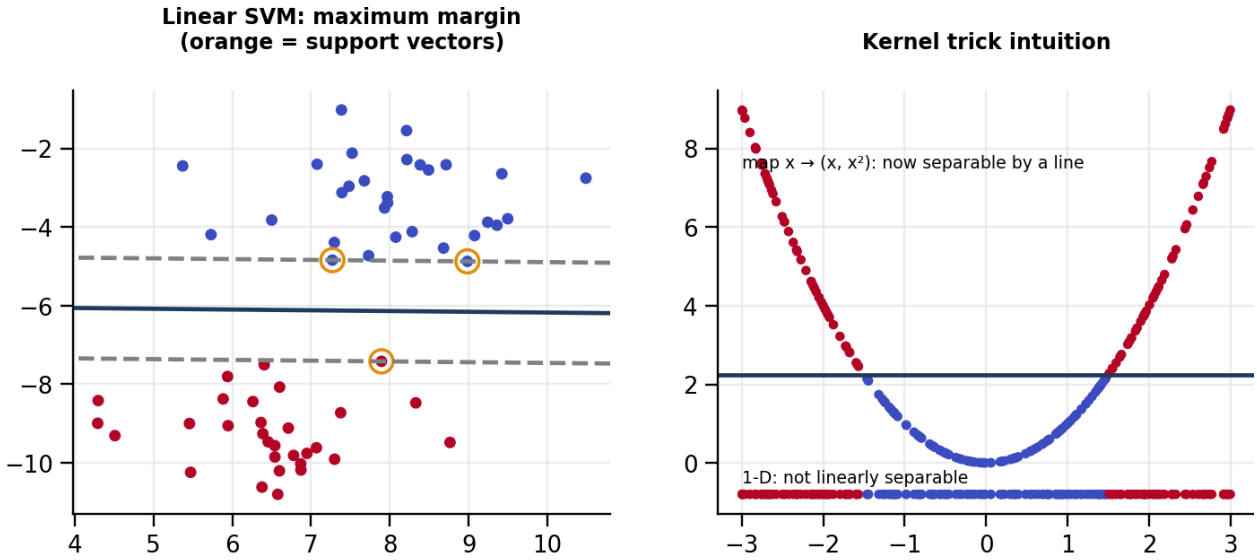


Figure: Maximum-margin hyperplane and the kernel trick idea.

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0 \qquad \text{margin} = \frac{2}{\|w\|}$$

SVM finds the hyperplane with the **widest margin**; only the **support vectors** (points on/inside the margin) define it. **C** trades margin width vs training errors (large C = hard margin, risk overfit). **Kernel trick:** replace dot products with K(x, x′) to work in a high-dimensional feature space without computing it: linear, polynomial $(x^\top x' + c)^d$, RBF $\exp(-\gamma\|x - x'\|^2)$. γ large = very wiggly boundary. Hinge loss: $\max(0, 1 - y \cdot f(x))$. Good for small/medium high-dimensional data (text); slow on >100k samples; needs scaling; SVR for regression.

K-Nearest Neighbours (KNN)

$$d(x, x') = \sqrt{\sum_j (x_j - x'_j)^2} \ \text{(Euclidean)} \qquad \sum_j |x_j - x'_j| \ \text{(Manhattan)}$$

Lazy learner: no training; to predict, find the k closest training points and take a majority vote (classification) or mean (regression), optionally distance-weighted. Small k → noisy/high variance; large k → smooth/high bias; choose k by CV (odd k for binary). Needs scaling; slow at prediction O(n·d) (use KD-tree/Ball-tree/ANN); suffers in high dimensions. The same idea powers vector search in RAG.

Naive Bayes

$$P(y \mid x_1, \ldots, x_d) \propto P(y) \prod_{j=1}^{d} P(x_j \mid y) \qquad \hat{y} = \arg\max_y \log P(y) + \sum_j \log P(x_j \mid y)$$

Assumes features are **conditionally independent** given the class ('naive'). Variants: **Gaussian** (continuous), **Multinomial** (word counts — text), **Bernoulli** (binary features). Use **Laplace**

smoothing (+1 counts) so an unseen word does not make the probability 0. Extremely fast, great baseline for spam/sentiment, works with little data; probabilities are poorly calibrated.

Decision boundaries compared

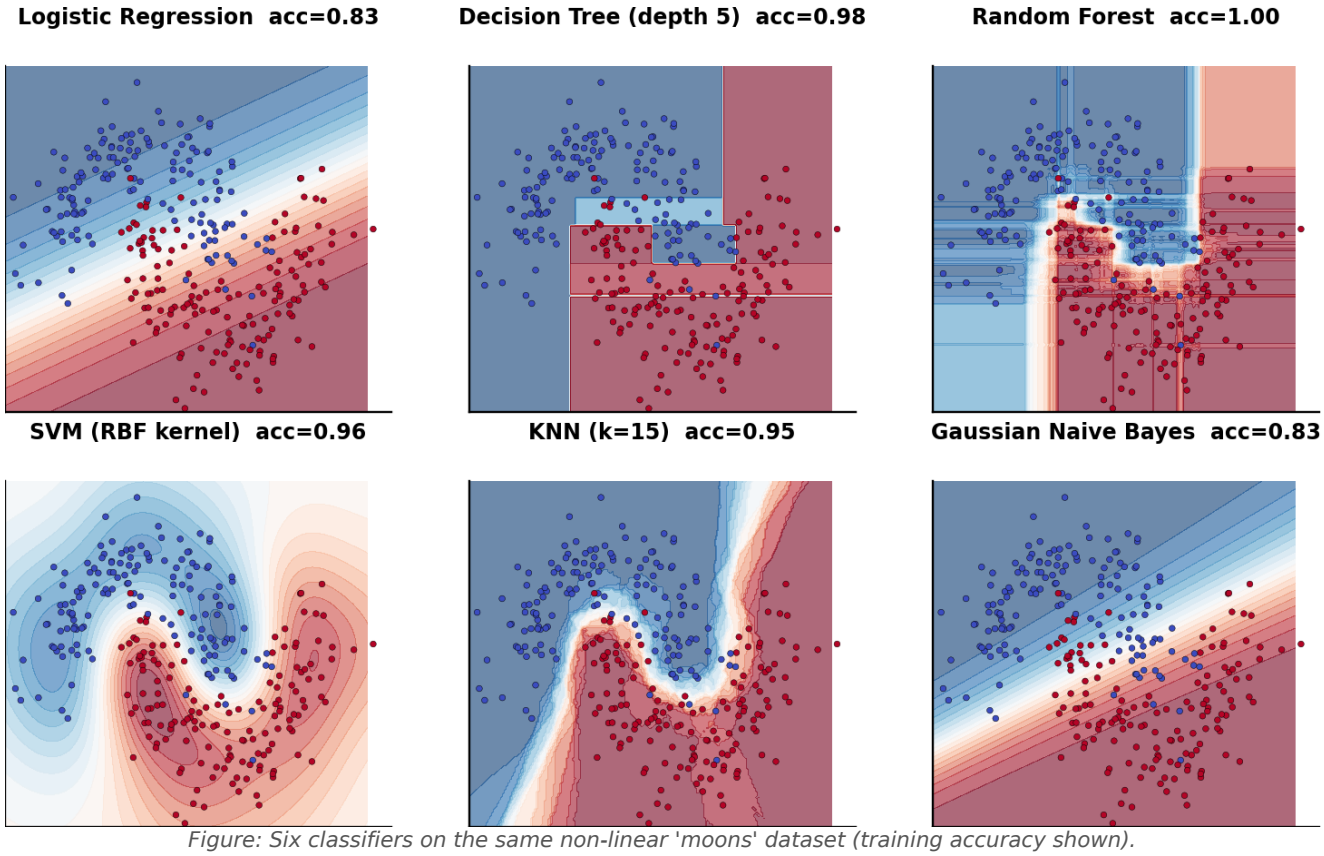


Figure: Six classifiers on the same non-linear 'moons' dataset (training accuracy shown).

Unsupervised learning: clustering & anomaly detection

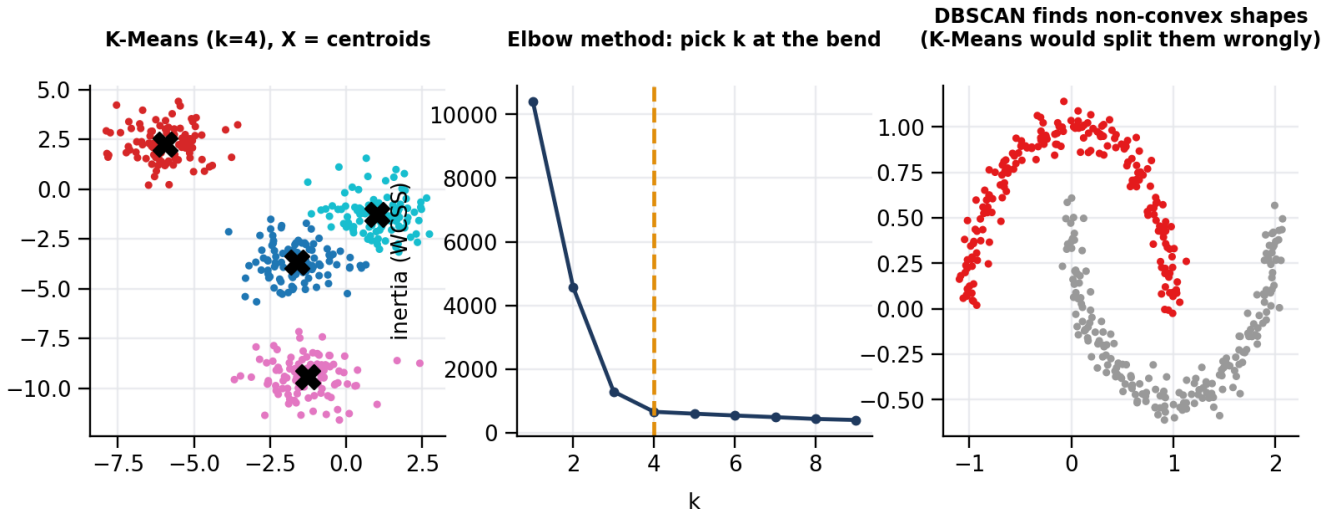


Figure: K-Means, the elbow method, and DBSCAN on non-convex clusters.

Algorithm	How it works	Pros / Cons
K-Means	init k centroids (k-means++) → assign each point to nearest centroid → recompute centroids as means → repeat until stable. Minimises WCSS = $\Sigma\|x - \mu_c\|^2$.	fast, simple / must choose k, assumes spherical equal-size clusters, sensitive to scale & outliers

Algorithm	How it works	Pros / Cons
Hierarchical (agglomerative)	start with each point as a cluster; repeatedly merge the closest pair (single, complete, average, Ward linkage); cut the dendrogram	no k upfront, dendrogram insight / $O(n^2)$ memory
DBSCAN	core points have ≥ min_samples within eps; clusters grow from core points; others are noise	arbitrary shapes, finds outliers / sensitive to eps, varying densities
Gaussian Mixture	soft clustering with EM algorithm; each cluster a Gaussian	probabilistic, elliptical / needs k
Isolation Forest	anomalies are isolated in fewer random splits	great for fraud/ops anomalies

Choosing k: elbow method, **silhouette score** s = (b − a)/max(a, b) where a = mean intra-cluster distance and b = mean nearest-cluster distance (range −1 to 1, higher is better), Davies-Bouldin index, business interpretability.

When to use which model?

Situation	Good first choice
Tabular data, need strong accuracy	Gradient boosting (XGBoost / LightGBM / CatBoost)
Need interpretability / regulation	Logistic/linear regression, shallow decision tree, + SHAP
Small dataset, many features (text)	Linear SVM, Logistic Regression, Naive Bayes
Quick robust baseline	Random Forest
Images, audio, raw text	Deep learning (CNN, transformers) / pre-trained models
No labels	K-Means, DBSCAN, PCA, Isolation Forest
Very large data, online learning	SGDClassifier, linear models, LightGBM
Similarity / recommendation	KNN / embeddings + vector search

2.3 Model Evaluation & Tuning

Classification metrics

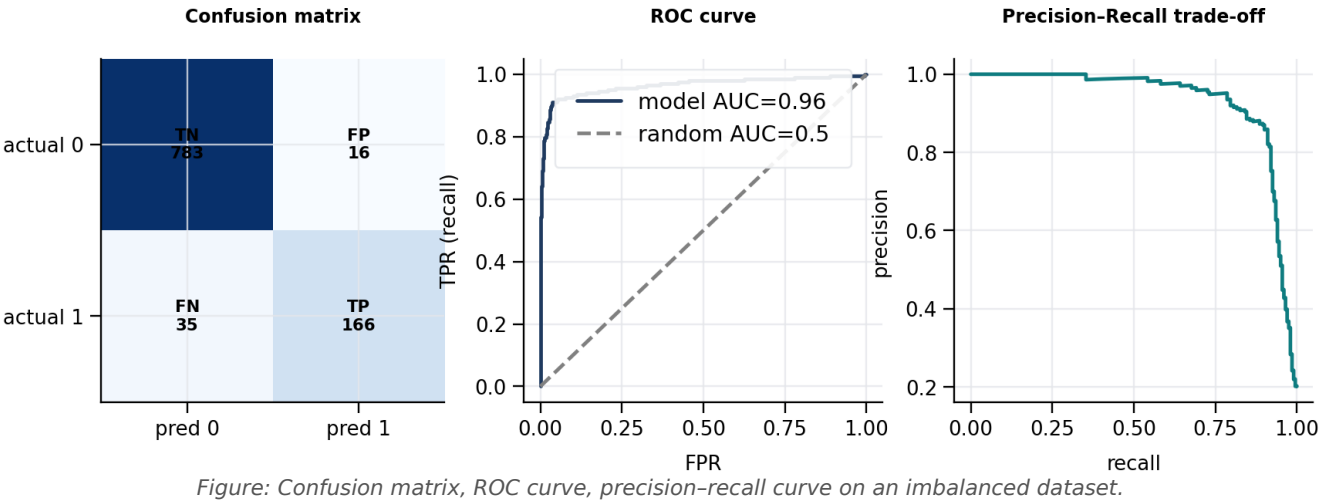


Figure: Confusion matrix, ROC curve, precision–recall curve on an imbalanced dataset.

$$\text{Accuracy} = \frac{TP + TN}{N} \qquad \text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN} \qquad F_1 = \frac{2PR}{P + R}$$

Metric	Answers	Prioritise when
Precision	Of predicted positives, how many are right?	false positives are costly: spam filter, recommending loans, sales calls budget
Recall (sensitivity, TPR)	Of actual positives, how many did we catch?	false negatives are costly: cancer, fraud, security incidents
Specificity (TNR)	Of actual negatives, how many correctly rejected?	medical screening
F1 / Fβ	harmonic balance (β > 1 weights recall)	imbalanced data, single number
ROC-AUC	probability a random positive is ranked above a random negative; threshold-independent	balanced-ish data, ranking quality
PR-AUC (avg precision)	precision–recall over thresholds	highly imbalanced data
Log-loss	quality of probabilities	calibrated probabilities needed
Macro vs micro vs weighted	averaging over classes	macro treats classes equally; micro = global counts

Regression metrics

$$\text{MAE} = \frac{1}{n}\sum|y-\hat{y}| \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum(y-\hat{y})^2} \quad R^2 = 1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2} \quad \text{MAPE} = \frac{100}{n}\sum\left|\frac{y-\hat{y}}{y}\right|$$

RMSE penalises large errors more (sensitive to outliers); MAE is robust and in original units; MAPE is scale-free but explodes near y = 0; adjusted R^2 penalises adding useless features.

Cross-validation

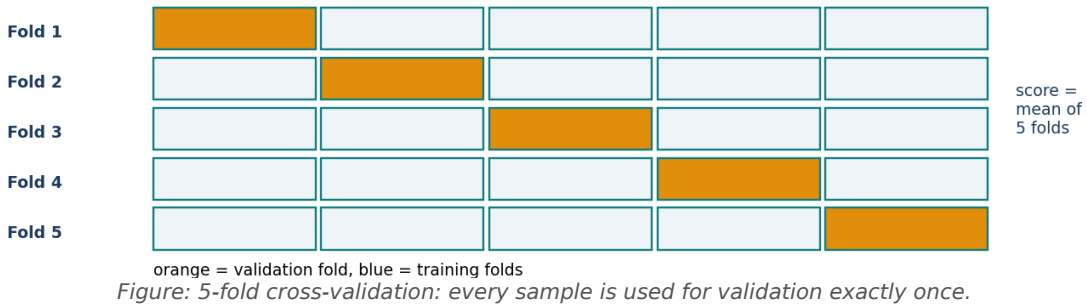


Figure: 5-fold cross-validation: every sample is used for validation exactly once.

- **K-Fold** (k = 5/10), **Stratified K-Fold** (keeps class ratio), **Leave-One-Out** (tiny data), **Group K-Fold** (no user in both train & val), **TimeSeriesSplit** (expanding window, train on past only).
- **Nested CV**: inner loop tunes hyper-parameters, outer loop estimates generalisation — avoids optimistic bias.

Hyper-parameter tuning

Method	How	Notes
Grid Search	try every combination	exhaustive, expensive: GridSearchCV
Random Search	sample combinations randomly	often better than grid for same budget (Bergstra & Bengio)
Bayesian optimisation	surrogate model (Gaussian process/TPE) proposes promising points	Optuna, Hyperopt — efficient

Method	How	Notes
Successive halving / Hyperband	kill bad configs early	HalvingGridSearchCV, Optuna pruners

tuning.py

```
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from scipy.stats import randint, uniform
from xgboost import XGBClassifier
params = {"n_estimators": randint(200, 800), "max_depth": randint(3, 10), "learning_rate": uniform(0.01, 0.2),
          "subsample": uniform(0.6, 0.4), "colsample_bytree": uniform(0.6, 0.4)}
search = RandomizedSearchCV(XGBClassifier(eval_metric="logloss"), params, n_iter=40, scoring="f1",
                            cv=StratifiedKFold(5, shuffle=True, random_state=42), n_jobs=-1)
search.fit(X_train, y_train); print(search.best_params_, search.best_score_)
```

Bias–variance trade-off, overfitting & underfitting

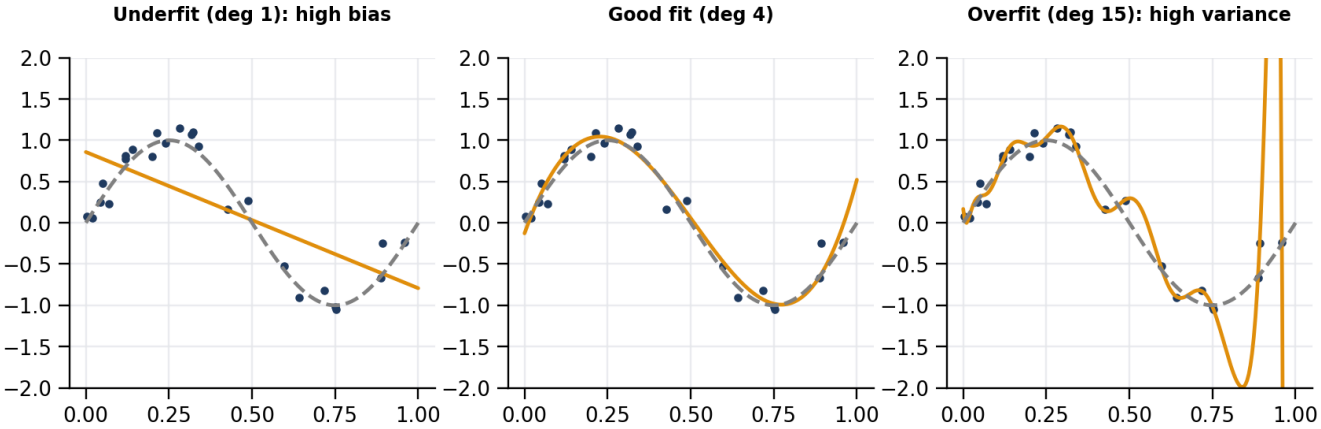


Figure: Polynomial degree 1 underfits, 4 fits well, 15 overfits the noise.

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}[\hat{f}(x)]^2 + \text{Var}[\hat{f}(x)] + \sigma^2_{\text{noise}}$$

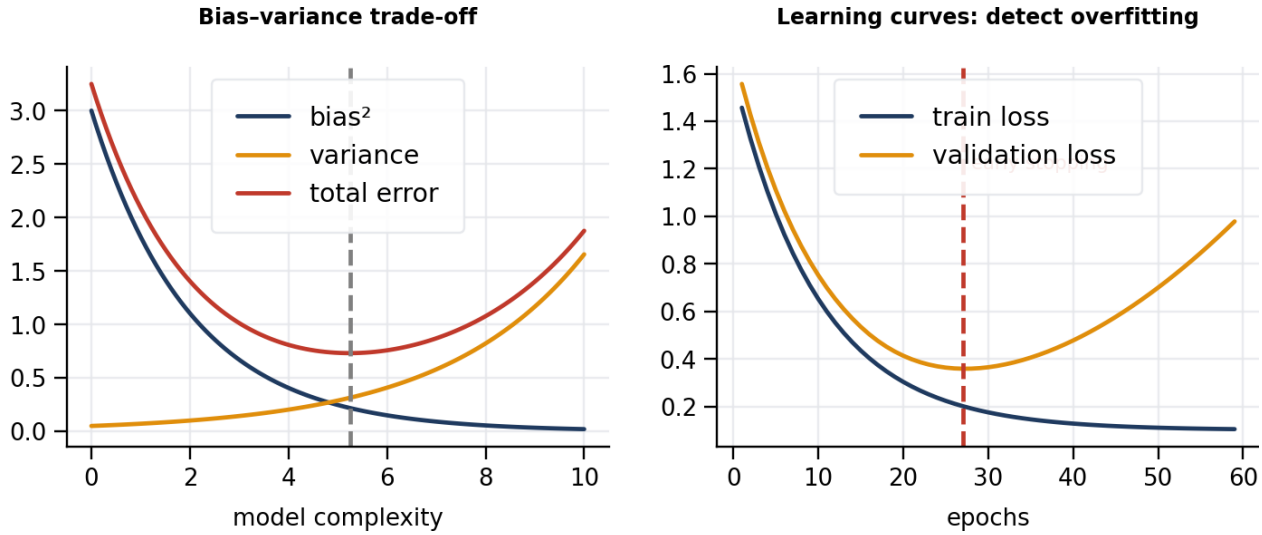


Figure: Total error is minimised at moderate complexity; validation loss reveals overfitting.

	Underfitting (high bias)	Overfitting (high variance)
Symptom	train error high, val error high	train error low, val error much higher
Fixes	more features, more complex model, less regularisation, train longer	more data, augmentation, regularisation (L1/L2, dropout), simpler model, early stopping, ensembling (bagging), feature selection, CV

Practical model-improvement strategies

1. Get a simple baseline first (majority class, linear model).
2. Do error analysis: look at misclassified examples, slice metrics by segment.
3. Improve data (labels, more examples of failure cases) before tuning.
4. Feature engineering with domain knowledge.
5. Try stronger model families (boosting), then tune.
6. Ensemble/stack diverse models.
7. Calibrate probabilities (Platt, isotonic) and tune thresholds to business cost.
8. Check for leakage whenever results look too good.

2.4 Model Persistence & Serving

persist.py

```
import joblib, pickle
joblib.dump(model, "churn_pipeline.joblib")           # preferred for sklearn (efficient with numpy arrays)
model = joblib.load("churn_pipeline.joblib")
with open("model.pkl", "wb") as f: pickle.dump(model, f) # generic Python objects
# Keras: model.save("model.keras")  PyTorch: torch.save(model.state_dict(), "model.pt")
# Portable: ONNX (skl2onnx / torch.onnx.export) → onnxruntime
```

Watch out: Pickle/joblib files can execute arbitrary code when loaded — only load files you trust; pin library versions (the sklearn version that saved must match the one that loads).

ML as a Service with FastAPI

main.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import joblib, pandas as pd

app = FastAPI(title="Churn API", version="1.0")
model = joblib.load("churn_pipeline.joblib")           # load ONCE at startup, not per request

class Customer(BaseModel):
    age: int = Field(ge=18, le=100); income: float; city: str; plan: str

class Prediction(BaseModel):
    churn_probability: float; churn: bool

@app.get("/health")
def health(): return {"status": "ok"}

@app.post("/predict", response_model=Prediction)
def predict(c: Customer):
    try: p = float(model.predict_proba(pd.DataFrame([c.model_dump()]))[0, 1])
    except Exception as e: raise HTTPException(status_code=500, detail=str(e))
    return Prediction(churn_probability=p, churn=p >= 0.5)
# run: uvicorn main:app --reload   → docs at http://127.0.0.1:8000/docs
```

Multi-model comparison project (template)

compare_models.py

```
from sklearn.model_selection import cross_validate
models = {"logreg": LogisticRegression(max_iter=1000), "rf": RandomForestClassifier(300),
          "xgb": XGBClassifier(eval_metric="logloss"), "svm": SVC(probability=True), "knn": KNeighborsClassifier(15
rows = []
for name, m in models.items():
    cv = cross_validate(Pipeline([("pre", pre), ("m", m)]), X, y, cv=5, scoring=["accuracy", "f1", "roc_auc"])
    rows.append({"model": name, **{k: v.mean() for k, v in cv.items() if k.startswith("test_")}})
print(pd.DataFrame(rows).sort_values("test_roc_auc", ascending=False))
```

2.5 Interview Questions — Classical ML

Q1. How do you handle missing data?

First understand why it is missing (MCAR/MAR/MNAR). Drop if tiny and random; impute median/mode or with KNN/iterative imputers; add a missing indicator if missingness is informative; or use models that handle NaN natively like XGBoost. Fit imputers on training data only.

Q2. Standardisation vs normalisation?

Standardisation rescales to mean 0, SD 1 (good default, keeps outliers' relative distance). Min-max normalisation maps to [0, 1] (useful for bounded inputs like pixels, sensitive to outliers).

Q3. Why is logistic regression called regression?

It regresses the log-odds of the positive class linearly on the features; a sigmoid then maps it to a probability and a threshold makes it a classifier.

Q4. Why not use MSE for logistic regression?

MSE with a sigmoid gives a non-convex loss with weak gradients when predictions are confidently wrong; log-loss is convex and is the maximum-likelihood objective for Bernoulli outputs.

Q5. L1 vs L2 regularisation?

L1 adds $|w|$ and drives some weights to exactly zero (feature selection); L2 adds w^2 and shrinks all weights smoothly, handling multicollinearity. Elastic Net combines both.

Q6. Gini vs entropy?

Both measure node impurity; Gini is slightly faster (no log) and they usually give similar trees. Entropy relates to information gain from information theory.

Q7. Bagging vs boosting?

Bagging trains independent models in parallel on bootstrap samples and averages them, reducing variance (Random Forest). Boosting trains models sequentially where each corrects the previous errors, reducing bias (XGBoost).

Q8. Why is Random Forest better than a single tree?

Averaging many decorrelated trees (bootstrap + random feature subsets) dramatically reduces variance while keeping low bias.

Q9. What makes XGBoost fast and accurate?

Second-order gradient optimisation, regularised objective, shrinkage, column/row subsampling, sparsity-aware split finding, histogram/approximate splits and parallel, cache-aware computation.

Q10. Explain the kernel trick.

It computes inner products in a high-dimensional feature space via a kernel function without explicitly transforming the data, letting a linear SVM learn non-linear boundaries.

Q11. How do you choose k in KNN and K-Means?

KNN: cross-validation over k (odd values). K-Means: elbow on inertia, silhouette score and business interpretability.

Q12. Precision vs recall — which for fraud detection?

Usually recall (missing fraud is costly) while keeping precision acceptable to avoid overwhelming investigators; tune the threshold on the PR curve against business costs.

Q13. Why can accuracy be misleading?

With imbalanced classes a trivial majority-class model gets high accuracy. Use precision, recall, F1, PR-AUC, and the confusion matrix.

Q14. What is ROC-AUC?

The area under the TPR-vs-FPR curve across all thresholds; equals the probability that a random positive is scored higher than a random negative. 0.5 = random, 1 = perfect.

Q15. Explain bias-variance trade-off.

Simple models make systematic errors (bias); complex models are sensitive to training noise (variance). Expected error = bias² + variance + irreducible noise, so we pick complexity that minimises the sum.

Q16. How do you detect and fix overfitting?

Large gap between training and validation scores, or learning curves diverging. Fix with more data, regularisation, simpler models, dropout/early stopping, cross-validation and ensembling.

Q17. Grid vs random vs Bayesian search?

Grid tries all combinations (expensive); random samples combinations and explores important parameters better per budget; Bayesian optimisation uses past results to choose promising settings (Optuna).

Q18. PCA vs t-SNE?

PCA is linear, preserves global variance, can transform new data, used for compression; t-SNE is non-linear, preserves local neighbourhoods, used only for visualisation.

Q19. How would you deploy a scikit-learn model?

Save the whole Pipeline with joblib, load it once at startup in a FastAPI app, validate inputs with Pydantic, expose /predict and /health, containerise with Docker and monitor predictions.

CHAPTER 3

Deep Learning: Neural Networks to Generative Models

ANN • Activations • Losses • Backpropagation • Optimizers • Regularisation • CNN & architectures (LeNet → EfficientNet) • Transfer learning • Detection (R-CNN, YOLO, SSD) • Segmentation (U-Net, Mask R-CNN) • RNN, LSTM, GRU • Seq2Seq & attention • GANs • VAEs

3.1 Neural Network Basics

The artificial neuron (perceptron)

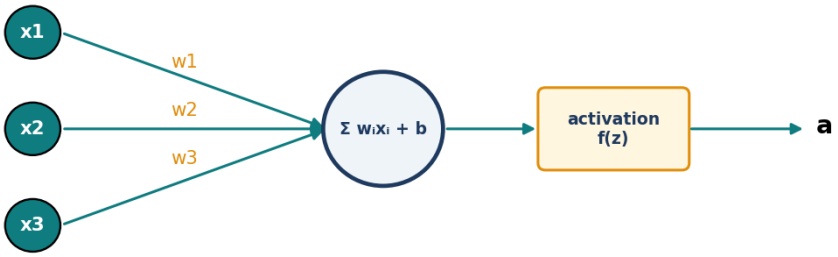


Figure: A neuron computes a weighted sum plus bias, then applies a non-linear activation.

$$z = \sum_{i=1}^{n} w_i x_i + b = w^T x + b \qquad a = f(z)$$

The 1958 perceptron used a step function and can only learn linearly separable problems — it famously cannot learn XOR. Stacking neurons into layers with **non-linear activations** fixes this: by the **Universal Approximation Theorem**, a network with one hidden layer and enough neurons can approximate any continuous function. Depth makes this far more parameter-efficient by learning hierarchical features.

ANN / Multi-Layer Perceptron architecture

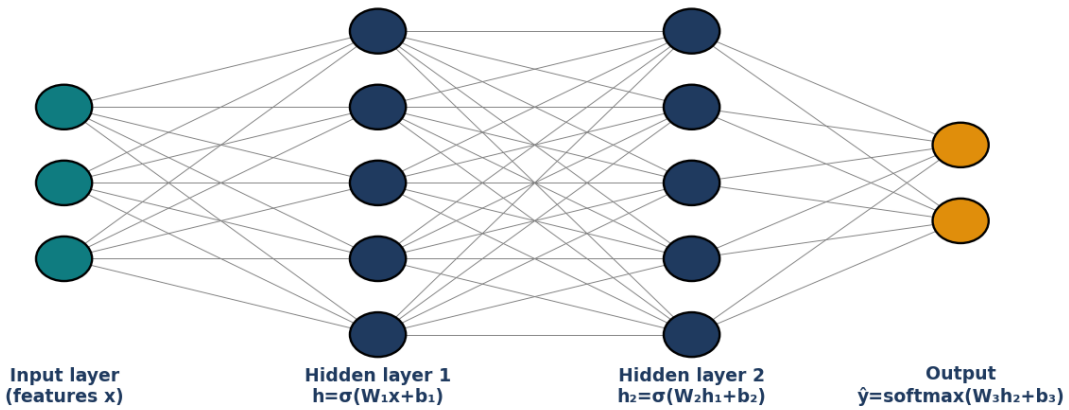


Figure: Fully-connected (dense) network: input → hidden layers → output.

$$h^{(1)} = f(W^{(1)}x + b^{(1)}), \quad h^{(l)} = f(W^{(l)}h^{(l-1)} + b^{(l)}), \quad \hat{y} = g(W^{(L)}h^{(L-1)} + b^{(L)})$$

forward propagation; $W^{(l)}$ has shape (units in layer l) × (units in layer $l-1$)

Parameter count of a dense layer = inputs × units + units (biases). Example: 784 → 128 → 10 has 784·128 + 128 + 128·10 + 10 = 101,770 parameters.

Activation functions

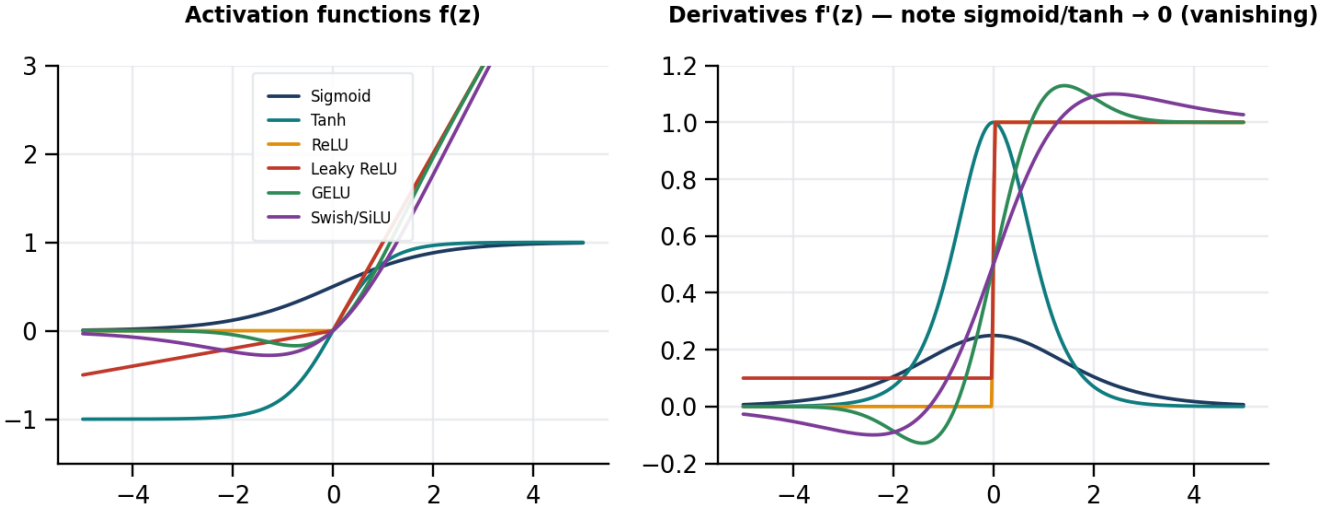


Figure: Activations and their derivatives. Saturating functions (sigmoid/tanh) have near-zero gradients for large |z|.

Activation	Formula	Range	Use / notes
Sigmoid	$1/(1+e^{-z})$	(0,1)	binary output layer; gradient ≤ 0.25 → vanishing; not zero-centred
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	(−1,1)	zero-centred; RNN/LSTM internals; still saturates
ReLU	max(0,z)	[0,∞)	default for hidden layers; cheap; no saturation for z>0; 'dying ReLU' if z always < 0
Leaky ReLU / PReLU	max(αz, z)	(−∞,∞)	fixes dying ReLU (α=0.01 or learned)
ELU / SELU	z if z>0 else $\alpha(e^{z}-1)$	(−α,∞)	smooth, self-normalising (SELU)
GELU	z·Φ(z)	≈(−0.17,∞)	transformers (BERT, GPT)
Swish / SiLU	z·σ(z)	≈(−0.28,∞)	EfficientNet, LLaMA (SwiGLU)
Softmax	$e^{z_k}/\Sigma e^{z_j}$	(0,1), sums to 1	multi-class output layer
Linear	z	(−∞,∞)	regression output

Loss functions

Task	Output layer	Loss	Formula
Regression	linear	MSE / MAE / Huber	$(1/n)\Sigma(y-\hat{y})^2$; Huber = MSE near 0, MAE for large errors
Binary classification	sigmoid	Binary cross-entropy	$-[y \log p + (1-y) \log(1-p)]$
Multi-class (one label)	softmax	Categorical CE (one-hot) / sparse CE (integer labels)	$-\Sigma\, y_k \log p_k$

Task	Output layer	Loss	Formula
Multi-label	sigmoid per class	BCE per class	sum of independent BCEs
Imbalanced detection	sigmoid/softmax	Focal loss	$-\alpha(1-p)^{\gamma}\log p$ (down-weights easy examples)
Embeddings / similarity	embedding	Contrastive / triplet / InfoNCE	max(0, d(a,p) − d(a,n) + margin)
Segmentation	per-pixel sigmoid	Dice / IoU loss + BCE	$1 - 2\lvert A\cap B\rvert/(\lvert A\rvert+\lvert B\rvert)$

Backpropagation — how networks learn

Training loop: **forward pass** (compute predictions and loss) → **backward pass** (compute gradients of the loss w.r.t. every weight using the chain rule, layer by layer from output to input) → **update** (optimizer step). Frameworks do this automatically with **autograd** by recording a computational graph.

$$\delta^{(L)} = \nabla_{\hat{y}} L \odot f'(z^{(L)}), \qquad \delta^{(l)} = (W^{(l+1)T}\delta^{(l+1)}) \odot f'(z^{(l)})$$

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} h^{(l-1)T}, \qquad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}, \qquad W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}$$

Tiny worked example: x = 2, w = 0.5, b = 0, sigmoid output, target y = 1. z = 1, ŷ = σ(1) = 0.731, BCE gradient wrt z = ŷ − y = −0.269, so ∂L/∂w = −0.269·2 = −0.538. With η = 0.1, w ← 0.5 + 0.0538 = 0.554 — the weight increases, pushing ŷ towards 1.

Vanishing & exploding gradients

Gradients are products of many layer derivatives. If each factor < 1 (sigmoid′ ≤ 0.25), the product shrinks exponentially → early layers stop learning (**vanishing**). If factors > 1 they blow up (**exploding**, NaN losses). Fixes: ReLU-family activations, careful initialisation (Xavier/Glorot for tanh: $\mathrm{Var}(w) = 2/(n_{in}+n_{out})$; He for ReLU: $\mathrm{Var}(w) = 2/n_{in}$), batch/layer normalisation, residual connections, gradient clipping, LSTM/GRU gates.

Optimizers

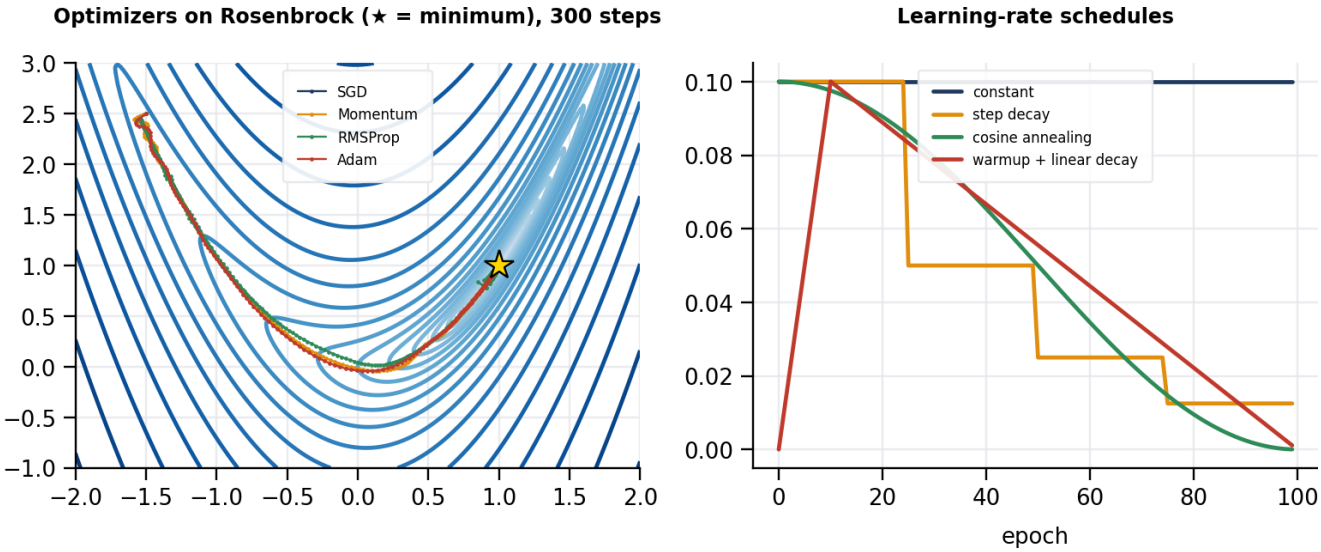


Figure: Optimizer trajectories on the Rosenbrock function, and common learning-rate schedules.

$$\text{SGD}: \ \theta \leftarrow \theta - \eta g \qquad \text{Momentum}: \ v \leftarrow \beta v + g, \ \theta \leftarrow \theta - \eta v$$

$$\text{RMSProp}: \ s \leftarrow \beta s + (1-\beta) g^2, \ \ \theta \leftarrow \theta - \frac{\eta g}{\sqrt{s} + \varepsilon}$$

$$\text{Adam}: \ m \leftarrow \beta_1 m + (1-\beta_1) g, \ \ v \leftarrow \beta_2 v + (1-\beta_2) g^2, \ \ \hat{m} = \frac{m}{1-\beta_1^t}, \ \hat{v} = \frac{v}{1-\beta_2^t}, \ \ \theta \leftarrow \theta - \frac{\eta \hat{m}}{\sqrt{\hat{v}} + \varepsilon}$$

Optimizer	Idea	Notes
Batch GD	gradient on full dataset	stable, slow, memory heavy
SGD	gradient on one sample	noisy, can escape shallow minima
Mini-batch SGD	gradient on batch (32–512)	the standard; GPU-efficient
Momentum / Nesterov	accumulate velocity	dampens oscillations, faster in ravines
AdaGrad	per-parameter LR ∝ 1/√Σg²	good for sparse features; LR decays to zero
RMSProp	exponential average of g²	fixes AdaGrad decay; good for RNNs
Adam	momentum + RMSProp + bias correction	default (β₁=0.9, β₂=0.999, η=1e-3)
AdamW	Adam with decoupled weight decay	default for transformers/LLMs

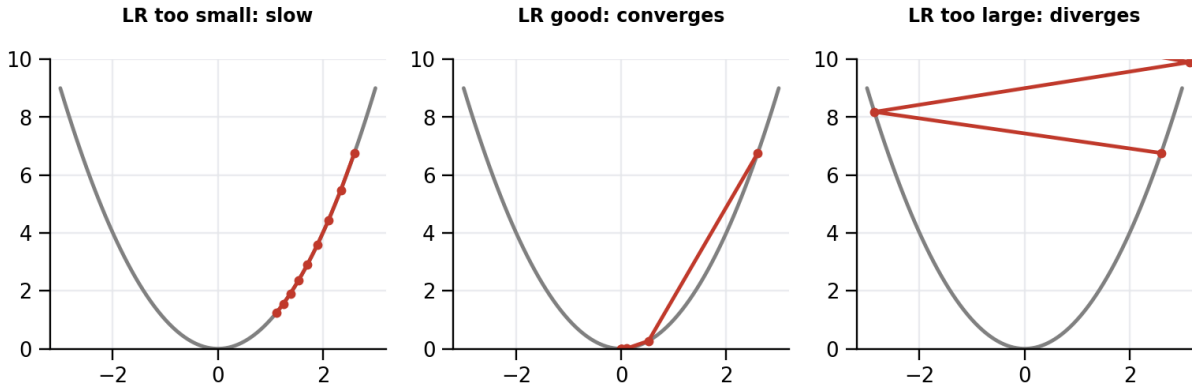


Figure: Effect of the learning rate — the single most important hyper-parameter.

Regularisation & training tricks

Technique	How it works
Dropout	randomly zero a fraction p of activations during training (scale by 1/(1−p)); acts like an ensemble of sub-networks; disabled at inference
L2 / weight decay	penalise large weights
Batch Normalisation	normalise each feature over the batch: $\hat{x} = (x - \mu_B)/\sqrt{(\sigma^2_B + \varepsilon)}$, then $\gamma\hat{x} + \beta$; faster, more stable training; uses running stats at inference
Layer Normalisation	normalise across features of each sample; independent of batch size → used in transformers and RNNs
Early stopping	stop when validation loss stops improving (patience), restore best weights
Data augmentation	flips, crops, rotations, colour jitter, mixup/cutmix; text: back-translation, synonym replacement
Gradient clipping	cap gradient norm (e.g. 1.0) — essential for RNNs and LLMs
LR scheduling / warm-up	start small, then decay (cosine, step, ReduceLROnPlateau)

Hands-on: ANN on MNIST with Keras

mnist_ann.py

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

(x_tr, y_tr), (x_te, y_te) = keras.datasets.mnist.load_data()
x_tr, x_te = x_tr.reshape(-1, 784).astype("float32") / 255, x_te.reshape(-1, 784).astype("float32") / 255

model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(256, activation="relu"), layers.BatchNormalization(), layers.Dropout(0.3),
    layers.Dense(128, activation="relu"), layers.Dropout(0.3),
    layers.Dense(10, activation="softmax")])
model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
cb = [keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True),
      keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=2)]
hist = model.fit(x_tr, y_tr, validation_split=0.1, epochs=30, batch_size=128, callbacks=cb)
print(model.evaluate(x_te, y_te))      # ≈ 0.98 accuracy
model.save("mnist_ann.keras")
```

same model in PyTorch

```
import torch, torch.nn as nn
class MLP(nn.Module):
    def __init__(self): super().__init__(); self.net = nn.Sequential(nn.Flatten(), nn.Linear(784, 256), nn.ReLU(),
    def forward(self, x): return self.net(x)
model, loss_fn = MLP(), nn.CrossEntropyLoss()           # CrossEntropyLoss = log-softmax + NLL (pass raw logits)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
for epoch in range(5):
    model.train()
    for xb, yb in train_loader:
        opt.zero_grad(); loss = loss_fn(model(xb), yb); loss.backward(); opt.step()   # forward → backward → updat
    model.eval()
    with torch.no_grad(): acc = sum((model(x).argmax(1) == y).sum().item() for x, y in val_loader) / len(val_ds)
```

3.2 Convolutional Neural Networks (CNN)

Dense layers on images explode in parameters (224×224×3 = 150k inputs per neuron) and ignore spatial structure. CNNs use three ideas: **local receptive fields** (each neuron looks at a small patch), **weight sharing** (the same filter slides over the whole image → translation equivariance) and **pooling** (downsampling → some translation invariance).

Convolution operation

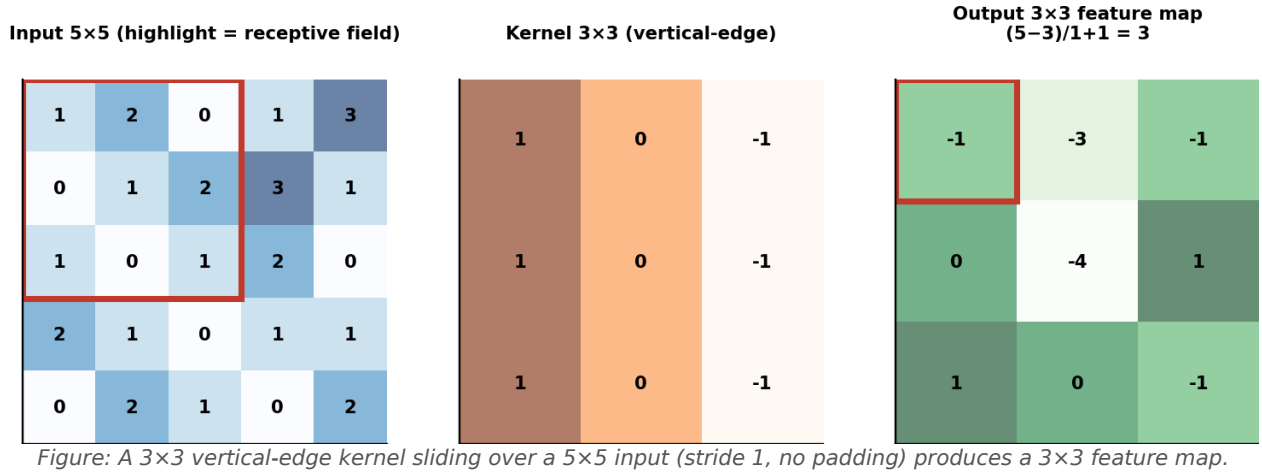


Figure: A 3×3 vertical-edge kernel sliding over a 5×5 input (stride 1, no padding) produces a 3×3 feature map.

$$(I * K)(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n) \qquad O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

W = input size, K = kernel size, P = padding, S = stride. 'same' padding (P = (K−1)/2) keeps the size; 'valid' = no padding. A conv layer with C_{in} input channels and C_{out} filters of size K×K has $(K \cdot K \cdot C_{in} + 1) \cdot C_{out}$ parameters — e.g. 3×3 conv, 64 → 128 channels: (9·64+1)·128 = 73,856.

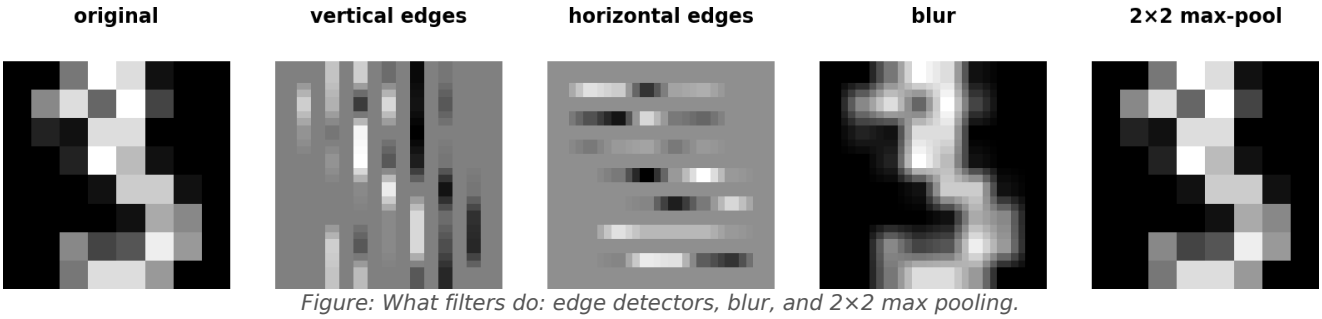


Figure: What filters do: edge detectors, blur, and 2×2 max pooling.

Concept	Meaning
Filter / kernel	small learned weight matrix detecting a pattern
Feature map	output of one filter over the image
Stride	step size of the sliding window; stride 2 halves the size
Padding	zeros around the border to control output size and keep edge info
Max / average pooling	take max/avg of each window (no parameters)
Global Average Pooling	average each feature map to one number → replaces big Flatten+Dense
1×1 convolution	mixes channels / reduces depth cheaply (Inception, ResNet bottlenecks)
Dilated (atrous) conv	spaced-out kernel → larger receptive field (DeepLab)
Depthwise-separable conv	depthwise (per-channel) + pointwise 1×1 → ~8–9× fewer FLOPs (MobileNet)
Transposed conv	learnable upsampling (U-Net decoder, GAN generator)
Receptive field	region of the input that affects one output neuron; grows with depth

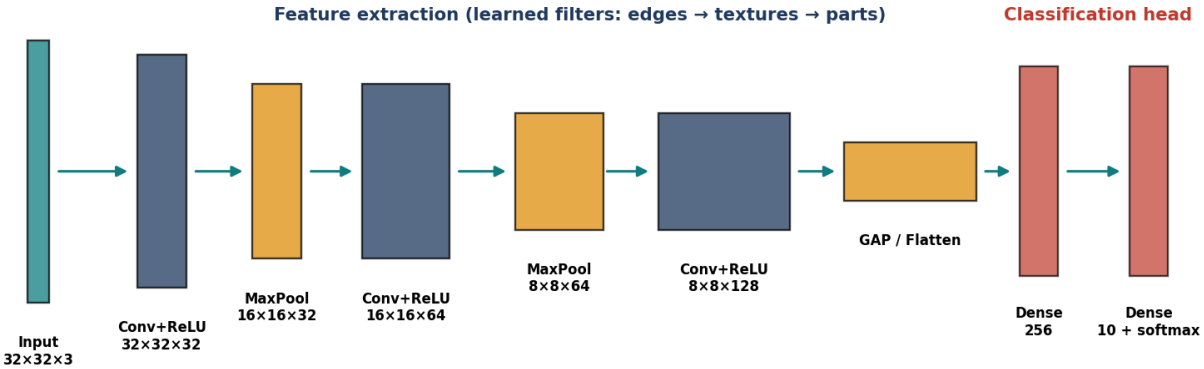


Figure: Typical CNN: conv/pool blocks extract features with increasing depth and decreasing size, then a classifier head.

Feature hierarchy: early layers learn edges and colours, middle layers textures and parts (eyes, wheels), deep layers whole objects.

Classic CNN architectures

Model (year)	Key idea	Depth / params	Why it matters
LeNet-5 (1998)	conv → pool → conv → pool → FC; tanh; digit recognition	5 layers / 60K	first practical CNN (cheques, MNIST)
AlexNet (2012)	ReLU, dropout, data augmentation, GPU training, LRN, 11×11 first conv	8 layers / 60M	won ImageNet by a huge margin → started the DL era
ZFNet (2013)	smaller first filters, deconv visualisation	8 / ~60M	showed how to visualise CNNs
VGG-16/19 (2014)	only 3×3 convs stacked (two 3×3 = one 5×5 receptive field, fewer params, more non-linearity)	16–19 / 138M	simple uniform design; heavy
GoogLeNet / Inception (2014)	Inception module: parallel 1×1, 3×3, 5×5, pool branches; 1×1 bottlenecks; auxiliary classifiers	22 / 6.8M	wide + efficient
ResNet (2015)	residual/skip connections y = F(x) + x; bottleneck blocks	18–152 / 25M (R50)	made 100+ layer networks trainable; default backbone
DenseNet (2017)	each layer receives all previous feature maps (concatenate)	121 / 8M	feature reuse, fewer params
MobileNet v1–v3	depthwise-separable convs, inverted residuals, width multiplier	~3–5M	mobile / edge
EfficientNet (2019)	compound scaling of depth, width, resolution with fixed ratios; MBConv + SE	B0 5M … B7 66M	best accuracy/efficiency trade-off
Vision Transformer (2020)	split image into 16×16 patches → tokens → transformer encoder	86M (ViT-B)	transformers for vision; needs big data/pre-training
ConvNeXt (2022)	modernised ResNet borrowing ViT design choices	~28M+	CNNs competitive with ViTs

ResNet — residual learning

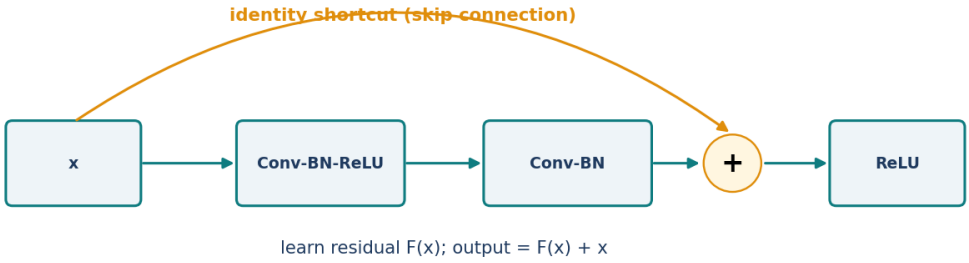


Figure: Residual block: the layers learn the residual F(x); the shortcut carries x unchanged.

$$y = F(x, \{W_i\}) + x \qquad \frac{\partial y}{\partial x} = \frac{\partial F}{\partial x} + I$$

the identity term gives gradients a direct highway to early layers

Problem: plain deep networks showed **degradation** — a 56-layer net had higher *training* error than a 20-layer net (not overfitting, an optimisation problem). If extra layers only need to learn identity, F(x) = 0 is easy. ResNet-50+ uses a **bottleneck** 1×1 (reduce) → 3×3 → 1×1 (expand).

Inception module

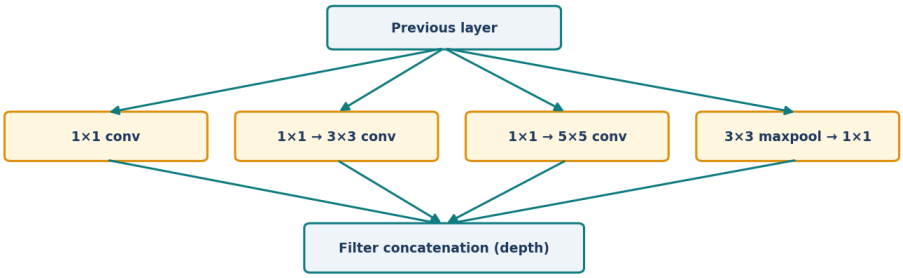


Figure: Inception: multi-scale parallel convolutions concatenated along depth; 1×1 convs reduce cost.

EfficientNet compound scaling

$$\text{depth} = \alpha^{\phi}, \quad \text{width} = \beta^{\phi}, \quad \text{resolution} = \gamma^{\phi}, \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

Transfer learning

Reuse a model pre-trained on a large dataset (ImageNet) for your task. Early layers learn general features (edges, textures) that transfer well. Two strategies:

Strategy	How	When
Feature extraction	freeze the backbone, train only a new head	small dataset similar to ImageNet
Fine-tuning	after training the head, unfreeze top layers (or all) and train with a low LR (1e-5)	more data or different domain
Train from scratch	random init	huge dataset, very different domain (medical, satellite)

transfer_learning.py

```
base = keras.applications.EfficientNetB0(include_top=False, weights="imagenet", input_shape=(224, 224, 3))
base.trainable = False                                          # 1) feature extraction
inp = keras.Input((224, 224, 3))
x = keras.layers.RandomFlip("horizontal")(inp); x = keras.layers.RandomRotation(0.1)(x)
x = keras.applications.efficientnet.preprocess_input(x)
x = base(x, training=False)                                     # keep BatchNorm in inference mode
x = keras.layers.GlobalAveragePooling2D()(x); x = keras.layers.Dropout(0.3)(x)
out = keras.layers.Dense(num_classes, activation="softmax")(x)
model = keras.Model(inp, out); model.compile("adam", "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=5)
base.trainable = True                                           # 2) fine-tune top layers
for layer in base.layers[:-30]: layer.trainable = False
model.compile(keras.optimizers.Adam(1e-5), "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=5)
```

Object detection

Classification says *what*; localisation adds *one box*; detection finds *all objects with boxes and classes*. Output per object: (class, confidence, x, y, w, h).

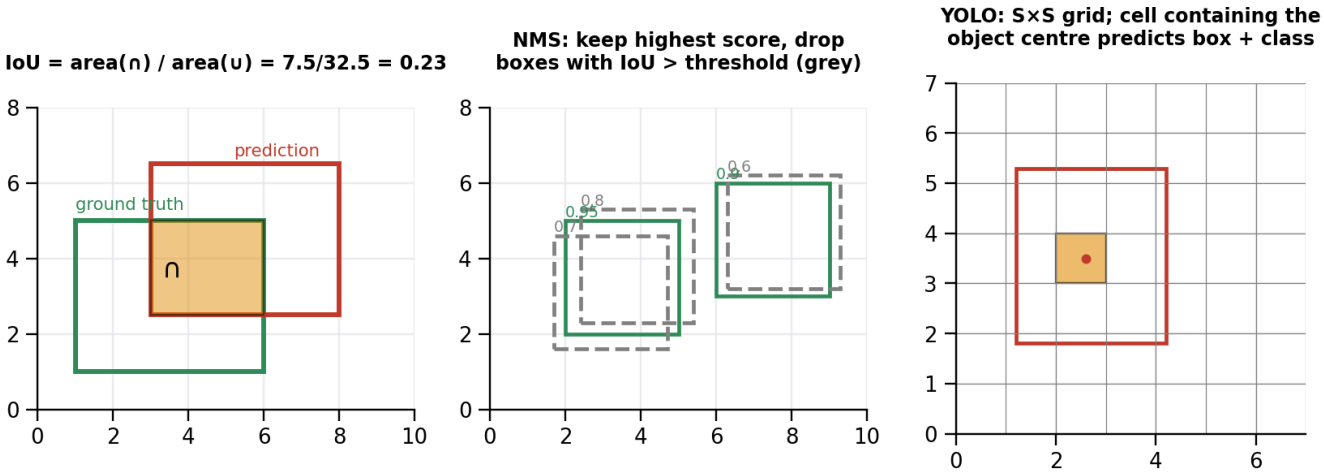


Figure: IoU, Non-Max Suppression, and the YOLO grid idea.

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} \qquad \text{mAP} = \frac{1}{C}\sum_{c=1}^{C} AP_c, \quad AP = \int_0^1 p(r)\,dr$$

A prediction is a true positive if IoU ≥ threshold (0.5). **mAP@0.5** (PASCAL VOC) or **mAP@[.5:.95]** (COCO, averaged over IoU thresholds). **Anchor boxes** are predefined box shapes the network refines with offsets. **NMS** removes duplicate boxes.

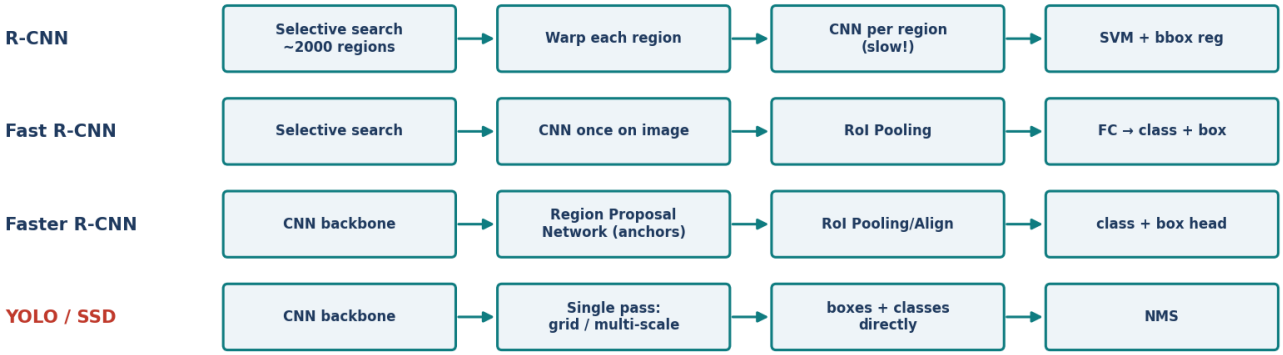


Figure: Evolution from two-stage (R-CNN family) to single-stage detectors.

Detector	Type	Key ideas	Speed
R-CNN (2014)	two-stage	selective search proposals → CNN features per region → SVM	~50 s/image
Fast R-CNN	two-stage	CNN once per image, RoI pooling, joint classification + box regression	~2 s
Faster R-CNN	two-stage	Region Proposal Network with anchors replaces selective search; end-to-end	~5–7 FPS, high accuracy
SSD (2016)	one-stage	default boxes at multiple feature-map scales	~45–60 FPS
YOLO v1 → v8+/v11	one-stage	grid cells predict boxes + class probs in one pass; later: anchors, FPN/PAN necks, anchor-free heads, CSP backbones	real-time (30–150+ FPS)
RetinaNet	one-stage	focal loss fixes foreground/background imbalance; FPN	accurate one-stage
DETR / RT-DETR	transformer	set prediction with Hungarian matching; no anchors or NMS	end-to-end

YOLO loss combines box coordinate loss (MSE/IoU-based such as CIoU), objectness confidence loss and classification loss. **Feature Pyramid Network (FPN)** merges deep semantic features with

high-resolution shallow features to detect small and large objects.

Image segmentation

Type	Output	Models
Semantic	class per pixel (all cars same label)	FCN, U-Net, DeepLab (atrous conv + ASPP), SegFormer
Instance	separate mask per object instance	Mask R-CNN
Panoptic	semantic + instance together	Panoptic FPN, Mask2Former
Promptable	mask for any prompt (point/box)	Segment Anything (SAM)

U-Net

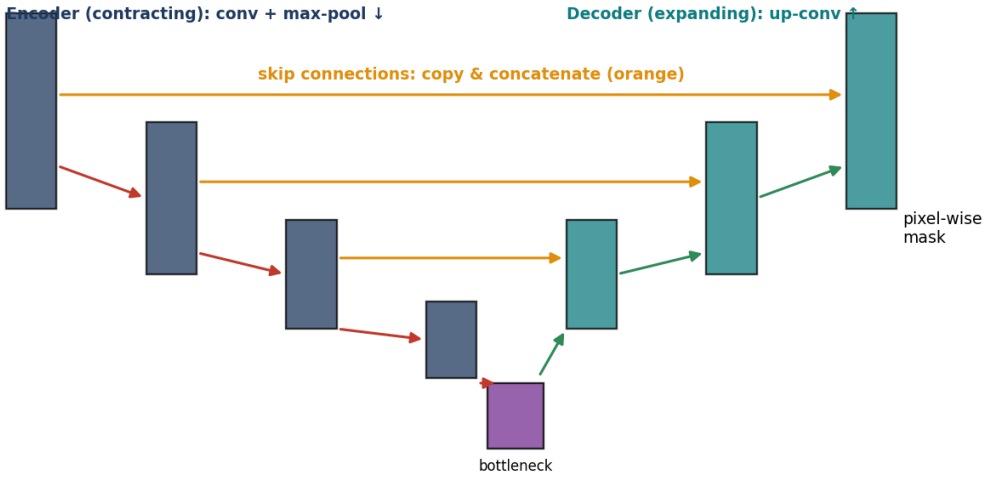


Figure: U-Net: contracting encoder, expanding decoder, and skip connections that restore fine spatial detail.

Designed for biomedical images with few training samples. The **encoder** captures context (what), the **decoder** upsamples with transposed convolutions to recover resolution (where), and **skip connections** concatenate encoder feature maps into the decoder so boundaries stay sharp. Output: per-pixel class map; loss: BCE + Dice. U-Net is also the denoising backbone of Stable Diffusion.

Mask R-CNN

Faster R-CNN + a third parallel branch: a small FCN predicting a binary mask (28×28) for each RoI. Replaces RoI Pooling with **RoI Align** (bilinear interpolation, no quantisation) for pixel-accurate masks. Loss = $L_{cls} + L_{box} + L_{mask}$.

CNN visualisation techniques

- **Filter visualisation**: plot first-layer kernels (edges, colours).
- **Feature-map / activation visualisation**: see what each layer responds to.
- **Saliency maps**: gradient of class score w.r.t. input pixels.
- **Grad-CAM**: weight final conv feature maps by the average gradient of the class score → heat-map of where the model looked. $L = \mathrm{ReLU}(\Sigma_k \alpha_k A^k)$, α_k = mean of $\partial y^c / \partial A^k$.
- **Occlusion sensitivity**: slide a grey patch and watch the score drop.
- **t-SNE of embeddings** to see class separation.

End-to-end computer vision project



3.3 Recurrent Neural Networks (RNN)

Sequences (text, speech, time series, logs) have order and variable length. An RNN processes one element at a time and keeps a **hidden state** h_t — a memory of everything seen so far — using the **same weights at every step**.

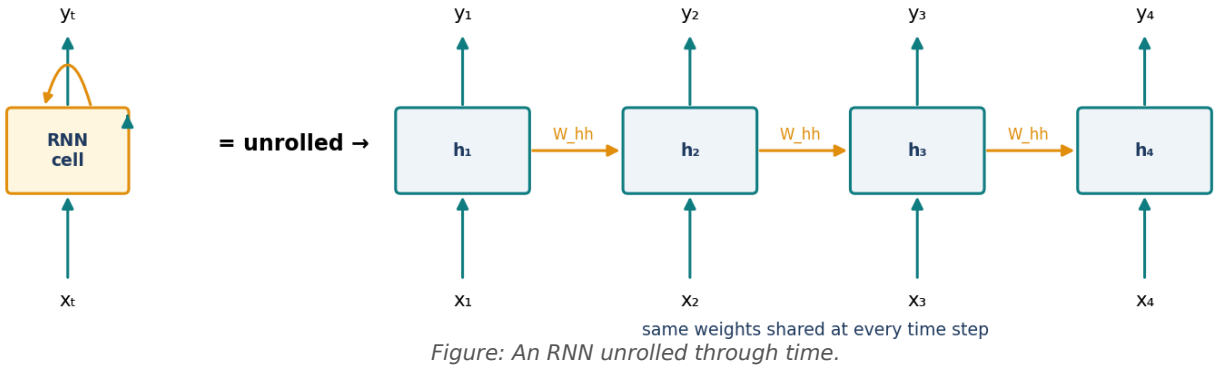


Figure: An RNN unrolled through time.

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \qquad \hat{y}_t = \text{softmax}(W_{hy}h_t + b_y)$$

Pattern	Example
One-to-one	image classification (not really sequential)
One-to-many	image captioning, music generation
Many-to-one	sentiment classification, anomaly in log sequence
Many-to-many (aligned)	NER / POS tagging (label per token)
Many-to-many (seq2seq)	translation, summarisation

Backpropagation Through Time & the vanishing gradient problem

$$\frac{\partial L_t}{\partial h_k} = \frac{\partial L_t}{\partial h_t} \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial L_t}{\partial h_t} \prod_{j=k+1}^{t} W_{hh}^T \,\text{diag}(1 - h_j^2)$$

BPTT unrolls the network over T steps and backpropagates through all of them. The gradient contains a **product of T Jacobians**: if the largest singular value of W_{hh} (times $\tanh' \le 1$) is < 1 the signal decays exponentially → the network cannot learn long-range dependencies ('The *cat*, which … for 50 words …, *was* hungry'). If > 1 it explodes. Fixes: gradient clipping (exploding), truncated BPTT, ReLU + identity init, and above all **gated architectures** (LSTM, GRU).

LSTM — Long Short-Term Memory

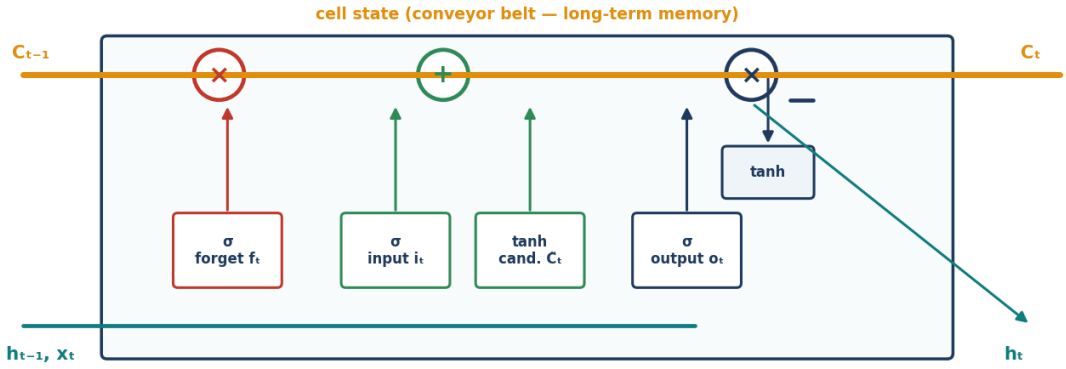


Figure: LSTM cell: the cell state C_t is a conveyor belt modified only by gated element-wise operations.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \qquad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \qquad h_t = o_t \odot \tanh(C_t)$$

Gate	Question it answers
Forget gate f_t	What fraction of old memory to keep? (0 = erase, 1 = keep)
Input gate i_t + candidate $\tilde{C}_t$	What new information to write?
Output gate o_t	What part of memory to expose as the hidden state?

Why it fixes vanishing gradients: the cell-state update is **additive** ($C_t = f_t \odot C_{t-1} + \ldots$), so $\partial C_t / \partial C_{t-1} = f_t$; when the forget gate is near 1, gradients flow across many steps almost unchanged (the 'constant error carousel'). An LSTM layer has $4 \cdot (h \cdot (h + x) + h)$ parameters.

GRU — Gated Recurrent Unit

$$z_t = \sigma(W_z[h_{t-1}, x_t]) \quad r_t = \sigma(W_r[h_{t-1}, x_t]) \quad \tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t]) \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

GRU merges forget and input gates into an **update gate** z and adds a **reset gate** r; no separate cell state. ~25% fewer parameters, trains faster, similar accuracy on many tasks. LSTM may win on very long or complex sequences.

Bidirectional RNN / LSTM

Two RNNs: one reads left→right, one right→left; outputs are concatenated [$\overrightarrow{h}_t$; $\overleftarrow{h}_t$]. Each position sees full past and future context — great for tagging and classification (NER, sentiment), **not** for real-time generation/forecasting (future is unknown). Stacked (deep) RNNs add layers vertically.

Sequence-to-sequence models & attention

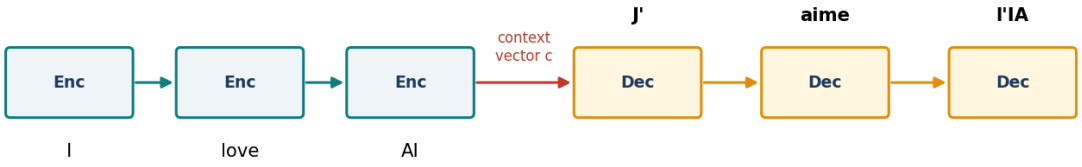


Figure: Encoder–decoder: the encoder compresses the input; the decoder generates output token by token.

The encoder reads the source and produces a context vector; the decoder (initialised with it) generates the target autoregressively, feeding each predicted token back in. **Teacher forcing** during training feeds the ground-truth previous token instead. Decoding uses greedy or **beam search**. Problem: one fixed vector cannot hold a long sentence. **Attention** (Bahdanau 2014) lets the decoder look at *all* encoder states at every step:

$$e_{t,i} = \text{score}(s_{t-1}, h_i), \qquad \alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}, \qquad c_t = \sum_i \alpha_{t,i} h_i$$

Attention type	Score function
Bahdanau (additive)	$v^T \tanh(W_1 s + W_2 h)$
Luong (dot / general)	$s^T h$ or $s^T W h$
Scaled dot-product (Transformer)	$q^T k / \sqrt{d_k}$

Attention weights are interpretable (alignment between source and target words) and directly led to the Transformer — 'attention is all you need' removed recurrence entirely.

Hands-on: time-series forecasting with LSTM

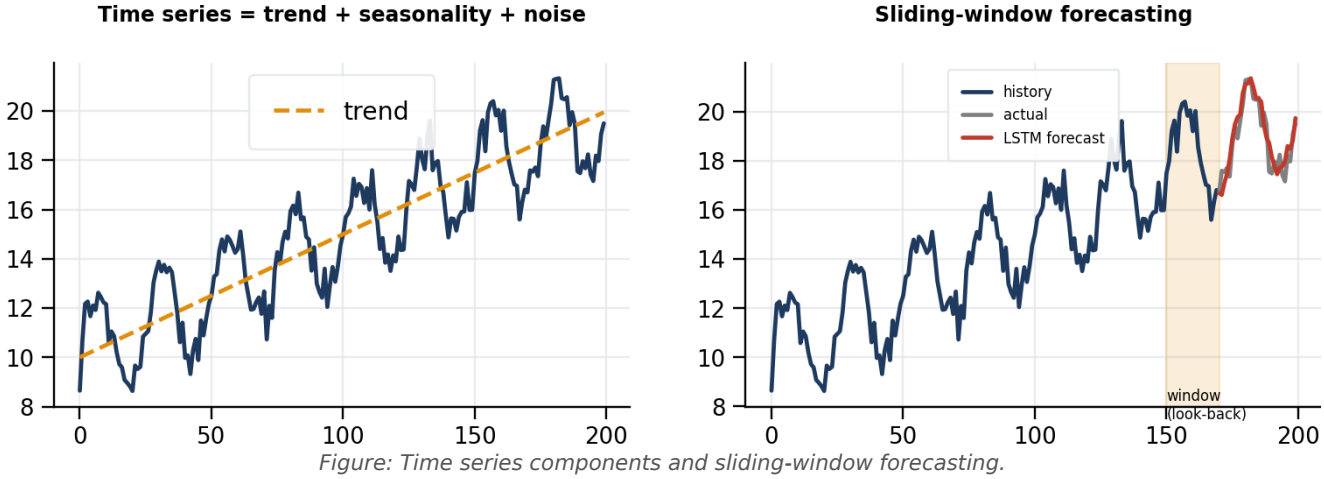


Figure: Time series components and sliding-window forecasting.

lstm_forecast.py

```
import numpy as np
from tensorflow import keras
def make_windows(series, look_back=30, horizon=1):
    X, y = [], []
    for i in range(len(series) - look_back - horizon + 1):
        X.append(series[i:i + look_back]); y.append(series[i + look_back + horizon - 1])
    return np.array(X)[..., None], np.array(y)            # (samples, timesteps, features=1)

scaled = (series - series[:train_end].mean()) / series[:train_end].std()   # scale with TRAIN stats only
X, y = make_windows(scaled)
split = int(0.8 * len(X)); X_tr, X_te, y_tr, y_te = X[:split], X[split:], y[:split], y[split:]  # no shuffling!
model = keras.Sequential([keras.Input((30, 1)), keras.layers.LSTM(64, return_sequences=True),
                          keras.layers.LSTM(32), keras.layers.Dense(1)])
model.compile("adam", "mse", metrics=["mae"])
model.fit(X_tr, y_tr, validation_split=0.1, epochs=50, batch_size=32, shuffle=False,
          callbacks=[keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)])
```

Always compare against naive baselines (last value, seasonal naive, moving average) and classical models (ARIMA/SARIMA, Prophet, gradient boosting on lag features). Metrics: MAE, RMSE, MAPE/sMAPE. Modern alternatives: Temporal CNNs, N-BEATS, Temporal Fusion Transformer, foundation models for time series.

3.4 Generative Models: GANs & VAEs

Discriminative models learn P(y|x) (classify). **Generative** models learn the data distribution P(x) (or P(x|condition)) so they can **sample new data**. Families: autoregressive (GPT, PixelCNN), VAEs, GANs, normalising flows, diffusion models.

GAN fundamentals

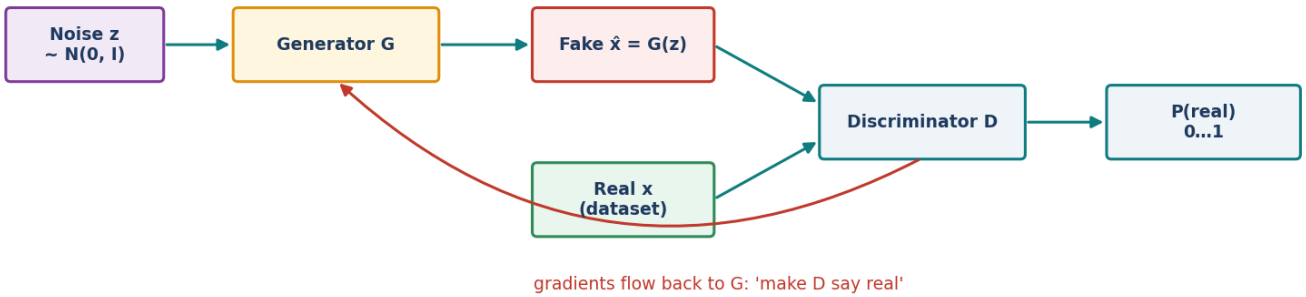


Figure: GAN: the generator tries to fool the discriminator; the discriminator tries to tell real from fake.

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]$$

1. Sample real batch x and noise z; generate fake G(z).
2. **Train D**: maximise log D(x) + log(1 − D(G(z))) — label real = 1, fake = 0 (BCE).
3. **Train G**: in practice maximise log D(G(z)) (the 'non-saturating' loss — stronger gradients early) — label fakes as real and backprop through D into G (D frozen).
4. Alternate. At the theoretical optimum $p_g = p_{data}$ and D(x) = 0.5 everywhere (the game minimises Jensen-Shannon divergence).

Problem	Description	Fixes
Mode collapse	G produces only a few kinds of outputs	minibatch discrimination, WGAN, unrolled GAN, diversity terms

Problem	Description	Fixes
Training instability / non-convergence	the two-player game oscillates	WGAN-GP, spectral norm, TTUR (different LRs), label smoothing
Vanishing gradients for G	D becomes too good	non-saturating loss, Wasserstein loss
Evaluation	no likelihood	FID (Fréchet Inception Distance, lower is better), Inception Score, human eval

Advanced GAN variants

Variant	Key idea	Use
DCGAN	conv/transposed-conv architecture, BatchNorm, LeakyReLU in D, no pooling	stable image GAN baseline
Conditional GAN (cGAN)	feed a label/condition y to both G and D: G(z, y)	generate a specific class
WGAN / WGAN-GP	Wasserstein (earth-mover) distance; critic outputs a score; Lipschitz via weight clipping or gradient penalty	stable training, meaningful loss curve
Pix2Pix	cGAN with paired images; U-Net generator + PatchGAN discriminator + L1 loss	sketch→photo, map→satellite
CycleGAN	unpaired translation with two generators and cycle-consistency loss $\|F(G(x)) - x\|$	horse↔zebra, summer↔winter
StyleGAN 1/2/3	mapping network z→w, style injection via AdaIN per layer, progressive growing, noise inputs	photorealistic faces, style mixing
SRGAN / ESRGAN	perceptual + adversarial loss	super-resolution
BigGAN	large-scale class-conditional, truncation trick	high-fidelity ImageNet

Variational Autoencoders (VAE)

A plain **autoencoder** compresses x → z → x̂ and is used for dimensionality reduction, denoising and anomaly detection (high reconstruction error = anomaly), but its latent space is irregular so you cannot sample from it. A **VAE** makes the encoder output a **distribution** q(z|x) = N(μ, σ²) and regularises it towards a prior N(0, I), producing a smooth, continuous latent space you can sample and interpolate in.

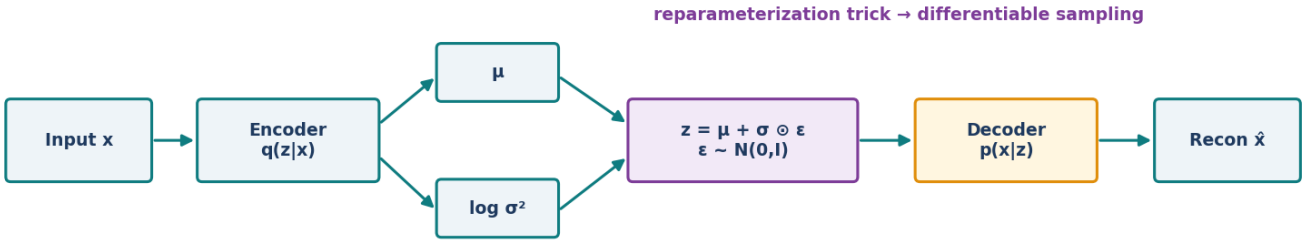


Figure: VAE with the reparameterization trick.

$$\mathcal{L}_{ELBO} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \,\|\, \mathcal{N}(0, I))$$

$$D_{KL} = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right) \qquad z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

Loss = reconstruction loss (MSE/BCE) + KL term. The **reparameterization trick** moves randomness into ε so gradients can flow through μ and σ. β-VAE weights the KL term by β > 1 for more disentangled factors. VAE samples tend to be blurrier than GANs, but VAEs are stable and give a likelihood bound. VQ-VAE uses a discrete codebook (used in DALL-E 1 and audio models); the VAE in Stable Diffusion compresses images into latents.

Comparison with modern GenAI

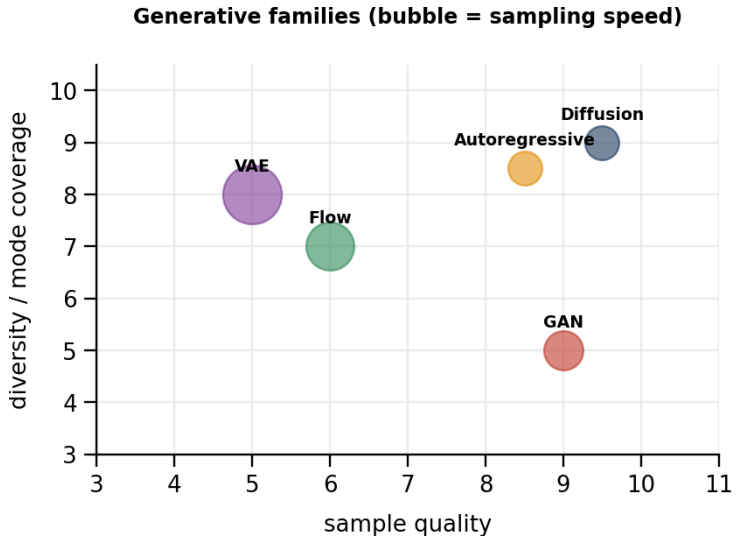


Figure: Qualitative trade-offs between generative families.

	GAN	VAE	Diffusion	Autoregressive (GPT)
Training	adversarial, unstable	stable (ELBO)	stable (MSE on noise)	stable (cross-entropy)
Sample quality	sharp	blurry	state of the art	excellent for text/code
Diversity	mode collapse risk	good	excellent	excellent
Sampling speed	one pass (fast)	fast	many steps (slow; improved by DDIM/distillation)	token by token
Today's use	super-resolution, real-time, StyleGAN	latent compression inside diffusion	images, video, audio (SD, DALL-E 3, Imagen, Sora)	LLMs, multimodal

3.5 Interview Questions — Deep Learning

Q1. Why do we need non-linear activation functions?

Without them, stacked linear layers collapse into a single linear transformation, so the network could only learn linear boundaries regardless of depth.

Q2. Why is ReLU preferred over sigmoid in hidden layers?

It does not saturate for positive inputs (gradient 1), so it avoids vanishing gradients, is cheap to compute and induces sparsity. Its weakness is dying ReLU, addressed by Leaky ReLU/GELU.

Q3. Explain backpropagation in simple terms.

After the forward pass computes the loss, we apply the chain rule backwards layer by layer to find how much each weight contributed to the error, then update every weight against its gradient.

Q4. What is the vanishing gradient problem and how do you fix it?

Gradients shrink exponentially as they are multiplied through many layers or time steps, so early layers learn very slowly. Fixes: ReLU, He/Xavier init, batch/layer norm, residual connections, LSTM/GRU gates, shorter paths.

Q5. SGD vs Adam?

SGD uses one global learning rate (with momentum often generalises very well, e.g. vision). Adam adapts per-parameter rates using first and second moment estimates with bias correction, converging faster with little tuning; AdamW is standard for transformers.

Q6. Batch norm vs layer norm?

Batch norm normalises each feature across the batch (depends on batch size, great for CNNs). Layer norm normalises across features within one sample (batch-independent, used in RNNs and transformers).

Q7. How does dropout prevent overfitting?

Randomly dropping neurons stops co-adaptation, forcing redundant robust features; it approximates averaging an ensemble of thinned networks. It is turned off at inference.

Q8. Why CNN over a dense network for images?

Local connectivity and weight sharing drastically reduce parameters, exploit spatial structure and give translation equivariance; pooling adds some invariance.

Q9. Calculate output size of conv layer: 32×32 input, 5×5 kernel, stride 1, padding 2.

$(32 - 5 + 2 \cdot 2)/1 + 1 = 32 \rightarrow 32 \times 32$ ('same').

Q10. What problem did ResNet solve?

The degradation problem: very deep plain networks were harder to optimise. Skip connections let layers learn residuals and give gradients an identity path, enabling 100+ layer networks.

Q11. Why use 1×1 convolutions?

To change channel depth cheaply (dimensionality reduction/bottleneck), add non-linearity, and mix information across channels.

Q12. Feature extraction vs fine-tuning?

Feature extraction freezes the pre-trained backbone and trains a new head; fine-tuning additionally unfreezes some layers and trains them with a small learning rate to adapt features to the new domain.

Q13. One-stage vs two-stage detectors?

Two-stage (Faster R-CNN) first proposes regions then classifies them — more accurate, slower. One-stage (YOLO, SSD) predicts boxes and classes in a single pass — real-time.

Q14. What is NMS and IoU?

IoU measures overlap of two boxes (intersection over union). NMS keeps the highest-confidence box and removes others overlapping it above an IoU threshold, eliminating duplicates.

Q15. Why does U-Net have skip connections?

Downsampling loses spatial detail; concatenating high-resolution encoder features into the decoder restores precise localisation for pixel-level masks.

Q16. Semantic vs instance segmentation?

Semantic labels every pixel by class without separating objects; instance segmentation separates each object instance (Mask R-CNN).

Q17. Explain LSTM gates.

Forget gate decides what to erase from cell state, input gate what new information to add, output gate what to expose as hidden state. The additive cell-state update keeps gradients flowing over long sequences.

Q18. LSTM vs GRU?

GRU has two gates (update, reset) and no separate cell state — fewer parameters, faster; LSTM has three gates and a cell state — sometimes better on longer, complex dependencies.

Q19. What is teacher forcing?

During seq2seq training, feeding the ground-truth previous token into the decoder instead of its own prediction, which speeds and stabilises training but causes exposure bias at inference.

Q20. Why did attention replace the fixed context vector?

A single vector bottlenecks long inputs; attention computes a fresh weighted sum of all encoder states for every output step, focusing on relevant parts.

Q21. Explain GAN training and mode collapse.

Generator and discriminator play a minimax game; G learns to produce samples D cannot distinguish from real. Mode collapse is when G produces limited variety that still fools D; WGAN-GP and minibatch discrimination help.

Q22. Autoencoder vs VAE?

An autoencoder maps input to a deterministic code and is used for compression/anomaly detection. A VAE learns a probabilistic latent distribution regularised to N(0,I) with a KL term, allowing sampling of new data.

Q23. What is the reparameterization trick?

Writing $z = \mu + \sigma \cdot \varepsilon$ with $\varepsilon \sim N(0, I)$ so sampling becomes a deterministic, differentiable function of μ and σ, letting gradients flow through the encoder.

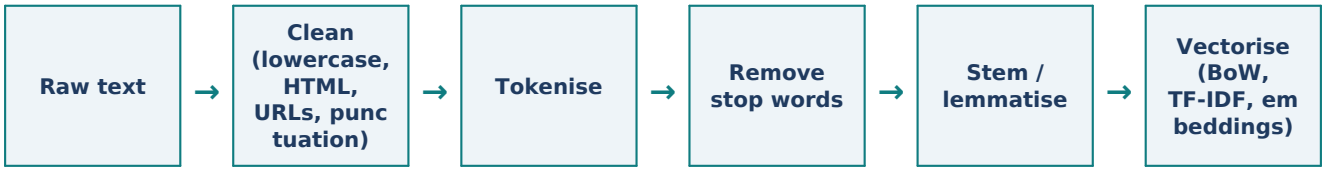
CHAPTER 4

NLP & Transformers: From Text Processing to LLMs

Text preprocessing • BoW, TF-IDF, n-grams • Word2Vec, GloVe, FastText • Sequence models recap • Transformer deep dive • BERT & variants • GPT & T5 • Tokenisation • LLM training (pre-training, SFT, RLHF, DPO) • Decoding • Prompting • Fine-tuning (LoRA/QLoRA) • Quantisation • LLM landscape • APIs • Projects

4.1 Classical NLP

Text preprocessing pipeline



Step	What	Example / tool
Tokenisation	split text into units (words, subwords, characters, sentences)	nltk.word_tokenize, spaCy, HF tokenizers
Normalisation	lowercase, unicode normalisation, expand contractions, fix spelling	"Don't" → "do not"
Stop-word removal	drop very common low-information words (the, is, at)	keep them for sentiment ("not")!
Stemming	chop suffixes with rules — fast, may produce non-words	Porter: studies → studi, running → run
Lemmatisation	dictionary + POS based base form — slower, real words	studies → study, better → good
POS tagging	noun/verb/adjective labels	spaCy token.pos_
NER	find entities (person, org, date)	spaCy doc.ents
Regex	patterns: emails, IDs, ticket numbers	re.findall(r'INC\d+', text)

Interview tip: For transformer models you usually do **minimal** preprocessing — the model's own tokenizer handles casing and subwords, and stop words/punctuation carry meaning. Heavy cleaning is for classical (BoW/TF-IDF) models.

Bag of Words, n-grams and TF-IDF

Bag of Words: a document becomes a vector of word counts over the vocabulary — order is lost. **n-grams** keep local order: bigrams 'not good', 'New Delhi'. Vectors are huge and sparse.

$$\mathrm{TF}(t,d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}} \qquad \mathrm{IDF}(t) = \log \frac{N}{1+\mathrm{df}(t)} \qquad \mathrm{TFIDF}(t,d) = \mathrm{TF}(t,d) \times \mathrm{IDF}(t)$$

TF-IDF up-weights words frequent in *this* document but rare across the corpus (discriminative) and down-weights words everywhere ('the'). **Example:** N = 1000 docs; 'refund' appears in 10 docs → IDF = log(1000/11) ≈ 4.5; 'the' in 990 → IDF ≈ 0.01. BM25 (used in search and hybrid RAG) is a refined TF-IDF with term-frequency saturation and length normalisation.

tfidf_classifier.py — a strong, fast baseline

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
clf = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_features=50000, sublinear_tf=True),
                    LogisticRegression(max_iter=1000, class_weight="balanced"))
clf.fit(train_texts, train_labels); print(clf.score(test_texts, test_labels))
```

Word embeddings

One-hot/BoW vectors treat 'good' and 'great' as unrelated (orthogonal). **Distributional hypothesis**: words in similar contexts have similar meanings. Embeddings are dense vectors (100–300 dims) learned so that similar words are close.

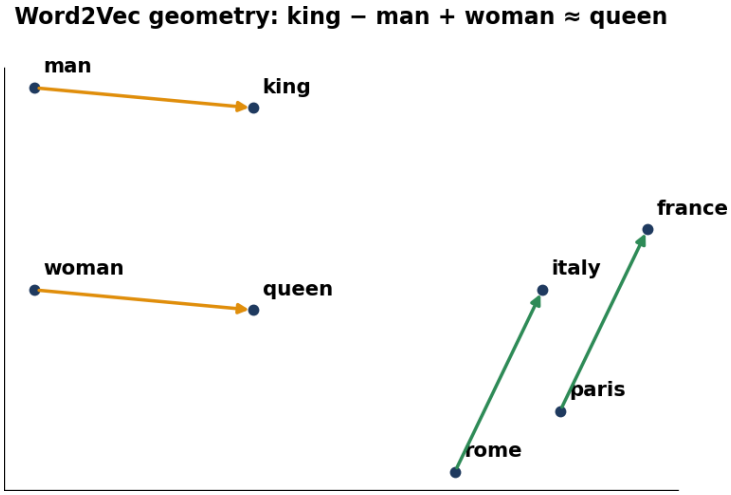


Figure: Embedding space captures relationships as directions (gender, capital-of).

Word2Vec (Mikolov 2013)

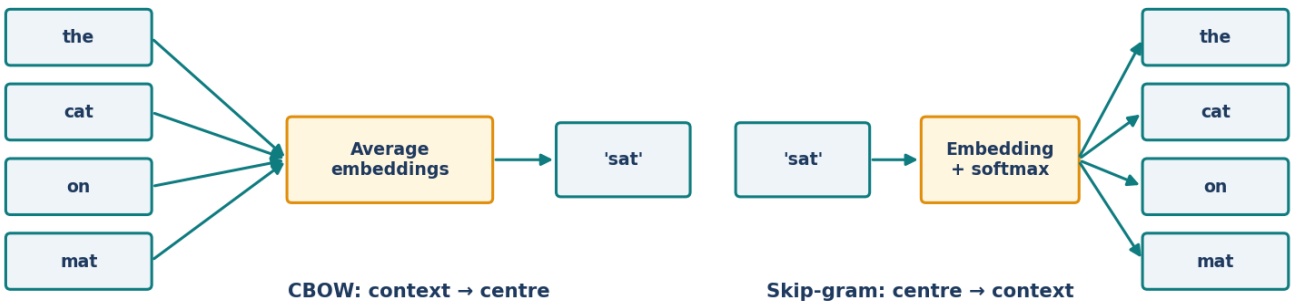


Figure: CBOW predicts the centre word from context; Skip-gram predicts context words from the centre word.

$$P(w_O \mid w_I) = \frac{\exp(v'^{T}_{w_O} v_{w_I})}{\sum_{w=1}^{V} \exp(v'^{T}_{w} v_{w_I})} \qquad \text{NEG}: \ \log \sigma(v'^{T}_{w_O} v_{w_I}) + \sum_{k=1}^{K} \log \sigma(-v'^{T}_{w_k} v_{w_I})$$

A shallow network with one hidden (embedding) layer. The full softmax over the vocabulary is expensive, so training uses **negative sampling** (distinguish the true context word from K random 'noise' words) or hierarchical softmax. Skip-gram is better for rare words; CBOW is faster. Window size controls syntactic (small) vs topical (large) similarity.

Model	Idea	Strength / weakness
Word2Vec	predict local context (CBOW / skip-gram)	fast, good analogies / one vector per word, no OOV handling
GloVe (Stanford)	factorise global word–word co-occurrence matrix: $w_i^T \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$	uses global statistics / static
FastText (Facebook)	word = sum of character n-gram vectors ('<wh', 'whe', ...)	handles out-of-vocabulary words and morphology
Contextual (ELMo, BERT, GPT)	vector depends on the sentence	'bank' (river) ≠ 'bank' (money) — solves polysemy

4.2 Sequence Models (recap) & the Transition to Transformers

RNN/LSTM/GRU and seq2seq with attention were covered in Chapter 3. Their limitations motivated the Transformer:

Limitation of RNNs	Transformer solution
Sequential computation — cannot parallelise across time steps; slow on GPUs	self-attention processes all tokens in parallel
Long-range dependencies degrade through many steps	any two tokens are connected by a path of length 1
Fixed-size hidden state bottleneck	every token keeps its own representation; attention reads all of them
Hard to scale to huge data	scales extremely well with data and compute (scaling laws)

Cost: self-attention is $O(n^2)$ in sequence length n (memory and compute), which motivated efficient attention (FlashAttention, sliding-window, sparse, linear attention) and KV caching.

4.3 Transformers — Architecture Deep Dive

'Attention Is All You Need' (Vaswani et al., 2017) introduced an encoder–decoder built only from attention and feed-forward layers. It is the foundation of BERT, GPT, T5, LLaMA, Gemini, Claude, ViT, Whisper and diffusion transformers.

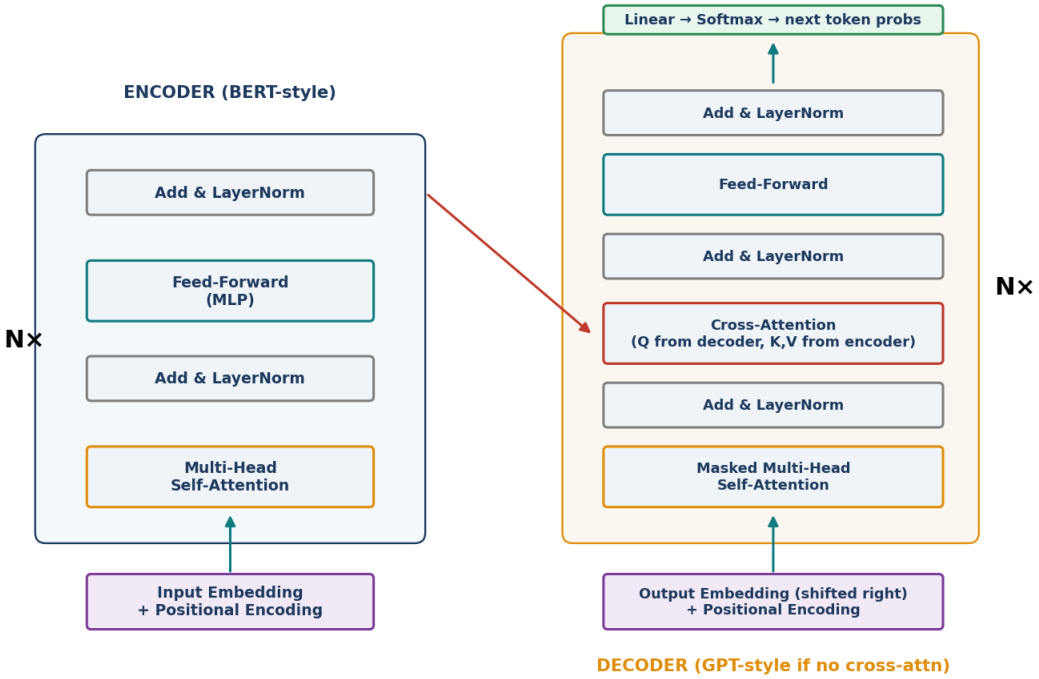


Figure: Transformer encoder (left) and decoder (right). Each is a stack of N identical blocks.

Step 1: Tokenisation and embeddings

Text → subword tokens → integer IDs → embedding lookup (vocab × d_{model} matrix, e.g. 50k × 768). The embedding is often multiplied by $\sqrt{d_{model}}$.

Step 2: Positional encoding

Attention is permutation-invariant — 'dog bites man' and 'man bites dog' would look identical — so position information is added.

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Each dimension is a sinusoid of a different frequency, so every position gets a unique pattern and relative offsets are linear functions. Alternatives: **learned** absolute embeddings (BERT, GPT-2), **RoPE** rotary embeddings (rotate Q and K by a position-dependent angle — LLaMA, most modern LLMs, good for long-context extension), **ALiBi** (linear distance bias on attention scores).

Step 3: Scaled dot-product self-attention

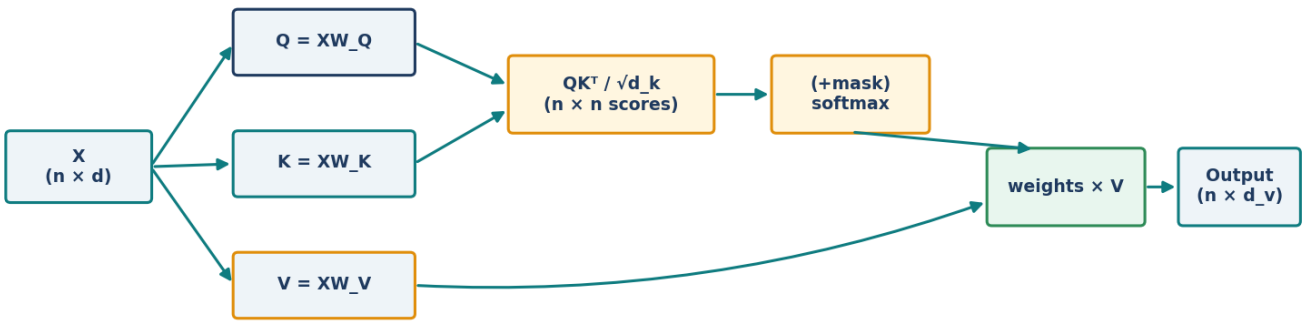


Figure: Each token creates a Query (what am I looking for?), Key (what do I contain?) and Value (what do I give?).

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \qquad \text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

1. Score: every query dotted with every key → n×n matrix of compatibility.
2. **Scale** by $\sqrt{d_k}$: dot products of d-dimensional vectors have variance ∝ d; without scaling, softmax saturates into one-hot outputs with tiny gradients.
3. **Mask** M: $-\infty$ for positions that must not be seen (future tokens in a decoder, padding tokens).
4. Softmax per row → attention weights summing to 1.
5. Weighted sum of value vectors → a new, context-aware representation of each token.

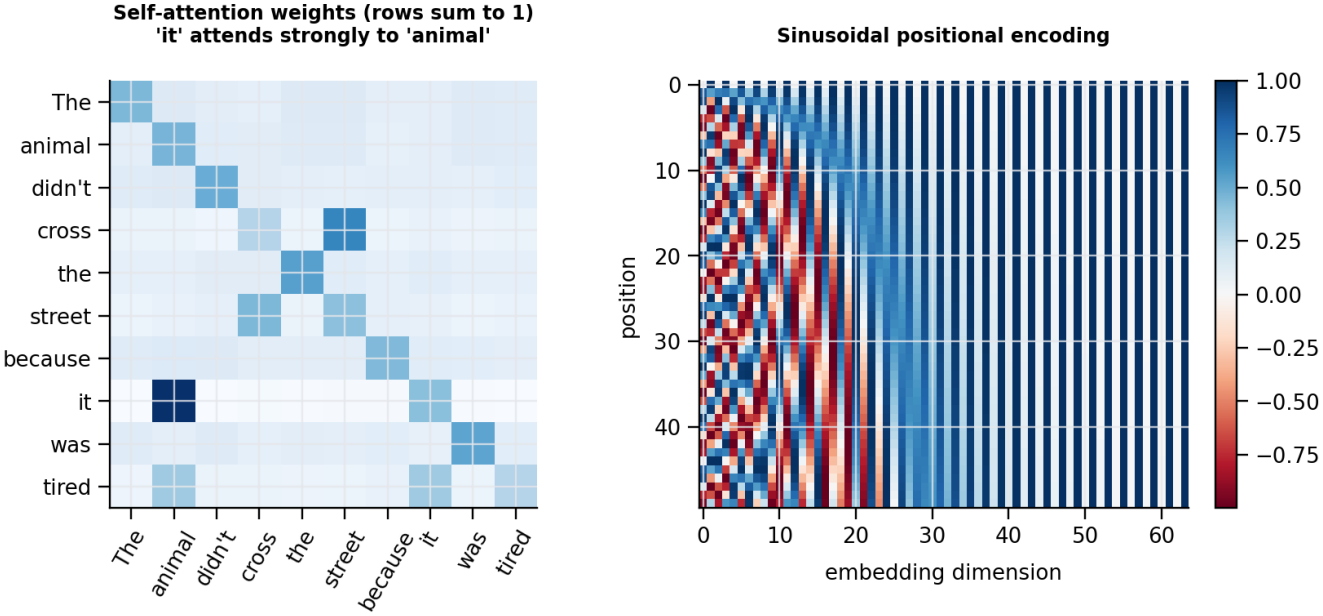


Figure: Left: attention weights for a sentence — the pronoun 'it' attends to 'animal'. Right: sinusoidal positional encodings.

Numeric mini-example: two tokens with $q_1 = [1, 0]$, $k_1 = [1, 0]$, $k_2 = [0, 1]$, $d_k = 2$. Scores = $[1, 0]/\sqrt{2} = [0.71, 0]$; softmax = [0.67, 0.33]; $\text{output}_1 = 0.67 \cdot v_1 + 0.33 \cdot v_2$.

Step 4: Multi-head attention

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O, \qquad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Instead of one attention with $d_{model} = 512$, use h = 8 heads of size 64 in parallel. Different heads learn different relations (syntax, coreference, position) — similar total cost. Variants for faster inference: **Multi-Query Attention** (all heads share K, V) and **Grouped-Query Attention** (groups of heads share K, V — LLaMA 2/3) which shrink the KV cache.

Step 5: Feed-forward network, residuals and layer norm

$$\text{FFN}(x) = W_2\,\text{GELU}(W_1 x + b_1) + b_2, \qquad x \leftarrow \text{LayerNorm}(x + \text{Sublayer}(x))$$

The FFN (expanding 4× then projecting back) is applied to each token independently — it is where much of the model's factual 'knowledge' is stored. Residual connections and LayerNorm keep deep stacks trainable. Modern LLMs use **Pre-LN** (normalise before the sub-layer), **RMSNorm**, and **SwiGLU** FFNs. **Mixture-of-Experts (MoE)** replaces one FFN with many expert FFNs and a router that picks the top-k per token — more parameters at the same compute (Mixtral, DeepSeek, reportedly GPT-4-class models).

Step 6: Decoder, masking and cross-attention

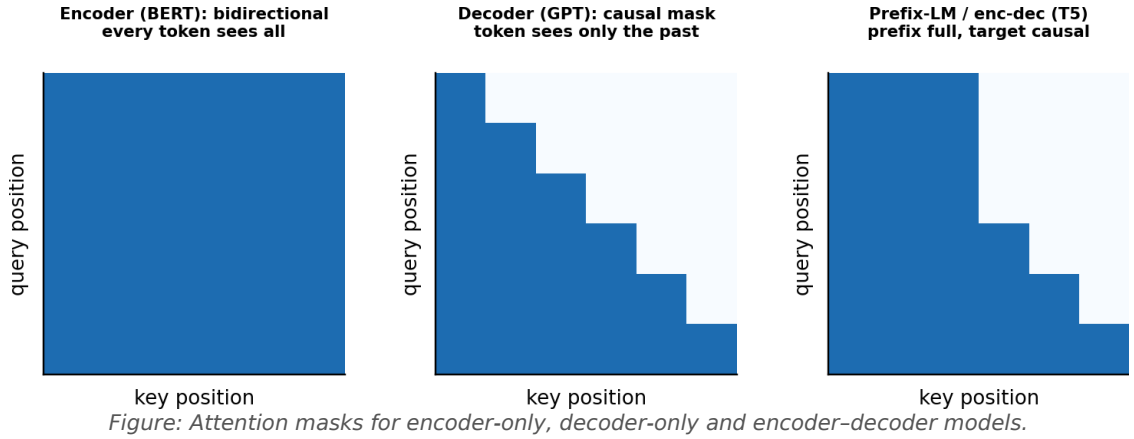


Figure: Attention masks for encoder-only, decoder-only and encoder–decoder models.

The decoder uses **masked (causal) self-attention** so position t only sees tokens ≤ t, then **cross-attention** where queries come from the decoder and keys/values from the encoder output. Finally a linear layer + softmax over the vocabulary gives next-token probabilities. During training all positions are computed in parallel (teacher forcing with the causal mask); at inference tokens are generated one by one.

Why do transformers work so well?

- Parallel training on GPUs/TPUs → can train on trillions of tokens.
- Direct global interactions between all tokens (no recurrence bottleneck).
- Flexible inductive bias: works for text, images (patches), audio, proteins, actions.
- Predictable **scaling laws**: loss falls smoothly as a power law with parameters, data and compute.
- Self-supervised objectives (masked/next-token prediction) can use unlimited unlabelled text.

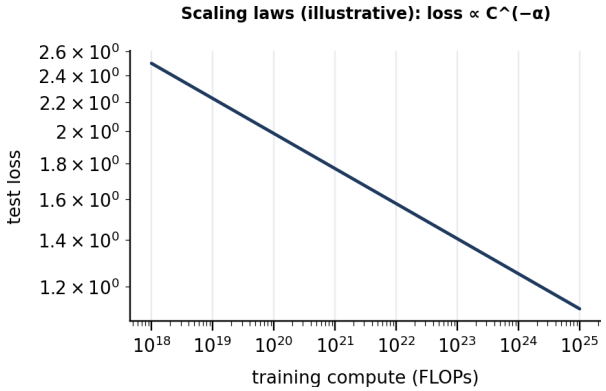


Figure: Scaling laws: test loss decreases as a power law in compute (illustrative).

Parameter count & complexity

Per layer ≈ $12 \cdot d_{model}^2$ parameters ($4d^2$ attention projections + $8d^2$ FFN). GPT-2 small: 12 layers, d = 768 → $12 \cdot 12 \cdot 768^2 \approx 85M$ + embeddings ≈ 124M. Attention compute $O(n^2 \cdot d)$, FFN $O(n \cdot d^2)$. **KV cache** stores past keys/values during generation so each new token costs O(n) instead of recomputing everything.

attention_from_scratch.py

```
import torch, torch.nn.functional as F, math
def scaled_dot_product_attention(Q, K, V, mask=None):          # Q,K,V: (batch, heads, seq, d_k)
    scores = Q @ K.transpose(-2, -1) / math.sqrt(Q.size(-1))
    if mask is not None: scores = scores.masked_fill(mask == 0, float("-inf"))
    return F.softmax(scores, dim=-1) @ V

class MultiHeadAttention(torch.nn.Module):
    def __init__(self, d_model=512, h=8):
        super().__init__(); self.h, self.d_k = h, d_model // h
        self.qkv = torch.nn.Linear(d_model, 3 * d_model); self.out = torch.nn.Linear(d_model, d_model)
    def forward(self, x, mask=None):
        B, T, D = x.shape
        q, k, v = self.qkv(x).view(B, T, 3, self.h, self.d_k).permute(2, 0, 3, 1, 4)
        y = scaled_dot_product_attention(q, k, v, mask)              # (B, h, T, d_k)
        return self.out(y.transpose(1, 2).reshape(B, T, D))
causal_mask = torch.tril(torch.ones(T, T))                           # decoder: see only the past
```

4.4 BERT, GPT, T5 and Their Variants

Family	Architecture	Pre-training objective	Best at
BERT (encoder-only)	bidirectional encoder	Masked LM + Next Sentence Prediction	understanding: classification, NER, QA extraction, embeddings, re-ranking
GPT (decoder-only)	causal decoder	next-token prediction (causal LM)	generation: chat, code, reasoning — all modern LLMs
T5 / BART (encoder–decoder)	full transformer	span corruption (T5) / denoising (BART)	seq2seq: translation, summarisation

BERT in detail

- **Input**: [CLS] sentence A [SEP] sentence B [SEP]; embeddings = token + segment (A/B) + position. WordPiece vocab ~30k.
- **Masked Language Modelling**: mask 15% of tokens — of those 80% → [MASK], 10% → random token, 10% unchanged (so the model can't rely on seeing [MASK] at fine-tuning time); predict originals using both left and right context.
- **Next Sentence Prediction**: is B the real next sentence? (later shown to be of little use; RoBERTa dropped it).
- **Sizes**: BERT-base 12 layers, 768 hidden, 12 heads, 110M; BERT-large 24 layers, 1024, 16 heads, 340M.
- **Fine-tuning**: add a task head — [CLS] vector → classifier; per-token vectors → NER tags; start/end pointers → extractive QA (SQuAD).
- **Sentence embeddings**: raw [CLS] is poor for similarity; **Sentence-BERT** fine-tunes siamese BERT with contrastive objectives → the embedding models used in RAG.

BERT variants

Variant	What changed	Benefit
RoBERTa	no NSP, dynamic masking, 10× more data, bigger batches, longer training	robustly better accuracy
DistilBERT	knowledge distillation from BERT (soft targets), 6 layers	40% smaller, 60% faster, ~97% performance
ALBERT	factorised embeddings + cross-layer parameter sharing; sentence-order prediction	far fewer params

Variant	What changed	Benefit
ELECTRA	replaced-token detection: a small generator corrupts tokens, discriminator detects which were replaced (learns from all tokens)	much more sample-efficient
DeBERTa	disentangled content & position attention, enhanced mask decoder	SOTA on GLUE/SuperGLUE
Domain BERTs	BioBERT, ClinicalBERT, FinBERT, LegalBERT, CodeBERT	domain vocabulary
Multilingual	mBERT, XLM-RoBERTa (100 languages)	cross-lingual transfer
Long-context	Longformer, BigBird (sparse attention), ModernBERT	long documents

Hands-on: fine-tuning BERT for classification

bert_finetune.py

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification, TrainingArguments, Trainer
import numpy as np, evaluate

ds = load_dataset("imdb"); ckpt = "distilbert-base-uncased"
tok = AutoTokenizer.from_pretrained(ckpt)
ds = ds.map(lambda b: tok(b["text"], truncation=True, max_length=256), batched=True)
model = AutoModelForSequenceClassification.from_pretrained(ckpt, num_labels=2)
f1 = evaluate.load("f1")
def metrics(p): return f1.compute(predictions=np.argmax(p.predictions, -1), references=p.label_ids)

args = TrainingArguments("out", learning_rate=2e-5, per_device_train_batch_size=16, num_train_epochs=2,
                         weight_decay=0.01, eval_strategy="epoch", save_strategy="epoch", load_best_model_at_end=T
Trainer(model=model, args=args, train_dataset=ds["train"], eval_dataset=ds["test"], tokenizer=tok, compute_metrics=
```

Optimising BERT for production

- **Distillation** (DistilBERT, TinyBERT): student learns teacher's soft probabilities (KL loss with temperature).
- **Quantisation** to INT8 (dynamic quantisation in PyTorch / ONNX Runtime) → ~2–4× faster on CPU.
- **Pruning** attention heads/weights; **ONNX export** + ONNX Runtime/TensorRT; shorter max_length; dynamic padding and batching.
- Use smaller models (MiniLM) when accuracy allows.

GPT family and T5

GPT-1 (2018, 117M) showed generative pre-training + fine-tuning; GPT-2 (1.5B) showed zero-shot abilities; **GPT-3** (175B) showed **in-context / few-shot learning** from prompts alone; InstructGPT/ChatGPT added instruction tuning + RLHF; GPT-4-class models added multimodality and stronger reasoning. **T5** casts every task as text-to-text ('summarize: …', 'translate English to German: …') with span-corruption pre-training; Flan-T5 is instruction-tuned.

4.5 Large Language Models (LLMs)

Tokenisation for LLMs

Algorithm	How	Used by
BPE (Byte-Pair Encoding)	start from characters/bytes; repeatedly merge the most frequent adjacent pair	GPT-2/3/4 (byte-level, tiktoken), LLaMA 3
WordPiece	merge pairs that maximise likelihood of training data	BERT
Unigram LM / SentencePiece	start with large vocab and prune; language-agnostic, treats space as a symbol	T5, LLaMA 1/2, Gemma

Subwords balance vocabulary size and OOV handling: 'unhappiness' → 'un', 'happi', 'ness'. Rough rule: 1 token ≈ 4 English characters ≈ 0.75 words. Pricing, context limits and latency are all counted in tokens. Indic languages often use more tokens per word.

How LLMs are trained

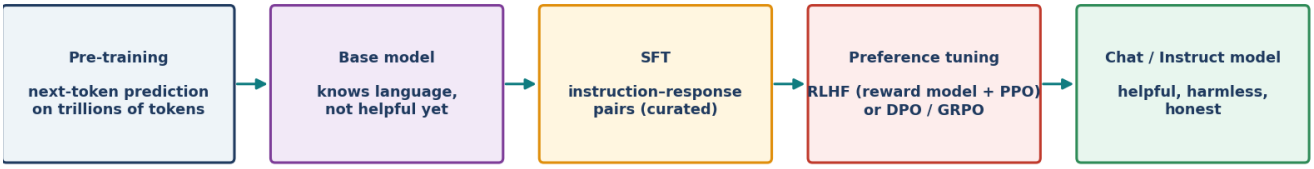


Figure: LLM training pipeline.

$$\mathcal{L}_{LM} = -\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \qquad \text{Perplexity} = \exp\left(\frac{1}{T}\mathcal{L}_{LM}\right)$$

Stage	Data	Objective	Result
1. Pre-training	trillions of tokens: web, books, code, papers (filtered, deduplicated)	next-token prediction	base model: knowledge + language, continues text
2. Supervised fine-tuning (SFT)	10k–1M high-quality (instruction, response) pairs, multi-turn chats	same LM loss on responses	follows instructions and chat format
3a. RLHF	humans rank multiple responses → train a reward model r(x, y)	optimise policy with PPO to maximise reward − β·KL(π ‖ π_{SFT})	helpful, harmless, preferred style
3b. DPO / ORPO / GRPO	preference pairs (chosen, rejected)	DPO: direct classification-style loss, no reward model or RL loop	simpler, stable alignment
4. Reasoning RL (newer)	verifiable tasks (math, code tests)	reinforcement learning on correct outcomes	long chain-of-thought 'reasoning models'

$$\mathcal{L}_{DPO} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)$$

y_w = preferred (chosen) response, y_l = rejected response

Decoding / sampling strategies

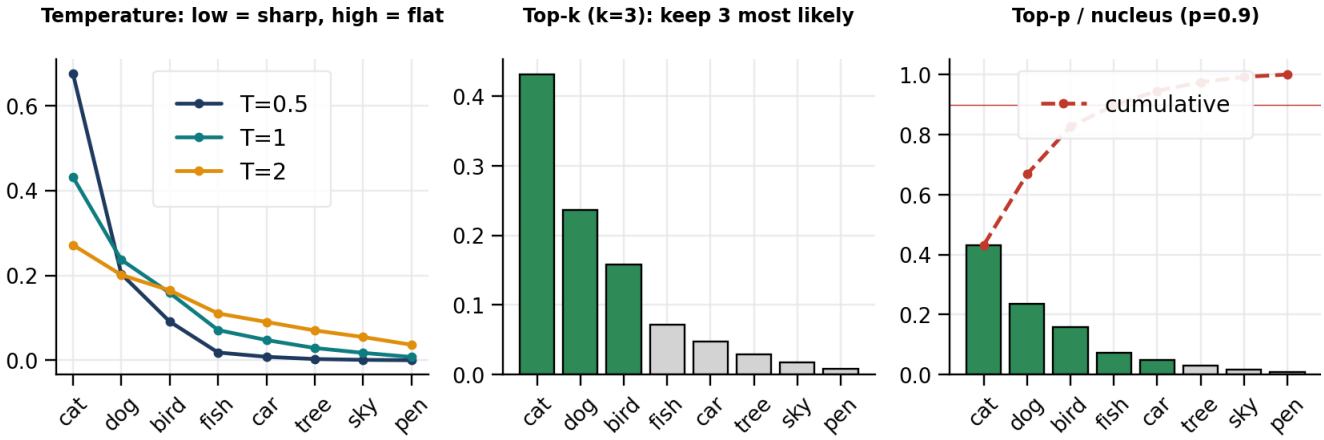


Figure: Temperature, top-k and top-p (nucleus) sampling reshape the next-token distribution.

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Strategy	Behaviour	Use
Greedy	always pick the most likely token	deterministic, can be repetitive
Beam search	keep top-B partial sequences	translation/summarisation; bland for chat
Temperature T	T < 1 sharper (focused), T > 1 flatter (creative); T → 0 ≈ greedy	0–0.3 factual/RAG/extraction; 0.7–1.0 creative
Top-k	sample only among k most likely tokens	removes long tail
Top-p (nucleus)	sample from smallest set with cumulative prob ≥ p	adapts to confidence; common p = 0.9–0.95
Penalties	repetition / frequency / presence penalties	reduce loops
Stop sequences, max_tokens	end generation	control length/cost
Structured output	constrain to JSON schema / grammar	reliable parsing, tool calls

Prompt engineering

Technique	Idea	Example
Zero-shot	instruction only	"Classify sentiment: …"
Few-shot (in-context learning)	include examples in the prompt	3 labelled tickets then the new one
Role / system prompt	set persona, rules, format	"You are a senior IT support engineer…"
Chain-of-Thought	ask for step-by-step reasoning before the answer	"Think step by step" / worked examples
Self-consistency	sample several CoT paths, majority vote	math problems
ReAct	interleave reasoning and tool actions	agents (Chapter 7)
Structured output	demand JSON with a schema	Pydantic + with_structured_output
Delimiters	separate instructions from data	### or XML-like tags

Technique	Idea	Example
Prompt chaining	split task into sequential prompts	extract → summarise → format
Tree/Graph-of-Thought	explore and evaluate multiple reasoning branches	planning/puzzles

Interview tip: A good prompt = **role + task + context + constraints + output format + examples**. Be specific, put instructions first, tell the model what to do (not only what not to do), and give it an 'I don't know' escape path.

Adapting LLMs: prompting vs RAG vs fine-tuning

Need	Approach
Change format/tone quickly	prompt engineering, few-shot
Fresh or private knowledge, citations	RAG (Chapter 6)
New skill, domain style, consistent structured behaviour, smaller cheaper model	fine-tuning (full or PEFT)
Take actions, use tools	agents with function calling (Chapter 7)

Parameter-Efficient Fine-Tuning (PEFT): LoRA & QLoRA

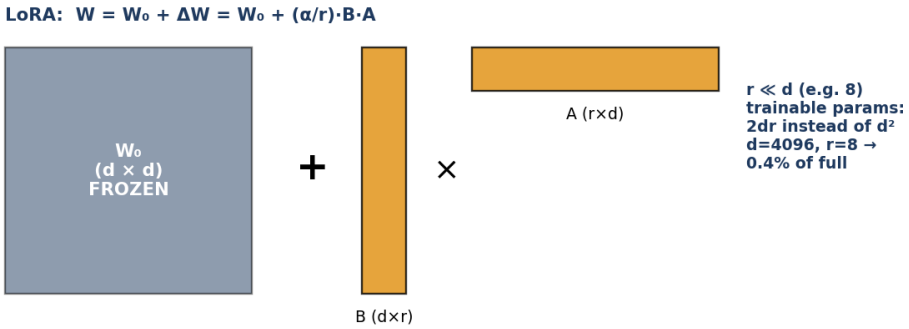


Figure: LoRA freezes W_0 and learns a low-rank update BA.

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} BAx, \qquad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k)$$

A is initialised randomly and B with zeros, so training starts exactly from the base model. Typically applied to attention projections (q, k, v, o) and sometimes MLP layers. After training, BA can be **merged** into W_0 → no inference latency, and many small adapters can be swapped on one base model. **QLoRA** loads the frozen base model in 4-bit NF4 quantisation (with double quantisation and paged optimizers) and trains LoRA adapters in 16-bit — fine-tune a 7–13B model on a single consumer GPU. Other PEFT: prefix/prompt tuning, adapters, IA³.

qlora_setup.py

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4", bnb_4bit_compute_dtype="bfloat16")
model = AutoModelForCausalLM.from_pretrained(base_model_id, quantization_config=bnb, device_map="auto")
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                                         target_modules=["q_proj", "k_proj", "v_proj", "o_proj"], task_type="CAUSA
model.print_trainable_parameters()     # e.g. ~0.1–1% of all parameters
# then train with transformers Trainer / TRL SFTTrainer on instruction data
```

Quantisation and efficient inference

Technique	What
Precision	FP32 (4 bytes/param) → FP16/BF16 (2) → INT8 (1) → INT4 (0.5). A 7B model ≈ 14 GB in FP16, ≈ 3.5–4 GB in 4-bit
Post-training quantisation	GPTQ, AWQ, bitsandbytes, GGUF (llama.cpp) — quantise weights after training
KV cache, paged attention	vLLM's PagedAttention manages KV memory like OS pages → high throughput
Continuous batching	add/remove requests from a running batch (vLLM, TGI)
Speculative decoding	small draft model proposes tokens, big model verifies several at once
FlashAttention	IO-aware exact attention kernel — faster, less memory
Distillation	train a small student on a big teacher's outputs

LLM applications & capabilities

Application	How
Summarisation	extractive (select sentences) vs abstractive (rewrite); map-reduce or refine for long docs; evaluate with ROUGE + LLM-judge for faithfulness
Sentiment / classification	zero/few-shot prompt with fixed label set + structured output, or fine-tuned BERT for scale
Information extraction	JSON schema extraction of entities, fields from invoices/tickets
Conversational AI	system prompt + chat history (memory) + tools/RAG; manage context window
Multilingual	translation, cross-lingual QA; check tokenizer efficiency and evaluate per language
Code	generation, explanation, test writing, review (Chapter 5)
Reasoning & planning	CoT, reasoning models, agents

LLM landscape

Provider	Models (families)	Notes
OpenAI	GPT-4o / GPT-4.1 / o-series reasoning / GPT-5-class	API, strong tools & structured outputs
Google	Gemini (Flash, Pro), Gemma (open weights)	long context, multimodal; Gemini API / Vertex AI
Anthropic	Claude (Haiku, Sonnet, Opus)	strong coding & long documents
Meta	LLaMA 2/3/4 (open weights)	self-hosting, fine-tuning
Mistral	Mistral, Mixtral (MoE)	efficient open models
Others	DeepSeek (V3, R1), Qwen, Phi (small), Cohere Command	open/small/enterprise options

Model names and versions change frequently — check provider docs for current IDs. Selection criteria: quality on *your* eval set, latency, cost per million tokens, context window, multimodality, tool-calling reliability, data privacy/residency, open vs closed weights, licence.

API integration (OpenAI, Gemini, Hugging Face)

llm_apis.py

```
# OpenAI
from openai import OpenAI
client = OpenAI()                                          # reads OPENAI_API_KEY
r = client.chat.completions.create(model="gpt-4o-mini", temperature=0,
        messages=[{"role": "system", "content": "You are a concise assistant."}, {"role": "user", "content": "Expla
print(r.choices[0].message.content, r.usage.total_tokens)

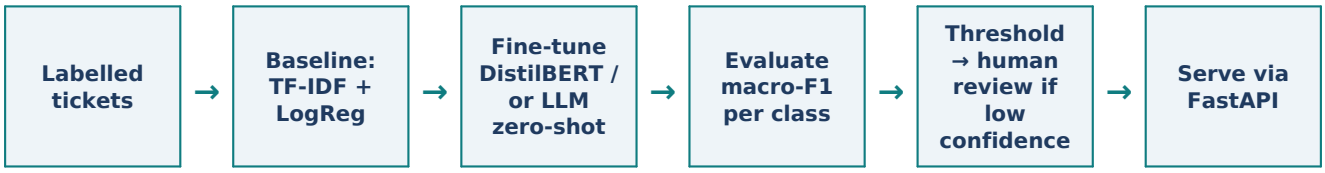
# Google Gemini (google-genai SDK)
from google import genai
g = genai.Client()                                         # reads GEMINI_API_KEY / GOOGLE_API_KEY
print(g.models.generate_content(model="gemini-2.5-flash", contents="Explain RAG in 2 lines.").text)

# Hugging Face local pipeline
from transformers import pipeline
summ = pipeline("summarization", model="facebook/bart-large-cnn"); print(summ(long_text, max_length=120)[0]["summa
sent = pipeline("sentiment-analysis"); print(sent("The update broke my login!"))
```

Watch out: Production API hygiene: keys in environment variables / secret manager (never in code or Git), timeouts + retries with exponential backoff, rate-limit handling (429), streaming for UX, token/cost logging, input/output guardrails, and fallbacks to another model.

4.6 Practical NLP Projects

Text classification (e.g. IT ticket routing)



Summarisation

$$\text{ROUGE-N} = \frac{\sum \text{overlapping n-grams}}{\sum \text{n-grams in reference}} \quad \text{(recall)}, \qquad \text{ROUGE-L}: \text{ longest common subsequence}$$

Also BLEU (precision of n-grams, translation), BERTScore (embedding similarity), and LLM-as-judge for faithfulness/coverage. For long documents: chunk → summarise each (map) → combine (reduce), or iterative refine.

Chatbots

Generation	Approach
Rule-based	patterns/decision trees (AIML) — predictable, brittle
Intent + entity (Rasa, Dialogflow)	classify intent, extract slots, scripted flows
LLM chatbot	system prompt + history + LLM
RAG chatbot	LLM grounded on company documents with citations
Agentic assistant	LLM + tools (create ticket, check order) + memory + human handoff

simple_chatbot.py

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.3)
prompt = ChatPromptTemplate.from_messages([("system", "You are a helpful bank support assistant. Be brief."),
                                           MessagesPlaceholder("history"), ("human", "{input}")])
chain, history = prompt | llm, []
while (q := input("You: ")) != "exit":
    ans = chain.invoke({"history": history, "input": q}).content
    history += [HumanMessage(q), AIMessage(ans)]; history = history[-10:]   # windowed memory
    print("Bot:", ans)
```

4.7 Interview Questions — NLP, Transformers & LLMs

Q1. Stemming vs lemmatisation?

Stemming chops suffixes with heuristic rules (fast, may give non-words like 'studi'); lemmatisation uses vocabulary and POS to return the dictionary form ('study'), slower but accurate.

Q2. Explain TF-IDF.

Term frequency in a document times inverse document frequency across the corpus; it highlights words that are frequent locally but rare globally, down-weighting common words.

Q3. Word2Vec vs BERT embeddings?

Word2Vec gives one static vector per word regardless of context; BERT produces contextual vectors that change with the sentence, handling polysemy like 'bank'.

Q4. Why do transformers need positional encoding?

Self-attention is permutation-invariant; without position information the model cannot distinguish word order.

Q5. Why divide by √d_k in attention?

Dot products grow with dimension, pushing softmax into saturated regions with tiny gradients; scaling keeps the variance around 1.

Q6. What are Q, K, V intuitively?

Query is what a token is looking for, Key is what each token offers for matching, Value is the information passed on once matched; output is a similarity-weighted sum of values.

Q7. Why multi-head attention?

Multiple heads attend in different representation subspaces simultaneously, capturing different relationships (syntax, coreference, position) at similar cost.

Q8. Encoder-only vs decoder-only vs encoder-decoder?

Encoder-only (BERT) is bidirectional for understanding tasks; decoder-only (GPT) is causal for generation; encoder-decoder (T5) maps an input sequence to an output sequence such as translation.

Q9. Explain BERT pre-training.

Masked language modelling (predict 15% masked tokens using both sides of context) and next sentence prediction; then fine-tune with a small task head.

Q10. What is the complexity of self-attention and how is it reduced?

$O(n^2)$ in sequence length for time and memory. Reduced with FlashAttention (IO-efficient), sparse/sliding-window attention, linear attention, and KV caching for generation.

Q11. What is a KV cache?

During autoregressive generation, keys and values of previous tokens are stored so each new token only computes its own Q/K/V and attends to the cache, avoiding recomputation.

Q12. Temperature vs top-p?

Temperature rescales logits to make the distribution sharper or flatter; top-p restricts sampling to the smallest set of tokens whose cumulative probability exceeds p. Use low temperature for factual tasks.

Q13. Explain RLHF.

Collect human preference rankings, train a reward model on them, then optimise the SFT model with PPO to maximise reward while a KL penalty keeps it close to the SFT model. DPO achieves similar alignment directly from preference pairs without a reward model.

Q14. What is LoRA and why is it efficient?

It freezes pretrained weights and learns a low-rank update BA in selected layers, training well under 1% of parameters, reducing memory, enabling adapter swapping, and merging back with no latency cost. QLoRA adds 4-bit quantisation of the base model.

Q15. When to fine-tune vs use RAG?

RAG for knowledge that changes or must be cited; fine-tuning for behaviour, style, format, or specialised skills and for distilling into a smaller model. They are often combined.

Q16. What is hallucination and how do you reduce it?

Fluent but unsupported output. Reduce with RAG grounding, instructions to answer only from context, low temperature, citations, self-verification/groundedness checks, structured outputs and evaluation.

Q17. What is perplexity?

Exponential of the average negative log-likelihood per token; lower means the model is less 'surprised' by the text.

Q18. What is in-context learning?

The model's ability to perform a new task from instructions and examples in the prompt, without weight updates — emerged at scale in GPT-3.

Q19. How do you evaluate an LLM application?

Task-specific golden datasets with automatic metrics (accuracy, F1, ROUGE), LLM-as-judge with rubrics, human evaluation, safety tests, plus latency and cost monitoring.

Q20. How would you reduce LLM inference cost?

Smaller or distilled models, quantisation, prompt compression and caching, semantic caching, batching (vLLM), shorter outputs, routing easy queries to cheap models.

CHAPTER 5

Generative AI Beyond Text: Images, Video, Audio, Multimodal

What GenAI is • Generative model types • Diffusion theory (DDPM, DDIM, score-based) • Latent/Stable Diffusion • Guidance • ControlNet, LoRA, DreamBooth • DALL-E, Imagen • Video • TTS & music • Multimodal (CLIP, LLaVA, Gemini) • Code generation • Content creation • Ethics & responsible AI

5.1 Introduction to Generative AI

Generative AI creates new content — text, images, audio, video, code, 3D, molecules — by learning the probability distribution of training data and sampling from it, usually conditioned on a prompt. What changed since ~2020: transformer scale, diffusion models, massive paired datasets (image–text), and alignment techniques made outputs controllable through natural language.

Model type	Core idea	Examples
Autoregressive	predict the next token given previous ones: $P(x) = \Pi\ P(x_t \mid x_{<t})$	GPT, Gemini, Claude, LLaMA; PixelCNN; AudioLM; Parti
VAE	encode to a latent distribution, decode samples	VQ-VAE, the VAE inside Stable Diffusion
GAN	generator vs discriminator game	StyleGAN, BigGAN, GigaGAN
Diffusion	learn to reverse a gradual noising process	Stable Diffusion, DALL-E 2/3, Imagen, Midjourney, Sora-type video models
Flow / flow-matching	invertible transforms or learned velocity fields from noise to data	Glow; rectified flow (Stable Diffusion 3, Flux)
Multimodal LLM	one model reads (and sometimes writes) text, images, audio, video	GPT-4o, Gemini, Claude, LLaVA

Domain	Applications
Text	chat, summarisation, drafting, translation, code
Image	marketing creatives, product shots, design ideation, editing/inpainting, super-resolution
Audio	text-to-speech, voice cloning (with consent), music, dubbing
Video	ad clips, animation, avatars, explainer videos
Science/industry	drug & protein design, synthetic data, materials, chip layout

5.2 Diffusion Models

Intuition

Destroy data slowly by adding Gaussian noise over T steps until it is pure noise (fixed **forward** process). Train a neural network to undo one small step of noise (learned **reverse** process). To generate: start from random noise and denoise step by step. Each step is an easy problem, which is why training is stable (just regression), unlike GANs.

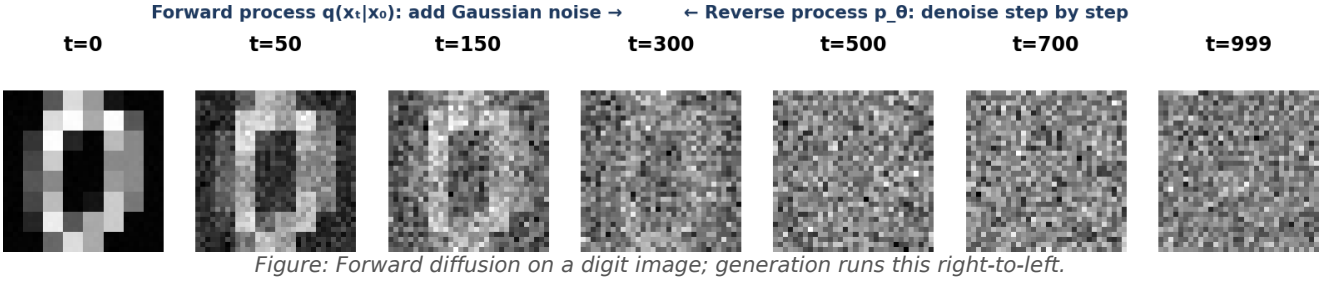


Figure: Forward diffusion on a digit image; generation runs this right-to-left.

DDPM mathematics

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\right)$$

forward step with a small variance schedule β₁…β_T (e.g. 1e-4 → 0.02, T = 1000)

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s \qquad x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

closed form: jump to any step t directly — makes training cheap

$$\mathcal{L}_{simple} = \mathbb{E}_{x_0,\, t,\, \varepsilon}\left[\left\| \varepsilon - \varepsilon_\theta\left(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon,\ t\right)\right\|^2\right]$$

the network predicts the noise that was added (simple MSE)

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon_\theta(x_t, t)\right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I)$$

one reverse (sampling) step

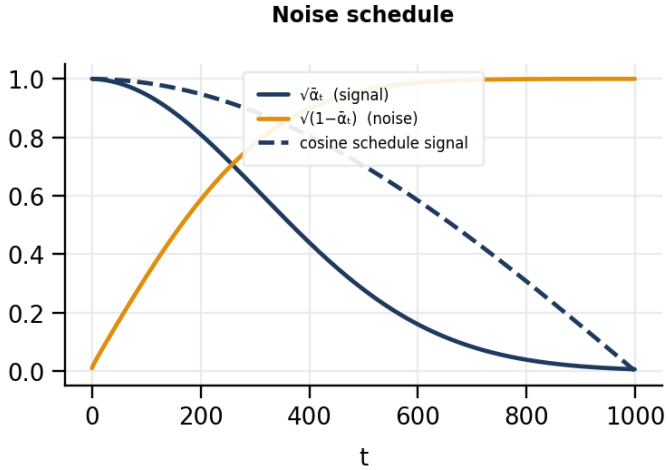


Figure: Signal vs noise level across timesteps; the cosine schedule destroys information more gradually.

Training loop	Sampling loop
1. take a clean image x_0 2. pick random $t \in [1, T]$ 3. sample noise ε, build x_t in closed form 4. predict $\hat{\varepsilon} = \varepsilon_\theta(x_t, t)$ 5. loss = $\|\varepsilon - \hat{\varepsilon}\|^2$, backprop	1. $x_T \sim N(0, I)$ 2. for t = T…1: predict noise, compute x_{t-1} with the formula above 3. return x_0 (DDPM needs ~1000 steps; DDIM/DPM-Solver 20–50)

Connections and improvements

- **Score-based view**: predicting noise is equivalent to estimating the score $\nabla_x \log p(x_t)$; sampling = Langevin dynamics / solving a reverse SDE. Diffusion is also a hierarchical VAE with a fixed encoder.
- **DDIM**: deterministic, non-Markovian sampler — same trained model, 10–50× fewer steps, enables image inversion/editing.
- **Schedulers / solvers**: DDPM, DDIM, Euler, Euler-a, DPM-Solver++, LCM — trade speed vs quality.
- **v-prediction** and **flow matching / rectified flow**: alternative targets that give straighter paths and fewer steps (SD3, Flux).
- **Distillation**: consistency models, LCM, SDXL-Turbo — 1–4 step generation.

Conditioning and classifier-free guidance (CFG)

$$\hat{\varepsilon} = \varepsilon_\theta(x_t, t, \varnothing) + w \cdot (\varepsilon_\theta(x_t, t, c) - \varepsilon_\theta(x_t, t, \varnothing))$$

Train one model with the text condition c randomly dropped (e.g. 10% of the time) so it learns both conditional and unconditional noise prediction. At sampling, extrapolate away from the unconditional prediction by guidance scale w (typically 5–9): higher w = closer prompt adherence but less diversity/over-saturated. Negative prompts replace ∅ with an unwanted concept.

Stable Diffusion (Latent Diffusion) architecture

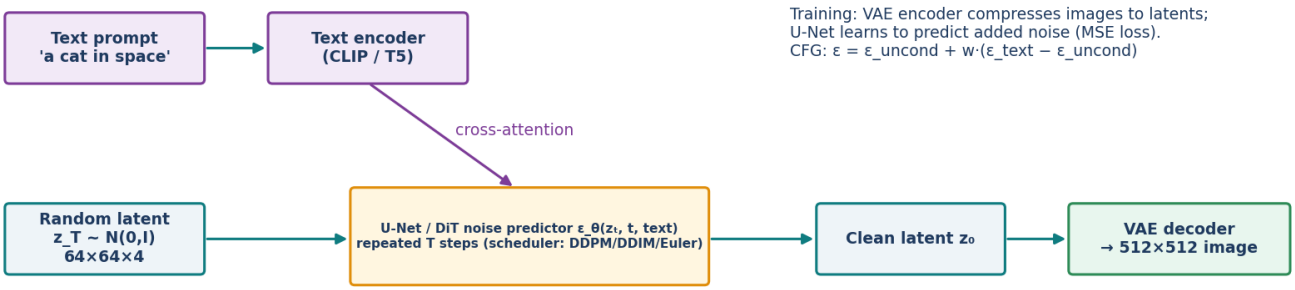


Figure: Stable Diffusion: text encoder → U-Net denoising in latent space with cross-attention → VAE decoder.

Component	Role	Detail
VAE	compress image ↔ latent	512×512×3 → 64×64×4 (48× fewer values) so diffusion is cheap
Text encoder	turn prompt into embeddings	CLIP ViT-L (SD 1.x), OpenCLIP (SD 2), two CLIPs (SDXL), CLIP + T5 (SD3)
U-Net (or DiT)	predict noise in latent space	ResNet blocks + self-attention + **cross-attention** (Q = image latents, K/V = text embeddings); timestep embedding; newer models use Diffusion Transformers

Component	Role	Detail
Scheduler	noise schedule + update rule	DDIM, Euler, DPM++
Safety checker	filter unsafe outputs	optional classifier

Task	How
Text-to-image	start from pure noise latent
Image-to-image	encode an image, add noise to step t (strength), denoise with new prompt
Inpainting / outpainting	denoise only masked region (extra mask channels)
ControlNet	trainable copy of the U-Net encoder conditioned on edges/pose/depth maps, joined with zero-convolutions → precise spatial control
LoRA / DreamBooth / Textual Inversion	personalise a subject or style: LoRA adapters; DreamBooth fine-tunes on 3–5 images with a rare token; Textual Inversion learns a new token embedding
IP-Adapter	image prompt conditioning (style/subject reference)
Upscaling	latent upscalers, ESRGAN

DALL-E, Imagen and others

Model	Approach
DALL-E 1 (2021)	discrete VAE tokens + autoregressive transformer over text+image tokens
DALL-E 2 (2022)	unCLIP: prior maps CLIP text embedding → CLIP image embedding; diffusion decoder generates image
DALL-E 3	diffusion trained on highly descriptive synthetic captions; prompts rewritten by an LLM → much better prompt following
Imagen (Google)	frozen large T5 text encoder + cascaded pixel diffusion (64 → 256 → 1024)
Midjourney	proprietary, aesthetic focus
SDXL / SD3 / Flux	open(-weight) latent diffusion / rectified-flow transformers

Hands-on: image generation with diffusers

sdxl_generate.py

```
import torch
from diffusers import StableDiffusionXLPipeline, DPMSolverMultistepScheduler
pipe = StableDiffusionXLPipeline.from_pretrained("stabilityai/stable-diffusion-xl-base-1.0",
                                                 torch_dtype=torch.float16).to("cuda")
pipe.scheduler = DPMSolverMultistepScheduler.from_config(pipe.scheduler.config)   # fast solver
img = pipe(prompt="isometric illustration of a modern bank branch, soft lighting, clean vector style",
           negative_prompt="blurry, low quality, text, watermark",
           num_inference_steps=30, guidance_scale=7.0,
           generator=torch.Generator("cuda").manual_seed(42)).images[0]          # seed → reproducible
img.save("bank.png")
```

Prompt anatomy for images: **subject** + **context/scene** + **style/medium** + **lighting/camera** + **quality modifiers**; use negative prompts and fixed seeds for iteration. Metrics: FID (realism vs a reference set), CLIP score (prompt alignment), human preference.

5.3 Advanced Generation: Video, Audio, Music, Multimodal

Video generation

Video adds a time dimension and requires **temporal consistency** (objects should not flicker or morph). Approaches: extend image U-Nets with temporal attention/convolution layers (Video LDM, AnimateDiff, Stable Video Diffusion); **diffusion transformers over spacetime patches** (Sora-style: compress video to a latent with a video VAE, cut into spacetime patches = tokens, denoise with a transformer); autoregressive video tokens. Tasks: text-to-video, image-to-video, video editing, frame interpolation. Challenges: compute cost, physics plausibility, long durations, evaluation (FVD).

Audio: speech recognition and text-to-speech

Task	Pipeline / models
ASR (speech → text)	audio → log-Mel spectrogram → encoder–decoder transformer (Whisper); CTC models (wav2vec 2.0)
Classic TTS	text → normalisation & phonemes → acoustic model (Tacotron 2, FastSpeech 2) → mel-spectrogram → **vocoder** (WaveNet, WaveGlow, HiFi-GAN) → waveform
End-to-end TTS	VITS (VAE + flow + GAN), neural codec language models (VALL-E: text + 3-sec prompt → audio codec tokens), diffusion TTS
Voice cloning	speaker embeddings / prompts; requires explicit consent and watermarking
Speech-to-speech	real-time voice assistants (native audio LLMs like GPT-4o / Gemini Live)

Mel spectrogram: short-time Fourier transform magnitude mapped to the perceptual Mel scale — the 'image' of audio most models consume. **Neural audio codecs** (EnCodec, SoundStream) turn audio into discrete tokens so language-model techniques apply.

Music generation

Symbolic (MIDI notes — Music Transformer) vs raw audio (Jukebox, MusicLM, MusicGen, Stable Audio, Suno/Udio-type systems). MusicGen: a transformer generating EnCodec tokens conditioned on text and optionally a melody. Issues: long-range structure, copyright of training data and outputs.

Multimodal GenAI models

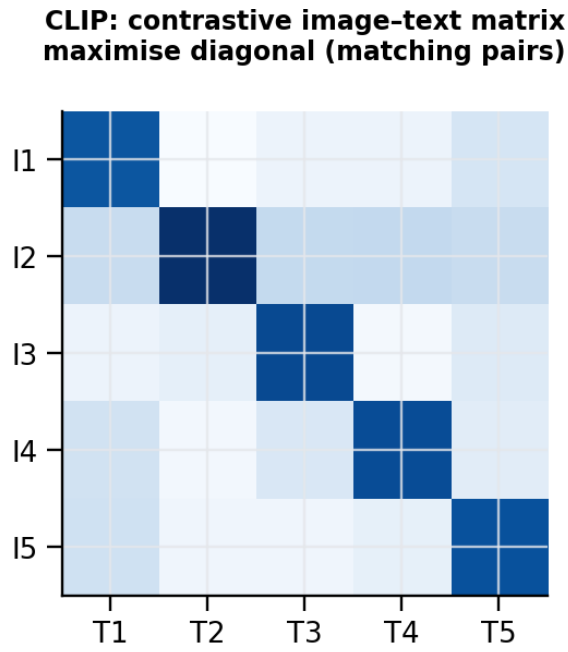


Figure: CLIP training: matching image–text pairs on the diagonal are pulled together.

$$\mathcal{L}_{CLIP} = -\frac{1}{2N}\sum_{i=1}^{N}\left[\log\frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ij}/\tau}} + \log\frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ji}/\tau}}\right], \qquad s_{ij} = \cos(I_i, T_j)$$

symmetric InfoNCE contrastive loss with temperature τ

Model / pattern	How it works	Use
CLIP / SigLIP	dual encoders trained contrastively on 400M+ image–text pairs → shared embedding space	zero-shot classification, image search, text encoder for diffusion, multimodal RAG
LLaVA-style VLM	vision encoder (CLIP ViT) → projection (MLP) → LLM tokens; instruction-tuned on image chats	visual QA, document/chart understanding
Flamingo / BLIP-2	frozen vision + frozen LLM bridged by perceiver/Q-Former with cross-attention	few-shot multimodal
Natively multimodal (GPT-4o, Gemini)	trained jointly on text, image, audio (and video) tokens	any-to-any assistants, live voice
Any-to-any	LLM + modality encoders + modality decoders (diffusion/TTS)	text + image + audio + video in and out

5.4 Creative Applications

Code generation with LLMs

Code LLMs (Codex, Code Llama, StarCoder, DeepSeek-Coder, GPT/Claude/Gemini) are trained on large code corpora with next-token prediction plus **fill-in-the-middle (FIM)** for editor completion. Uses: autocomplete (GitHub Copilot), generating functions/tests, explaining legacy code, refactoring, SQL generation, code review, agentic coding (plan → edit files → run tests → fix).

$$\text{pass@}k = \mathbb{E}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

probability at least one of k samples passes unit tests (n samples generated, c correct) — HumanEval/MBPP metric

- Best practices: give context (file, interfaces, versions), ask for tests, run and verify, review for security (SQL injection, secrets), check licences, never trust generated code blindly.
- Risks: hallucinated APIs/packages (supply-chain 'slopsquatting'), insecure patterns, outdated library versions.

AI content creation (copywriting, marketing)

Use	Technique
Ad copy & email variants	few-shot with brand voice examples; generate N variants; A/B test
SEO blogs	outline → section-by-section generation → fact check → human edit
Personalisation at scale	template + customer attributes + guardrails
Social media visuals	diffusion + brand LoRA + text overlays added by designers
Localisation	translation + cultural adaptation + human review

Interview tip: Brand voice consistency: store a style guide + approved examples and inject them (or retrieve them via RAG) into the prompt; use low temperature for compliance-sensitive copy and higher for ideation; always keep a human approval step for published content.

Ethics & Responsible AI

Risk	Example	Mitigation
Bias & fairness	loan model or image generator under-representing groups	diverse data, bias audits (demographic parity, equal opportunity), fairness constraints, red-teaming
Hallucination / misinformation	invented facts, fake citations	RAG, citations, verification, disclaimers, human review
Deepfakes & impersonation	fake voice/video of real people	consent, watermarking (C2PA content credentials, SynthID), detection, usage policies
Copyright & IP	training on copyrighted works; outputs resembling them	licensed/opt-out data, output filters, legal review, attribution
Privacy	PII memorised or leaked; sensitive prompts	PII masking, data minimisation, retention policies, on-prem/private endpoints
Security	prompt injection, jailbreaks, data exfiltration via tools	input/output guardrails, least-privilege tools, sandboxing
Toxicity & harmful content	hate speech, self-harm content	moderation models, safety tuning, refusal policies
Environmental cost	large training/inference energy	smaller/efficient models, caching, green regions
Accountability & transparency	who is responsible for AI decisions?	model cards, documentation, explainability, human-in-the-loop, audit logs

Principles common to most frameworks (Microsoft Responsible AI Standard, OECD, NIST AI RMF, EU AI Act risk tiers): **fairness, reliability & safety, privacy & security, inclusiveness, transparency, accountability**. In interviews, connect each principle to a concrete engineering control.

5.5 Interview Questions — Generative AI

Q1. Explain diffusion models simply.

Gradually add noise to data until it becomes pure noise, and train a network to predict and remove that noise one step at a time. Generation starts from random noise and repeatedly denoises into a sample.

Q2. Why does the diffusion network predict noise instead of the image?

Predicting ε gives a simple, well-scaled MSE target at every noise level, equivalent to learning the score function, and it trains more stably than predicting x_0 directly.

Q3. What is the role of the VAE in Stable Diffusion?

It compresses images into a much smaller latent space (64×64×4 for 512×512 images) so diffusion runs cheaply there, then decodes the final latent back to pixels.

Q4. How does the text prompt control the image?

The prompt is encoded by a text encoder (CLIP/T5) and injected into the U-Net through cross-attention, where image latents are queries and text embeddings are keys and values; classifier-free guidance strengthens adherence.

Q5. What is classifier-free guidance?

Running the model with and without the prompt and extrapolating: ε = ε_uncond + w(ε_cond − ε_uncond). Higher w improves prompt alignment but reduces diversity.

Q6. DDPM vs DDIM?

DDPM is a stochastic Markov sampler needing hundreds to 1000 steps; DDIM is a deterministic non-Markovian sampler using the same model with 20–50 steps and allows inversion for editing.

Q7. GAN vs diffusion?

GANs generate in one pass (fast) but train unstably and can mode-collapse; diffusion trains stably with excellent quality and diversity but samples slowly (mitigated by better solvers and distillation).

Q8. What is ControlNet?

An add-on network that copies the diffusion encoder and conditions it on structural inputs like edges, pose or depth, connected via zero-initialised convolutions so it adds control without damaging the base model.

Q9. How does CLIP work?

Image and text encoders are trained contrastively so matching pairs have high cosine similarity and mismatched pairs low, creating a shared embedding space for zero-shot classification and retrieval.

Q10. How do multimodal LLMs see images?

A vision encoder converts image patches into embeddings, a projector maps them into the LLM's token space, and the LLM attends to them like text tokens; natively multimodal models train on all modalities jointly.

Q11. What is a vocoder?

The model that converts an acoustic representation like a mel-spectrogram into a raw audio waveform, e.g. WaveNet or HiFi-GAN.

Q12. How do you evaluate generative image models?

FID for realism and diversity against real images, CLIP score for prompt alignment, and human preference studies; plus safety and bias audits.

Q13. What responsible-AI controls would you put on a GenAI marketing tool?

Brand and safety guardrails, moderation filters, PII protection, copyright checks, watermarking/content credentials, human approval before publishing, audit logs, and bias reviews of generated imagery.

Q14. What is pass@k?

The probability that at least one of k generated code samples passes the unit tests; the standard metric for code generation benchmarks.

CHAPTER 6

RAG: Retrieval-Augmented Generation

Fundamentals • Architecture • Loaders, chunking, embeddings, vector DBs • Retrieval, re-ranking, query transformation • 18+ RAG types • Pipelines in code • Multi-document RAG & metadata filtering • Evaluation • Production • Enterprise use cases • Interview Q&A

6.1 What is RAG and Why Do We Need It?

Retrieval-Augmented Generation (RAG) is a technique where, before an LLM answers a question, the system first **retrieves** relevant information from an external knowledge source (documents, databases, APIs) and then **augments** the prompt with that information so the LLM can **generate** a grounded answer.

Think of it as an **open-book exam** for the LLM. Without RAG, the model answers from memory (its training weights). With RAG, the model is handed the relevant pages of the book first, then answers.

> **Key idea:** RAG = Retrieve (find relevant context) + Augment (put it into the prompt) + Generate (LLM writes the answer using that context).

6.1.1 Problems with a plain LLM that RAG solves

LLM limitation	What it means	How RAG fixes it
Knowledge cutoff	Model only knows data up to its training date.	Retrieve fresh documents at query time; just re-index new data.
Hallucination	Model confidently invents facts when it doesn't know.	Answer is grounded in retrieved text; prompt says 'answer only from context'.
No private data	Model never saw your company PDFs, tickets, policies.	Index private data in your own vector store; no retraining needed.
No citations	You can't verify where an answer came from.	Each retrieved chunk carries source metadata, so you can cite it.
Costly to update	Fine-tuning for every data change is slow and expensive.	Updating knowledge = adding/removing documents in the index.
Context window limits	You cannot paste 10,000 documents into one prompt.	Retrieve only the top few most relevant chunks.

6.1.2 A brief history (useful for interviews)

- **2020 – the original RAG paper** (Lewis et al., Facebook AI): combined a dense retriever (DPR) with a seq2seq generator (BART) and trained them together. The term 'RAG' comes from this paper.
- **REALM (Google, 2020)**: retrieval during pre-training. **FiD – Fusion-in-Decoder**: encodes each passage separately and fuses in the decoder.

- **2023 onward**: with ChatGPT/GPT-4, Gemini, Llama, RAG became the default pattern for enterprise LLM apps, usually *without* training the retriever: an off-the-shelf embedding model + vector DB + LLM.
- **Later evolution**: Advanced RAG → Modular RAG → Self-RAG, Corrective RAG, GraphRAG, Agentic RAG, Multimodal RAG.

6.1.3 RAG vs Fine-tuning vs Long Context vs Prompting

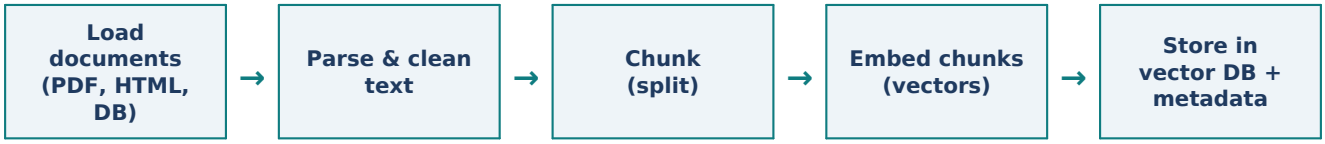
Approach	What changes	Best for	Weakness
Prompt engineering	Only the instructions	Format, tone, simple tasks	No new knowledge
RAG	The context given at runtime	Fresh/private/large factual knowledge, citations	Depends on retrieval quality; extra latency
Fine-tuning	Model weights	Style, domain language, output format, specific behaviour/skills	Expensive, knowledge goes stale, hard to cite, can forget
Long-context LLM	Paste everything into prompt	Small corpora (fits in window), one-off analysis	Cost per call, 'lost in the middle', doesn't scale to millions of docs
CAG (Cache-Augmented Gen.)	Pre-load whole small corpus into KV cache	Small, static knowledge base, very low latency	Limited by context size; no dynamic retrieval

> **Interview tip:** 'Fine-tuning teaches the model *how* to behave; RAG gives the model *what* to know.' In real systems they are often combined: a fine-tuned model that is also fed retrieved context.

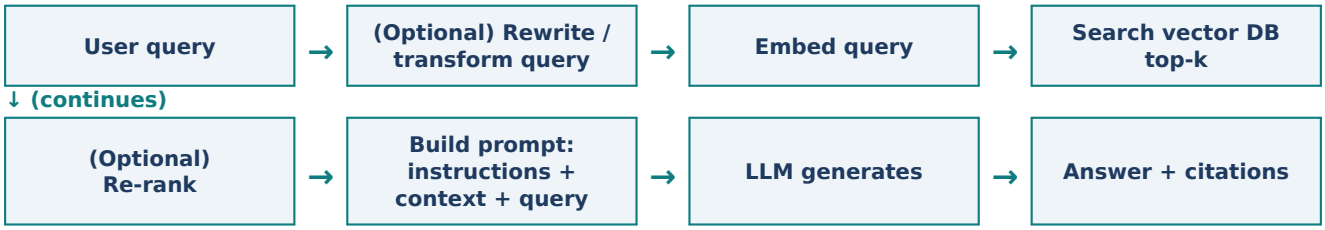
6.2 RAG Architecture: The Big Picture

Every RAG system has two pipelines: an **offline indexing pipeline** (run once / periodically to prepare the knowledge) and an **online query pipeline** (run on every user question).

6.2.1 Offline: Indexing (Ingestion) pipeline



6.2.2 Online: Query (Retrieval + Generation) pipeline



6.2.3 What actually happens, step by step (worked example)

Suppose an HR bot has the company leave-policy PDF indexed, and the user asks: "*How many sick leaves do I get?*"

1. **Indexing (done earlier):** the 40-page policy PDF was split into ~120 chunks of ~500 tokens. Each chunk was turned into a 768-dimensional vector by an embedding model and stored with metadata {source: 'leave_policy.pdf', page: 7}.

2. **Query embedding:** the question is converted into a vector using the *same* embedding model.
3. **Similarity search:** the vector DB compares the query vector with all chunk vectors (via an ANN index such as HNSW) and returns the top-k (e.g. 4) closest chunks by cosine similarity.
4. **Augmentation:** the chunks are inserted into a prompt template: 'Answer only using the context below. If not found, say you don't know. Context: ... Question: ...'.
5. **Generation:** the LLM reads the context and writes: 'You are entitled to 12 sick leaves per year (Leave Policy, p.7).'

Watch out: Retrieval quality caps answer quality. If the right chunk is not retrieved, even the best LLM cannot answer correctly — 'garbage in, garbage out'. Most RAG debugging is retrieval debugging.

6.3 Components of RAG — In Depth

Below is every building block of a RAG system, what it does, what happens inside it, and the key design choices.

6.3.1 Document Loaders & Parsing

Loaders read raw data from sources: PDFs, Word, HTML/web pages, Markdown, Confluence/SharePoint, Notion, databases, CSVs, emails, APIs. Parsing converts them into clean text + metadata.

- **Inside:** text extraction (pypdf, PyMuPDF, pdfplumber), OCR for scanned files (Tesseract), layout-aware parsing for tables/headers (Unstructured, LlamaParse, Docling, Azure Document Intelligence).
- **Metadata** captured here is gold: source, title, page, section, date, author, department, access permissions.
- **Cleaning:** remove headers/footers, page numbers, boilerplate, duplicate text, broken hyphenation; normalise whitespace and encoding.
- **Tables** are a common pain point: convert to Markdown/HTML or create a textual summary per table.

6.3.2 Chunking (Text Splitting)

Documents are too long to embed and retrieve as a whole, so we split them into smaller **chunks**. A chunk is the unit that gets embedded, retrieved and shown to the LLM.

Why chunk? (1) Embedding models have input limits (e.g. 512 tokens). (2) One vector for a 50-page document averages out all meaning and becomes useless for matching. (3) Smaller relevant chunks = less noise and fewer tokens in the prompt.

Strategy	How it works	When to use
Fixed-size	Split every N characters/tokens, with overlap.	Quick baseline; uniform text.
Recursive character	Try separators in order (\n\n, \n, '. ', ' ') so splits fall on paragraph/sentence boundaries. LangChain's RecursiveCharacterTextSplitter.	Default choice for most text.
Sentence-based	Split on sentences (nltk/spaCy) and group sentences up to a size.	Prose, articles.
Document-structure	Split by Markdown headers, HTML tags, code functions, PDF sections.	Docs with clear structure, code, manuals.
Semantic chunking	Embed sentences; start a new chunk where similarity between neighbouring sentences drops (topic change).	Mixed-topic docs; quality over speed.

Strategy	How it works	When to use
Agentic / LLM-based	LLM decides boundaries or extracts self-contained 'propositions'.	High-value corpora; expensive.
Parent-child (small-to-big)	Index small child chunks for precise matching, return larger parent chunk for context.	Need both precision and context.
Sentence-window	Index single sentences; at retrieval expand to ±k surrounding sentences.	Fine-grained QA.

Chunk size and overlap — the trade-off

- **Too small** (e.g. 100 tokens): precise matches but loses context; answer may be split across chunks.
- **Too large** (e.g. 2000 tokens): more context but diluted embeddings, noisy retrieval, more tokens/cost.
- **Typical starting point:** 256–1024 tokens with 10–20% overlap. Overlap ensures a sentence cut at a boundary still appears whole in one chunk.
- Always tune using an evaluation set — there is no universally best size.

6.3.3 Embeddings

An **embedding model** converts text into a dense vector of numbers (e.g. 384, 768, 1024, 1536 or 3072 dimensions) such that texts with similar *meaning* lie close together in vector space. 'How do I reset my password?' and 'Forgot login credentials' end up near each other even with no shared words.

What happens inside

- Text → tokenizer → transformer encoder (e.g. BERT-like) → token embeddings → **pooling** (mean pooling or [CLS] token) → one vector → usually L2-normalised.
- Trained with **contrastive learning**: pull (query, relevant passage) pairs together, push irrelevant pairs apart.
- This is a **bi-encoder**: query and documents are encoded independently, so document vectors can be pre-computed and stored. Fast, scalable.

Similarity measures

Cosine similarity = (A · B) / (‖A‖ ‖B‖), range −1 to 1; measures angle (direction). **Dot product** = A · B; equals cosine when vectors are normalised. **Euclidean (L2) distance**: smaller = more similar. Use the metric the embedding model was trained for.

Popular embedding models

Type	Examples
Open-source (sentence-transformers)	all-MiniLM-L6-v2 (384-d, fast), all-mpnet-base-v2 (768-d), BAAI bge-small/base/large, e5, gte, nomic-embed
API-based	OpenAI text-embedding-3-small/large, Google Gemini embedding models, Cohere embed, Voyage AI
Sparse / learned sparse	BM25 (classical), SPLADE (learned sparse)
Late interaction	ColBERT — keeps one vector per token, matches token-to-token (MaxSim)

Watch out: The query and the documents MUST be embedded with the same model. If you change the embedding model, you must re-embed the whole corpus. Some models (e5, bge) expect prefixes like 'query: ' and 'passage: '.

Choosing a model: check the **MTEB leaderboard** (retrieval tasks), language support, max input length, dimension (storage cost), latency, and whether data can leave your network.

6.3.4 Vector Database & Indexing

A vector database stores vectors + original text + metadata and answers the query: '*give me the k vectors closest to this one*'. Comparing against every vector (exact / brute-force kNN) is O(N) — too slow for millions — so we use **Approximate Nearest Neighbour (ANN)** indexes that trade a little accuracy for big speed gains.

Index	Idea	Notes
Flat (brute force)	Compare with every vector.	Exact, 100% recall; fine up to ~100k vectors.
IVF (Inverted File)	Cluster vectors with k-means; search only the nearest few clusters (nprobe).	Fast; recall depends on nprobe.
HNSW	Multi-layer 'small-world' graph; greedy walk from coarse top layer to fine bottom layer.	Most popular; high recall, fast; more memory.
PQ (Product Quantization)	Compress vectors into short codes.	Huge memory savings; some accuracy loss. Often IVF-PQ.

Vector store	Type / Notes
FAISS	Library by Meta (not a DB server); in-memory, very fast, many index types.
ChromaDB	Lightweight, easy local dev, persistent mode, metadata filters.
Pinecone	Fully managed cloud, serverless.
Weaviate / Qdrant / Milvus	Open-source servers; hybrid search, filtering, scaling.
pgvector	Vector extension for PostgreSQL — keep vectors next to relational data.
Elasticsearch / OpenSearch / Azure AI Search	Strong keyword (BM25) + vector = native hybrid search.

Metadata filtering: restrict search, e.g. *department* = '*HR*' *AND year* ≥ *2024*. Pre-filtering (filter then search) is accurate; post-filtering (search then filter) can return fewer than k results.

6.3.5 Retriever

The retriever takes the query and returns candidate chunks. Main strategies:

- **Dense retrieval**: embedding similarity (semantic meaning). Good for paraphrases; weak for exact IDs, codes, rare names.
- **Sparse / keyword retrieval (BM25, TF-IDF)**: exact term matching weighted by term rarity. Great for error codes, product SKUs, names; misses synonyms.
- **Hybrid retrieval**: both, merged (see Hybrid RAG).
- **MMR (Maximal Marginal Relevance)**: picks results that are relevant *and* diverse, avoiding 4 near-duplicate chunks. Score = λ·sim(q,d) − (1−λ)·max sim(d, already selected).
- **Similarity threshold**: only return chunks above a score; lets the system say 'I don't know' when nothing is relevant.
- **Top-k**: number of chunks to return. Higher k → better recall but more noise and tokens. Common pattern: retrieve 20–50, re-rank, keep 3–5.

BM25 in one line

BM25 scores a document higher when it contains the query terms often (term frequency, with saturation k1), when those terms are rare across the corpus (IDF), and normalises for document length (b). No neural network involved.

6.3.6 Query Transformation (pre-retrieval)

User queries are often short, vague, or conversational. Transforming them improves retrieval:

Technique	What happens
Query rewriting	LLM rewrites the query into a clear, search-friendly form (fix typos, expand acronyms).
Condensing (conversational)	Follow-up 'what about its price?' + chat history → standalone 'What is the price of iPhone 16?'
Multi-query	LLM generates 3–5 paraphrases; retrieve for each; union the results.
HyDE	LLM writes a hypothetical answer; embed that answer instead of the question (answer-to-answer matching).
Step-back prompting	Ask a more general question first ('What are the principles of X?') to fetch background context.
Query decomposition	Split complex question into sub-questions; retrieve for each; combine.
Query routing	Classify the query and send it to the right index/tool (HR docs vs IT docs vs SQL DB vs web).
Self-query / metadata extraction	LLM extracts filters from text: 'policies from 2024 about travel' → filter year=2024, query='travel policy'.

6.3.7 Re-ranking (post-retrieval)

The first-stage retriever is fast but approximate. A **re-ranker** re-scores the top candidates more accurately.

- **Cross-encoder**: feeds [query + chunk] *together* into one transformer, which outputs a relevance score. Because it sees both texts jointly (full attention across them), it is far more accurate than a bi-encoder — but too slow to run over millions of docs, so it is used only on the top 20–100.
- Examples: ms-marco-MiniLM cross-encoders, BAAI bge-reranker, Cohere Rerank, Jina reranker; or LLM-based reranking (ask the LLM to score/order).
- Classic two-stage pattern: **retrieve broadly (high recall) → re-rank precisely (high precision)**.

	Bi-encoder (embeddings)	Cross-encoder (re-ranker)
Input	Query and doc encoded separately	Query + doc encoded together
Pre-compute docs?	Yes — store vectors	No — must run per (query, doc) pair
Speed	Very fast (ANN search)	Slow
Accuracy	Good	Better
Role	Stage 1: candidate retrieval	Stage 2: re-ranking top-N

6.3.8 Context Construction & Compression

- **Deduplicate** overlapping chunks; **merge** adjacent chunks from the same document.
- **Ordering**: LLMs pay most attention to the beginning and end of the context ('**lost in the middle**' effect), so put the most relevant chunks first/last.
- **Contextual compression**: extract only relevant sentences from each chunk (LLM extractor or embedding filter) to save tokens and reduce noise.
- **Token budget**: make sure system prompt + context + history + question + expected answer fit in the context window.

6.3.9 Prompt Augmentation

The retrieved context is placed into a prompt template. A good RAG prompt:

- Defines the role and the task.
- Says to answer **only** from the provided context.
- Gives an explicit fallback: 'If the answer is not in the context, say I don't know.'
- Asks for citations (chunk IDs / source names).
- Clearly delimits context (e.g. XML-like tags or ### markers) to separate data from instructions.

Typical RAG prompt template

```
You are a helpful assistant for ACME HR.
Answer the question using ONLY the context below.
If the answer is not in the context, reply: "I don't know based on the available documents."
Cite sources as [source, page].

### CONTEXT
[1] (leave_policy.pdf, p.7) Employees are entitled to 12 sick leaves per year...
[2] (leave_policy.pdf, p.8) Unused sick leave cannot be carried forward...

### QUESTION
How many sick leaves do I get?
```

6.3.10 Generator (LLM)

The LLM reads the augmented prompt and produces the answer. Choices: GPT-4o/4.1, Gemini (Flash/Pro), Claude, Llama, Mistral, etc. Settings matter: a **low temperature (0–0.3)** for factual RAG; structured output (JSON) when needed; **streaming** to reduce perceived latency.

6.3.11 Supporting components

- **Memory / chat history**: for multi-turn conversations (condense question + history).
- **Caching**: embedding cache, retrieval cache, and **semantic cache** (reuse answers for semantically similar past questions).
- **Guardrails**: input filtering (prompt injection, PII), output checks (groundedness, toxicity).
- **Orchestration frameworks**: LangChain, LangGraph (stateful graphs, loops — ideal for CRAG/Self-RAG/agentic RAG), LlamaIndex (data/index-centric), Haystack, Semantic Kernel.
- **Observability**: LangSmith, Langfuse, Arize Phoenix — trace each retrieval and LLM call.
- **Evaluation**: RAGAS, DeepEval, TruLens (see Section 6.6).

6.4 Types of RAG — Explained One by One

RAG variants are not separate 'products' — they are **patterns** that improve a particular weakness of basic RAG (bad queries, missed keywords, irrelevant chunks, multi-step questions, relationships across documents, images, etc.). Real systems mix several of them. A useful mental map:

Category	Types	Fixes which weakness
Paradigms	Naive, Advanced, Modular	Overall maturity of the pipeline
Better retrieval	Hybrid, Multi-Query / RAG-Fusion, HyDE, Parent-Document / Sentence-Window, Contextual Retrieval, RAPTOR	Missing or poor-quality chunks
Self-checking / control flow	Corrective RAG, Self-RAG, Adaptive RAG, Iterative / Multi-hop RAG	Irrelevant context, hallucinations, complex questions
Autonomous	Agentic RAG, Multi-agent RAG	Need for tools, planning, multiple sources
Different data shapes	Graph RAG, Structured / SQL RAG, Multimodal RAG	Relationships, tables, images
Interaction	Conversational RAG	Follow-up questions with history

6.4.1 Naive (Simple / Vanilla) RAG

Core idea: The basic 'retrieve-then-read' pipeline: index → retrieve top-k by vector similarity → stuff into prompt → generate. No query rewriting, no re-ranking, no checks.



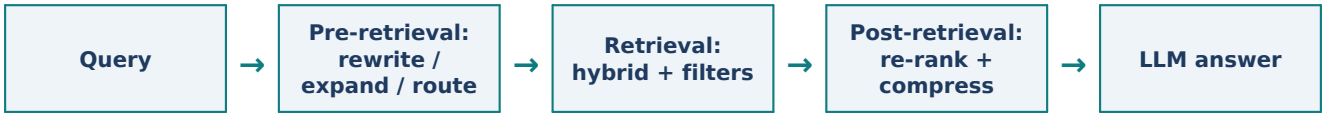
What happens inside

1. Documents are chunked with a fixed size and embedded once.
2. The raw user query is embedded with the same model.
3. Top-k nearest chunks by cosine similarity are returned (no matter how relevant they truly are).
4. Chunks are concatenated into the prompt ('stuff' strategy).
5. LLM generates in a single pass.

When to use	Pros	Cons
Prototypes, POCs, small clean corpora, simple factual Q&A.	Easy to build (~30 lines), cheap, low latency, easy to explain.	Low precision (irrelevant chunks), low recall (misses keyword matches), vague queries fail, no hallucination check, 'lost in the middle'.

6.4.2 Advanced RAG

Core idea: Naive RAG plus optimisations **before** retrieval (pre-retrieval) and **after** retrieval (post-retrieval) to improve quality.



What happens inside

1. **Indexing improvements**: better chunking (semantic/structural), richer metadata, cleaned data, sliding windows.
2. **Pre-retrieval**: query rewriting, expansion, HyDE, multi-query, routing, metadata extraction.
3. **Retrieval**: hybrid search, fine-tuned embeddings, metadata filtering.
4. **Post-retrieval**: cross-encoder re-ranking, context compression, deduplication, re-ordering.
5. **Generation**: tight prompts, citations, low temperature.

When to use	Pros	Cons
Most production RAG systems.	Much better precision/recall; still a linear, predictable pipeline.	More components to tune, more latency and cost.

6.4.3 Modular RAG

Core idea: Treat RAG as **interchangeable modules** (search, memory, routing, fusion, re-ranking, prediction, task adapters) that can be rearranged, including loops and branches — not only a fixed chain.



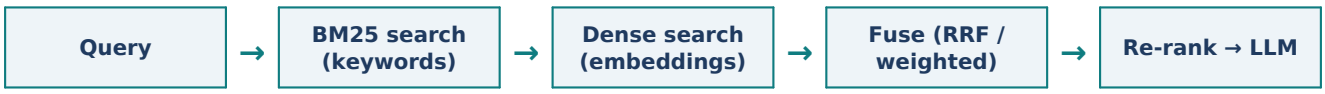
What happens inside

1. A router or orchestrator decides which modules to run for a query.
2. Modules can be swapped (e.g. Chroma → Qdrant, BM25 → SPLADE) without rewriting the system.
3. Patterns like retrieve→read, rewrite→retrieve→read, generate→read (HyDE), iterative retrieve-generate loops are all compositions of modules.
4. This is the paradigm that frameworks like LangGraph and LlamaIndex enable.

When to use	Pros	Cons
Large systems with many data sources and evolving requirements.	Flexible, maintainable, testable per module.	Design complexity; needs good orchestration and evaluation.

6.4.4 Hybrid RAG (Hybrid Search)

Core idea: Combine **sparse keyword search (BM25)** and **dense semantic search (embeddings)**, then fuse the results. Gets the exact-match strength of keywords and the meaning-match strength of vectors.



What happens inside

1. The same query runs in two retrievers in parallel.
2. BM25 catches exact tokens: 'error E1043', 'policy HR-17', product names, acronyms.
3. Dense search catches paraphrases: 'cannot log in' ≈ 'authentication failure'.
4. Results are merged with RRF or weighted score fusion; duplicates collapse.
5. Optionally a cross-encoder re-ranks the fused list.

Reciprocal Rank Fusion (RRF)

Scores from BM25 and cosine similarity are on different scales, so instead of adding raw scores we fuse **ranks**: RRF(d) = Σ over retrievers of 1 / (k + rank(d)), with k ≈ 60. A document ranked high by both retrievers wins. Alternative: **weighted score fusion** after normalising scores, e.g. 0.7·dense + 0.3·sparse (the 'alpha' parameter in Weaviate etc.).

RRF in Python

```
def rrf(result_lists, k=60):
    scores = {}
    for results in result_lists:                    # e.g. [bm25_ids, dense_ids]
        for rank, doc_id in enumerate(results, start=1):
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)
```

When to use	Pros	Cons
Enterprise docs, technical/IT tickets, legal, e-commerce search — anywhere with IDs, codes, jargon.	Consistently better recall than either alone; robust to query style.	Two indexes to maintain; weight tuning; slightly more latency.

In LangChain: *EnsembleRetriever([bm25_retriever, vector_retriever], weights=[0.4, 0.6])* implements weighted RRF.

6.4.5 Multi-Query RAG & RAG-Fusion

Core idea: One query phrasing may miss relevant chunks. Generate several reformulations with an LLM, retrieve for each, then merge. **RAG-Fusion** = multi-query + RRF ranking of the merged results.



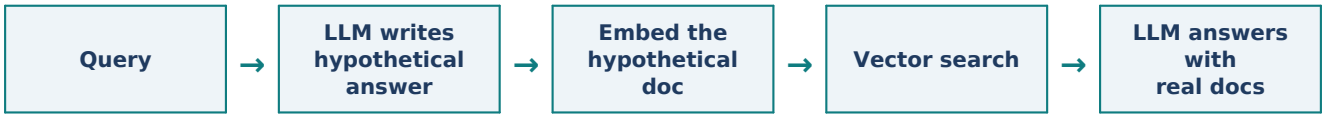
What happens inside

1. LLM prompt: 'Generate 4 different versions of this question to retrieve relevant documents.'
2. Each variant is embedded and searched independently (can run in parallel).
3. Results are unioned (multi-query) or rank-fused with RRF (RAG-Fusion), so chunks found by many variants rise to the top.
4. Top fused chunks go to the LLM.

When to use	Pros	Cons
Ambiguous or short queries; users who phrase things differently from the docs.	Higher recall; covers multiple perspectives.	Extra LLM call + N retrievals → more latency/cost; can drift off-topic.

6.4.6 HyDE (Hypothetical Document Embeddings)

Core idea: Questions and answers look different in vector space. So ask the LLM to write a **fake (hypothetical) answer** first, embed that, and search with it — answer-to-answer similarity is stronger than question-to-answer.



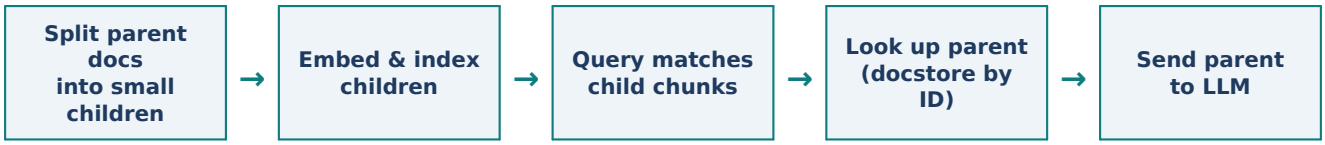
What happens inside

1. Query: 'What causes HNSW memory growth?' → LLM drafts a paragraph that sounds like documentation.
2. The draft may contain factual errors — that is fine; we only use it to find *similar real* documents.
3. The draft's embedding lands closer to real relevant passages than the short question would.
4. The final answer is generated only from the real retrieved documents.

When to use	Pros	Cons
Short queries, zero-shot domains without fine-tuned embeddings.	Simple, often big recall boost.	Extra LLM call; if the LLM knows nothing about the domain, the fake answer can mislead retrieval.

6.4.7 Parent-Document, Small-to-Big & Sentence-Window RAG

Core idea: Small chunks give precise matches; big chunks give enough context. Decouple them: **search on small pieces, return the bigger piece**.



What happens inside

1. Each document is split into large **parent** chunks (e.g. 2000 tokens), each split into small **child** chunks (e.g. 200 tokens).
2. Only children are embedded; each child stores its parent_id in metadata.
3. At query time the precise child match is found, then the parent is fetched from a docstore and passed to the LLM.
4. **Sentence-window** variant: index individual sentences and return a window of ±k sentences around the hit.
5. **Auto-merging** (LlamaIndex): if many children of the same parent are retrieved, replace them with the parent.

When to use	Pros	Cons
Long technical docs, contracts, manuals where an answer needs surrounding context.	Precise retrieval + rich context.	Needs a docstore + ID mapping; more tokens per retrieved hit.

6.4.8 Contextual Retrieval (Contextual RAG)

Core idea: A chunk like 'Revenue grew 3% over the previous quarter' has lost its context (which company? which quarter?). Before embedding, use an LLM to **prepend a short context** to every chunk explaining where it comes from.



What happens inside

1. For each chunk, the LLM sees the whole document and writes e.g. 'This chunk is from ACME's Q2 2024 10-Q filing, discussing revenue.'
2. This context is prepended and the combined text is embedded (contextual embeddings) and BM25-indexed (contextual BM25).

3. Prompt caching keeps the cost of passing the full document many times manageable.
4. Typically combined with hybrid search and re-ranking (popularised by Anthropic).

When to use	Pros	Cons
Corpora where chunks are ambiguous out of context: financial reports, legal docs, long manuals.	Significantly fewer retrieval failures; works with any retriever.	Indexing cost (one LLM call per chunk); slower ingestion.

6.4.9 RAPTOR (Hierarchical / Tree-Summarisation RAG)

Core idea: Build a **tree of summaries**: cluster chunks, summarise each cluster, cluster the summaries, summarise again... Retrieval can then hit both detailed leaves and high-level summaries.



What happens inside

1. Chunks are embedded and clustered (e.g. Gaussian Mixture Models).
2. Each cluster is summarised by an LLM → new 'summary nodes', which are embedded too.
3. The process repeats to build higher abstraction levels.
4. At query time, search the collapsed tree (all levels) so broad questions match summaries and specific ones match leaves.

When to use	Pros	Cons
Questions needing a holistic understanding of long documents ('What is the main theme of this book?').	Handles both broad and specific questions.	Expensive indexing; must rebuild summaries when data changes.

6.4.10 Conversational RAG (RAG with Memory)

Core idea: In chat, follow-ups like 'and for interns?' cannot be retrieved on their own. Use chat history to create a **standalone question** before retrieval.



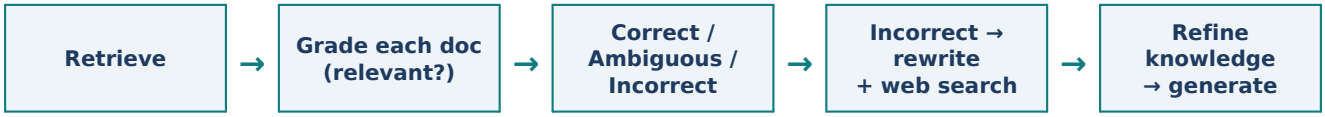
What happens inside

1. History: 'How many sick leaves do employees get?' → 'And for interns?'
2. Condense step → 'How many sick leaves do interns get?'
3. Retrieval uses the standalone question; generation sees context + recent history.
4. Memory strategies: last N turns (window), summary memory, or retrieval over past turns.
5. In LangChain: create_history_aware_retriever; in LangGraph: messages in state + a checkpointer (thread_id).

When to use	Pros	Cons
Chatbots, support assistants, any multi-turn interface.	Natural conversations; correct retrieval on follow-ups.	Extra LLM call; long histories cost tokens; condensing can lose intent.

6.4.11 Corrective RAG (CRAG)

Core idea: Don't blindly trust retrieval. A **retrieval evaluator** grades the retrieved documents; if they are poor, the system **corrects** itself — rewriting the query and/or falling back to web search — before generating.



What happens inside

1. Retrieve top-k chunks as usual.
2. A lightweight evaluator (small model or LLM with a yes/no structured output) scores each chunk's relevance.
3. **Correct** (confident relevant): keep and refine — strip irrelevant sentences ('decompose-then-recompose').
4. **Incorrect**: discard and go to web search (e.g. Tavily) with a rewritten query.
5. **Ambiguous**: combine refined internal docs + web results.
6. Generate the final answer from the corrected knowledge.

CRAG in LangGraph (skeleton)

Corrective RAG graph

```
class State(TypedDict): question: str; docs: list; web_needed: bool; answer: str

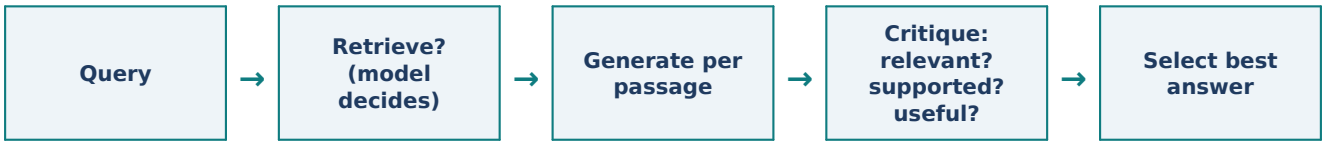
def retrieve(s): return {"docs": retriever.invoke(s["question"])}
def grade(s):
    good = [d for d in s["docs"]
            if grader.invoke({"q": s["question"], "doc": d.page_content}).score == "yes"]
    return {"docs": good, "web_needed": len(good) == 0}
def rewrite(s): return {"question": rewriter.invoke(s["question"])}
def web_search(s): return {"docs": s["docs"] + tavily.invoke(s["question"])}
def generate(s):
    return {"answer": rag_chain.invoke({"context": s["docs"], "question": s["question"]})}

g = StateGraph(State)
nodes = {"retrieve": retrieve, "grade": grade, "rewrite": rewrite, "web": web_search, "generate": generate}
for name, fn in nodes.items(): g.add_node(name, fn)
g.add_edge(START, "retrieve"); g.add_edge("retrieve", "grade")
g.add_conditional_edges("grade", lambda s: "rewrite" if s["web_needed"] else "generate")
g.add_edge("rewrite", "web"); g.add_edge("web", "generate"); g.add_edge("generate", END)
app = g.compile()
```

When to use	Pros	Cons
When the knowledge base may be incomplete; open-domain assistants; high-accuracy Q&A.	Robust to bad retrieval; fewer hallucinations; graceful fallback.	Extra grading calls (latency, cost); web data can be noisy/untrusted.

6.4.12 Self-RAG (Self-Reflective RAG)

Core idea: The model itself decides **whether to retrieve**, and **critiques** both the retrieved passages and its own output using special **reflection tokens**, then picks the best supported answer.



What happens inside

1. Original paper (Asai et al., 2023) trains an LM to emit reflection tokens: **[Retrieve]** (needed?), **[IsREL]** (is passage relevant?), **[IsSUP]** (is the output supported by the passage?), **[IsUSE]** (is the response useful?).
2. If retrieval is not needed (e.g. 'write a poem'), it answers directly.
3. Otherwise it generates candidate segments for each passage and scores them with the critique tokens; the best-scoring segment wins.
4. In practice (LangGraph implementations) the reflection tokens are replaced by LLM **graders**: document-relevance grader → generate → **hallucination grader** (is the answer grounded?) → **answer grader** (does it address the question?) → loop back to rewrite/regenerate if not.

When to use	Pros	Cons
High-stakes answers where faithfulness must be verified (legal, medical, finance, compliance).	Self-verification, adaptive retrieval, fewer hallucinations.	Many LLM calls, higher latency; loops need max-iteration limits.

Interview tip: CRAG vs Self-RAG: CRAG focuses on *fixing the retrieved knowledge* (grade docs → fall back to web). Self-RAG focuses on *reflecting on the whole process* — whether to retrieve, and whether the generated answer is supported and useful.

6.4.13 Adaptive RAG

Core idea: Not every question needs the same pipeline. A **query classifier/router** estimates complexity and picks the strategy: no retrieval, single-step RAG, or multi-step RAG (or web search).



What happens inside

1. A small classifier or LLM with structured output labels the query (e.g. A: no retrieval, B: single-step, C: multi-step).
2. 'What is 2+2?' → answer directly; 'What is our refund policy?' → single retrieval; 'Compare refund policies of 2022 and 2024 and explain the impact' → iterative retrieval.
3. Often combined with routing to different sources: vector store vs web search.
4. Popular LangGraph example: router → (vectorstore → CRAG/Self-RAG checks) or (web search).

When to use	Pros	Cons
Mixed traffic with both trivial and complex questions.	Saves cost/latency on simple queries; better on hard ones.	Classifier errors route queries wrongly; more design work.

6.4.14 Iterative / Multi-hop RAG

Core idea: Some questions need facts that depend on other facts: 'Who is the manager of the person who wrote the leave policy?' Retrieve → reason → retrieve again using what was learned, until enough information is gathered.



What happens inside

1. Decompose: Hop 1 'Who wrote the leave policy?' → 'Priya'.
2. Hop 2 uses the answer: 'Who is Priya's manager?' → 'Rahul'.
3. Techniques: IRCoT (interleave retrieval with chain-of-thought), ITER-RETGEN (use the generation to retrieve again), FLARE (retrieve when the model's next sentence has low-confidence tokens).
4. Stop condition: model judges info sufficient or max hops reached.

When to use	Pros	Cons
Multi-hop QA, research questions, comparisons across documents.	Answers questions single-shot RAG cannot.	Latency grows with hops; error in hop 1 propagates.

6.4.15 Agentic RAG

Core idea: An **LLM agent** (ReAct: Reason + Act loop) treats retrieval as one of several **tools** and autonomously plans: which source to query, whether to search again, whether to call a calculator/SQL/API, and when it is done.



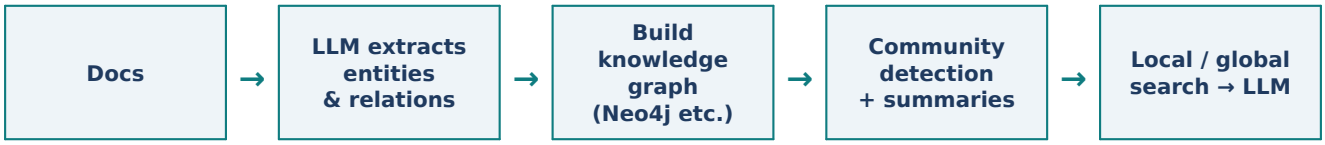
What happens inside

1. Tools are described to the LLM (name, description, arguments); e.g. search_hr_docs, search_it_docs, web_search, run_sql.
2. Agent loop: Thought → Action (tool call) → Observation → Thought ... (ReAct).
3. It can reformulate queries, query multiple indexes, cross-check, and decide when it has enough.
4. **Multi-agent RAG**: a supervisor routes to specialist agents (researcher, retriever, SQL agent, writer, reviewer).
5. Built with LangGraph (create_react_agent / custom graphs), LlamaIndex agents, CrewAI, AutoGen, OpenAI/Google agent SDKs.

When to use	Pros	Cons
Many heterogeneous sources, open-ended tasks, research assistants, customer support needing actions.	Most flexible and capable; handles complex, multi-source tasks.	Less predictable, higher cost/latency, harder to evaluate/debug; needs guardrails and step limits.

6.4.16 Graph RAG (Knowledge-Graph RAG)

Core idea: Vector search finds similar *text* but not *relationships*. Graph RAG extracts **entities and relationships** into a knowledge graph and retrieves by traversing it — great for 'connect the dots' and global questions.



What happens inside

1. Indexing: an LLM extracts triples like (Priya) —[MANAGES]→ (Payments Team), (Payments Team) —[OWNS]→ (Refund API).
2. Stored in a graph DB (Neo4j, Memgraph) or in-memory graph.

3. Microsoft GraphRAG additionally clusters the graph into **communities** (Leiden algorithm) and pre-writes a summary for each community.
4. **Local search**: find entities in the query, traverse their neighbourhood, collect related text.
5. **Global search**: map-reduce over community summaries to answer 'What are the main themes across all incident reports?'
6. Alternative: Text-to-Cypher — LLM writes a graph query.

When to use	Pros	Cons
Highly connected data: org charts, supply chains, fraud, research literature, incident root-cause analysis, holistic 'summarise everything' questions.	Multi-hop reasoning, explainable paths, global understanding.	Expensive and complex indexing; extraction errors; harder to update.

6.4.17 Structured / SQL RAG (Text-to-SQL)

Core idea: When knowledge lives in tables (sales, orders, tickets), similarity search is the wrong tool. The LLM **generates a query** (SQL/Pandas/Cypher), executes it, and explains the result.



What happens inside

1. Relevant table schemas, column descriptions and few-shot example queries are retrieved (schema RAG) and placed into the prompt.
2. LLM generates SQL: 'SELECT COUNT(*) FROM tickets WHERE priority='P1' AND created_at >= ...'.
3. Query is validated and run with a **read-only** user; errors can be sent back for self-correction.
4. Result rows are summarised in natural language. Hybrid systems route between SQL and document RAG.

When to use	Pros	Cons
Analytics questions, aggregations, numbers ('How many', 'total', 'average').	Exact numeric answers; uses existing databases.	SQL errors, security risk (SQL injection, data access) — needs strict permissions.

6.4.18 Multimodal RAG

Core idea: Retrieve and reason over **images, tables, charts, audio, video** — not only text.



What happens inside

1. Option A: **shared embedding space** (CLIP, SigLIP, multimodal embedding models) — embed images and text into the same space and search across them.
2. Option B: **summarise to text** — a vision LLM captions every image/table; the caption is embedded, but the original image is returned to the generator (multi-vector retriever).
3. Option C: **page-image retrieval** (e.g. ColPali) — embed screenshots of whole PDF pages; no OCR/parsing needed.

4. A multimodal LLM reads the retrieved text + images to answer.

When to use	Pros	Cons
Manuals with diagrams, slide decks, invoices, medical images, product catalogues.	Uses information plain-text RAG throws away.	Expensive storage and inference; harder evaluation.

6.4.19 Other names you may hear

Name	One-line meaning
Speculative RAG	A small 'drafter' model writes multiple drafts from different document subsets; a large 'verifier' model picks the best — faster and accurate.
Branched / Routing RAG	Router sends the query to one of several specialised indexes or pipelines.
Federated RAG	Retrieves from multiple distributed/private data sources without centralising data.
Long-context RAG / LongRAG	Retrieve larger units (whole sections/documents) because modern LLMs have big windows.
Memory RAG	Long-term memory of user facts stored in a vector store and retrieved per conversation.
Cache-Augmented Generation (CAG)	Alternative to RAG: preload a small knowledge base into the model's KV cache; no retrieval step.

6.4.20 Summary comparison of RAG types

Type	Key mechanism	Extra LLM calls	Complexity	Best for
Naive	Embed → top-k → generate	0	Low	Prototypes
Advanced	Pre + post retrieval optimisation	0–2	Medium	Production baseline
Modular	Swappable modules, routing, loops	Varies	Medium–High	Large systems
Hybrid	BM25 + dense + RRF	0	Low–Medium	IDs, codes, jargon
Multi-Query / Fusion	N query variants + RRF	1	Low	Vague queries
HyDE	Embed a fake answer	1	Low	Short queries
Parent-Document	Search small, return big	0	Low–Medium	Long docs
Contextual	LLM context prepended per chunk	1 per chunk (index)	Medium	Ambiguous chunks
RAPTOR	Tree of cluster summaries	Many (index)	High	Holistic questions
Conversational	Condense with history	1	Low	Chatbots
Corrective (CRAG)	Grade docs → web fallback	k + 1–2	Medium	Incomplete KB
Self-RAG	Reflect: retrieve? relevant? supported?	Several	High	High-stakes accuracy
Adaptive	Route by complexity	1	Medium	Mixed traffic
Multi-hop	Iterative retrieve-reason	Several	High	Multi-step questions
Agentic	Agent + tools (ReAct)	Many	High	Multi-source tasks
Graph	Knowledge graph traversal	Many (index)	High	Relationships, global Qs

Type	Key mechanism	Extra LLM calls	Complexity	Best for
SQL / Structured	Text-to-SQL + execute	1–2	Medium	Numeric / tabular
Multimodal	Image/table embeddings	Varies	High	Visual docs

6.5 Hands-on: Building RAG in Python

Three compact implementations, from scratch to framework, using tools you already work with (sentence-transformers, ChromaDB, Gemini, LangChain).

6.5.1 Naive RAG from scratch (no framework)

rag_basic.py

```
# pip install sentence-transformers chromadb google-genai
import chromadb
from sentence_transformers import SentenceTransformer
from google import genai

embedder = SentenceTransformer("all-MiniLM-L6-v2")
db = chromadb.PersistentClient(path="./chroma_db")
col = db.get_or_create_collection("docs", metadata={"hnsw:space": "cosine"})
llm = genai.Client()                # reads GEMINI_API_KEY / GOOGLE_API_KEY from env

def chunk(text, size=500, overlap=50):
    return [text[i:i + size] for i in range(0, len(text), size - overlap)]

def index(doc_id, text, source):
    chunks = chunk(text)
    col.add(ids=[f"{doc_id}_{i}" for i in range(len(chunks))], documents=chunks,
            embeddings=embedder.encode(chunks).tolist(),
            metadatas=[{"source": source}] * len(chunks))

def retrieve(query, k=4):
    res = col.query(query_embeddings=embedder.encode([query]).tolist(), n_results=k)
    return list(zip(res["documents"][0], res["metadatas"][0], res["distances"][0]))

def answer(query):
    hits = retrieve(query)
    context = "\n\n".join(f"[{i+1}] ({m['source']}) {d}" for i, (d, m, _) in enumerate(hits))
    prompt = f"""Answer ONLY from the context. If not found, say "I don't know". Cite [n].
Context:
{context}

Question: {query}"""
    return llm.models.generate_content(model="gemini-2.5-flash", contents=prompt).text
```

6.5.2 Hybrid retrieval (BM25 + dense) with RRF

hybrid.py

```
# pip install rank_bm25
from rank_bm25 import BM25Okapi
import numpy as np

corpus = [...]                                          # list of chunk strings
bm25 = BM25Okapi([c.lower().split() for c in corpus])
doc_vecs = embedder.encode(corpus, normalize_embeddings=True)

def hybrid_search(query, k=5, pool=20, rrf_k=60):
    sparse = np.argsort(bm25.get_scores(query.lower().split()))[::-1][:pool]
    dense = np.argsort(doc_vecs @ embedder.encode(query, normalize_embeddings=True))[::-1][:pool]
    scores = {}
    for ranking in (sparse, dense):
        for rank, idx in enumerate(ranking, 1):
            scores[idx] = scores.get(idx, 0) + 1 / (rrf_k + rank)
    return [corpus[i] for i in sorted(scores, key=scores.get, reverse=True)[:k]]
```

6.5.3 Re-ranking with a cross-encoder

rerank.py

```
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank(query, docs, top_n=3):
    scores = reranker.predict([(query, d) for d in docs])
    return [d for _, d in sorted(zip(scores, docs), reverse=True)[:top_n]]

final_docs = rerank(q, hybrid_search(q, k=20))
```

6.5.4 The same pipeline with LangChain (LCEL)

rag_langchain.py

```
# pip install langchain langchain-community langchain-chroma langchain-huggingface
#             langchain-google-genai pypdf
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

docs = PyPDFLoader("policy.pdf").load()
chunks = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=100).split_documents(docs)
emb = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vs = Chroma.from_documents(chunks, emb, persist_directory="db")
retriever = vs.as_retriever(search_type="mmr", search_kwargs={"k": 4, "fetch_k": 20})

prompt = ChatPromptTemplate.from_template(
    "Answer only from context. If unknown, say so.\nContext:\n{context}\n\nQuestion: {question}")
fmt = lambda ds: "\n\n".join(f"[{d.metadata.get('page')}] {d.page_content}" for d in ds)
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)

chain = ({"context": retriever | fmt, "question": RunnablePassthrough()}
         | prompt | llm | StrOutputParser())
print(chain.invoke("How many sick leaves do I get?"))
```

Watch out: Libraries change fast — package names and model IDs (e.g. Gemini model versions) get updated. If an import fails, check the current docs for the package version you installed.

6.6 Evaluating RAG

You must evaluate the **retriever** and the **generator** separately — otherwise you cannot tell which one is failing.

6.6.1 Retrieval metrics (need labelled relevant chunks per query)

Metric	Meaning	Formula / intuition
Hit Rate@k	Did at least one relevant chunk appear in top-k?	Fraction of queries with ≥1 hit
Recall@k	Of all relevant chunks, how many did we retrieve in top-k?	relevant retrieved / total relevant
Precision@k	Of the k retrieved, how many are relevant?	relevant retrieved / k
MRR	How high is the first relevant result?	mean of 1 / rank of first relevant
nDCG@k	Rewards relevant results at higher ranks; supports graded relevance	DCG / ideal DCG
MAP	Mean of average precision across queries	Order-aware precision

6.6.2 Generation / end-to-end metrics (RAG Triad and RAGAS)

Metric	Question it answers	Compares
Faithfulness / Groundedness	Is every claim in the answer supported by the retrieved context? (hallucination check)	Answer ↔ Context
Answer relevancy	Does the answer actually address the question?	Answer ↔ Question
Context precision	Are the relevant chunks ranked at the top? Is the context free of noise?	Context ↔ Question / ground truth
Context recall	Does the context contain all the info needed for the ground-truth answer?	Context ↔ Ground truth
Answer correctness	Is the answer factually equal to the reference answer?	Answer ↔ Ground truth

> **Key idea: RAG Triad** (TruLens): Context relevance (query↔context), Groundedness (context↔answer), Answer relevance (query↔answer). If all three are high, the system is unlikely to hallucinate.

6.6.3 How to evaluate in practice

1. Build a **golden dataset**: 50–200 real questions with reference answers and source chunks (manually, or synthetically generated with an LLM and then reviewed).
2. Run retrieval metrics first; fix chunking/embeddings/hybrid/reranking until recall@k is good.
3. Run generation metrics with **LLM-as-a-judge** (RAGAS, DeepEval, TruLens, LangSmith evaluators).
4. Track latency (p50/p95), cost per query, and token usage.
5. Add **human evaluation** and user feedback (thumbs up/down) in production; feed failures back into the test set.
6. Run evaluation in CI on every change (prompt, chunk size, model) — regression testing for RAG.

RAGAS sketch (API varies by version)

```
# pip install ragas datasets
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall
from datasets import Dataset

data = Dataset.from_dict({"question": qs, "answer": answers,
                          "contexts": contexts, "ground_truth": refs})
print(evaluate(data, metrics=[faithfulness, answer_relevancy, context_precision, context_recall]))
```

6.7 Failure Modes, Fixes & Production Concerns

6.7.1 Common failure points and fixes

Symptom	Likely cause	Fix
Right doc exists but not retrieved	Bad chunking, weak embeddings, vocabulary mismatch	Hybrid search, better chunking, query rewriting/HyDE, domain/fine-tuned embeddings
Retrieved but ranked low (cut off by top-k)	First-stage retriever imprecise	Retrieve more (k=20–50) + cross-encoder rerank
Fails on IDs, codes, names	Dense-only retrieval	Add BM25 (hybrid), metadata filters
Answer ignores the context / hallucinates	Weak prompt, noisy context, high temperature	Strict grounding prompt, compression, temp 0, faithfulness checks (Self-RAG)
Answer is incomplete	Info spread across chunks; k too small	Parent-document retrieval, larger k, multi-hop / decomposition
Correct chunk but wrong answer	Lost in the middle; too much context	Reorder, fewer but better chunks
Outdated answers	Stale index; old versions of docs	Incremental re-indexing, date metadata, filter/boost recent
Follow-up questions fail	No history in retrieval	Conversational RAG (condense question)
Wrong format	No format instructions	Structured output / JSON schema
Slow responses	Many LLM calls, large k, reranker	Caching, streaming, smaller models for grading, async/parallel retrieval

6.7.2 Production checklist

- **Data pipeline**: incremental ingestion (only new/changed docs, via content hashes), deletion handling, versioning, scheduled re-index.
- **Security / access control**: store permissions (ACLs) as metadata and filter at query time so users only retrieve documents they are allowed to see. Never rely on the LLM to hide data.
- **Prompt injection**: retrieved documents may contain instructions ('ignore previous instructions...'). Delimit context, tell the model to treat it as data, sanitise sources, limit tool permissions.
- **PII & compliance**: mask sensitive data before indexing; data residency for embedding/LLM APIs.
- **Latency**: stream tokens, parallelise retrievers, cache embeddings and frequent answers (semantic cache), use smaller models for routing/grading.
- **Cost**: fewer/shorter chunks, cheaper embedding model, batch embedding, prompt caching.
- **Scale**: ANN index choice (HNSW/IVF-PQ), sharding, managed vector DB.

- **Observability**: log query, retrieved chunk IDs + scores, prompt, answer, latency, cost; trace with LangSmith/Langfuse/Phoenix.
- **Fallbacks**: 'I don't know' path, human handoff, web search fallback.
- **Feedback loop**: collect user feedback, review failures weekly, add to evaluation set.

6.8 Multi-Document RAG, Metadata Filtering & Enterprise Applications

Multi-document RAG

Real knowledge bases contain thousands of documents of different types, owners, versions and permissions. Problems unique to multi-document RAG: answers spread across documents, conflicting versions, near-duplicate content, and questions scoped to one document ('in the 2024 travel policy…').

Technique	How it helps
Rich metadata at ingestion	doc_id, title, source, section, page, doc_type, department, version, effective_date, language, ACL groups
Metadata filtering	restrict search: where={'department': 'HR', 'year': {'$gte': 2024}}; pre-filter inside the ANN search
Self-query retriever	LLM converts natural-language constraints into filters + a semantic query
Document-level routing	summary index first (pick relevant documents) → chunk index within them (two-stage / hierarchical retrieval)
Per-document diversification	MMR or cap chunks per document so one long doc doesn't dominate top-k
Versioning	keep only the latest effective version active; filter is_latest=True; or show the effective date in citations
Comparison questions	decompose: retrieve for each document separately, then compare (multi-query / sub-question engine)
Deduplication	hash or embedding similarity to drop near-duplicate chunks at ingestion

metadata_filtering.py

```
import chromadb
col = chromadb.PersistentClient("db").get_or_create_collection("kb", metadata={"hnsw:space": "cosine"})
col.add(ids=ids, documents=chunks, embeddings=vecs,
        metadatas=[{"source": "travel_policy.pdf", "dept": "HR", "year": 2024, "page": p, "acl": "all_staff"} for p

def search(query, user_groups, dept=None, k=5):
    where = {"$and": [{"acl": {"$in": user_groups}}] + ([{"dept": dept}] if dept else [])}
    return col.query(query_embeddings=embed([query]), n_results=k, where=where)

# LangChain equivalent:
retriever = vectorstore.as_retriever(search_kwargs={"k": 5, "filter": {"dept": "HR"}})
```

Conversational RAG with LangGraph (memory + grading)

conversational_rag_graph.py

```
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver
from langchain_core.messages import HumanMessage, AIMessage

class State(TypedDict):
    messages: Annotated[list, add_messages]; question: str; docs: list

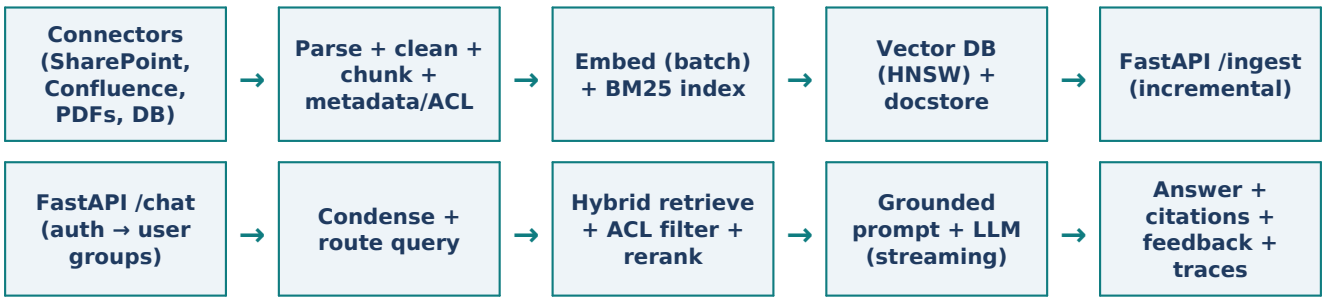
def condense(s):        # follow-up + history → standalone question
    hist = "\n".join(f"{m.type}: {m.content}" for m in s["messages"][-6:])
    return {"question": llm.invoke(f"Rewrite the last user message as a standalone question.\n{hist}").content}
def retrieve(s): return {"docs": retriever.invoke(s["question"])}
def generate(s):
    ctx = "\n\n".join(f"[{d.metadata['source']} p{d.metadata.get('page')}] {d.page_content}" for d in s["docs"])
    ans = llm.invoke(f"Answer ONLY from context, cite sources, else say you don't know.\nContext:\n{ctx}\nQ: {s['q
    return {"messages": [AIMessage(ans.content)]}

g = StateGraph(State)
g.add_node("condense", condense); g.add_node("retrieve", retrieve); g.add_node("generate", generate)
g.add_edge(START, "condense"); g.add_edge("condense", "retrieve"); g.add_edge("retrieve", "generate"); g.add_edge(
app = g.compile(checkpointer=MemorySaver())
cfg = {"configurable": {"thread_id": "user-42"}}          # memory is kept per thread
app.invoke({"messages": [HumanMessage("How many sick leaves do employees get?")]}, cfg)
app.invoke({"messages": [HumanMessage("And for interns?")]}, cfg)
```

Real-world use cases

Use case	Description	Design notes
Enterprise search	natural-language search over SharePoint, Confluence, wikis, tickets with answers + links	hybrid search, ACL filtering, connectors, freshness, click feedback
Document Q&A	ask questions over contracts, policies, manuals, research papers	layout-aware parsing, tables, page citations, parent-document retrieval
Knowledge assistants / copilots	HR bot, IT helpdesk bot, sales enablement, developer docs assistant	conversational memory, routing by domain, tools for actions (create ticket), human handoff
Customer support	answer customers from FAQs, product docs, order DB	RAG + SQL tool, tone guardrails, escalation, CSAT feedback loop
Incident / IT-ops intelligence	suggest resolutions from similar past incidents and runbooks	BM25 for error codes, time filters, GraphRAG for service dependencies
Compliance & legal	check documents against regulations and policies	high-precision retrieval, Self-RAG verification, audit trails
Healthcare / research	literature and guideline QA	domain embeddings, strict citations, safety disclaimers

Hands-on: enterprise RAG application blueprint



project structure

```
rag_app/
├── app/
│   ├── main.py            # FastAPI: /ingest, /chat (StreamingResponse), /feedback, /health
│   ├── config.py          # settings via pydantic-settings / .env
│   ├── ingestion/         # loaders.py, chunking.py, embed.py, upsert.py (content-hash based)
│   ├── retrieval/         # hybrid.py (BM25 + dense + RRF), rerank.py, filters.py (ACL)
│   ├── graph/             # state.py, nodes.py (condense, retrieve, grade, generate), graph.py
│   ├── prompts/           # versioned prompt templates
│   └── observability.py   # tracing (LangSmith/Langfuse), token & latency logging
├── eval/                  # golden_set.jsonl, run_ragas.py (runs in CI)
├── tests/                 # pytest: unit tests for chunking, retrieval, API contract
├── Dockerfile  docker-compose.yml (api + vector DB)  requirements.txt
```

6.9 Interview Questions & Answers

Answers are written to be **speakable** in an interview: start with a one-line definition, then the 'why', then an example or trade-off.

6.9.1 Basics

Q1. What is RAG?

RAG is an architecture where we retrieve relevant information from an external knowledge source and add it to the LLM's prompt so the answer is grounded in that information. It has an indexing pipeline (load, chunk, embed, store) and a query pipeline (embed query, retrieve, augment prompt, generate).

Q2. Why use RAG instead of just asking the LLM?

LLMs have a knowledge cutoff, don't know private data, and hallucinate. RAG gives them fresh, domain-specific, citable context at query time without retraining.

Q3. RAG vs fine-tuning — when do you choose which?

RAG for knowledge that is large, private or frequently changing, and when citations are needed. Fine-tuning for behaviour: style, format, domain language, or a specialised task. They can be combined. Updating RAG is just re-indexing; updating fine-tuning requires retraining.

Q4. With 1M+ token context windows, is RAG still needed?

Yes, for most real systems. Corpora are often far larger than any window; long prompts cost more and are slower on every call; models suffer 'lost in the middle'; and RAG gives access control, freshness and citations. Long context does reduce the need for very aggressive chunking, and for small static corpora CAG/long-context can be simpler.

Q5. Explain the components of a RAG pipeline.

Document loader and parser, cleaner, chunker, embedding model, vector store (with ANN index and metadata), retriever, optional query transformer and re-ranker, prompt template, LLM generator, plus memory, caching, guardrails and evaluation.

Q6. What does 'grounding' mean?

Making the LLM base its answer on supplied evidence rather than its parametric memory, ideally with citations to that evidence. Faithfulness measures how well the answer is grounded.

Q7. What is parametric vs non-parametric memory?

Parametric memory is knowledge stored in model weights. Non-parametric memory is the external index the retriever searches. RAG combines both — the original RAG paper used exactly these terms.

6.9.2 Chunking & Embeddings

Q8. Why do we chunk documents?

Embedding models have input limits; one vector for a long document blurs its meaning; and smaller chunks let us send only relevant text to the LLM, reducing noise and cost.

Q9. How do you choose chunk size and overlap?

Start around 256–1024 tokens with 10–20% overlap, adapted to the document type and embedding model limit, then tune on an evaluation set by measuring recall@k and answer quality. Small chunks are precise but lack context; large chunks have context but dilute the embedding.

Q10. What is semantic chunking?

Splitting where the meaning changes: embed consecutive sentences and break the chunk where the similarity between neighbours drops below a threshold, so each chunk stays on one topic.

Q11. What is an embedding?

A dense numerical vector representation of text where semantically similar texts are close together. It is produced by a transformer encoder followed by pooling, trained with contrastive learning.

Q12. Cosine similarity vs dot product vs Euclidean?

Cosine measures the angle between vectors (ignores magnitude). Dot product includes magnitude; for normalised vectors it equals cosine. Euclidean measures straight-line distance. Use what the embedding model was trained with; with normalised vectors all give the same ranking.

Q13. Bi-encoder vs cross-encoder?

A bi-encoder embeds query and document separately, so document vectors can be precomputed — fast, used for retrieval. A cross-encoder processes query and document together and outputs a relevance score — more accurate but slow, so it is used to re-rank the top candidates.

Q14. How do you choose an embedding model?

Check MTEB retrieval scores, language and domain fit, max input length, vector dimension (storage and speed), latency, cost, and data-privacy needs (API vs self-hosted). Then validate on your own golden dataset.

Q15. What happens if you change the embedding model?

All documents must be re-embedded, because vectors from different models live in different spaces and are not comparable.

Q16. When would you fine-tune an embedding model?

When the domain vocabulary is specialised (legal, medical, internal jargon) and off-the-shelf recall is poor. Train on (query, relevant passage) pairs — often synthetic pairs generated by an LLM — with a contrastive loss such as MultipleNegativesRankingLoss.

6.9.3 Retrieval & Vector DBs

Q17. What is a vector database and why not a normal DB?

It stores vectors with metadata and supports fast similarity search via ANN indexes. Normal databases index exact values, not 'closest in high-dimensional space'. (pgvector adds this to Postgres.)

Q18. What is ANN and why do we need it?

Approximate Nearest Neighbour search finds very close (not guaranteed exact) neighbours much faster than comparing with every vector. Brute force is O(N) per query, which is too slow for millions of vectors.

Q19. Explain HNSW.

Hierarchical Navigable Small World: a multi-layer graph where the top layers have few long-range links and lower layers have many short links. Search starts at the top, greedily moves to the closest node, and descends layer by layer. Parameters: M (links per node), efConstruction, efSearch (recall vs speed).

Q20. Explain IVF and PQ.

IVF clusters vectors with k-means and searches only the nprobe nearest clusters. PQ splits each vector into sub-vectors and replaces each with a codebook ID, compressing memory heavily. IVF-PQ combines both for billion-scale search.

Q21. What is hybrid search and why does it help?

Combining BM25 keyword search with dense vector search and fusing the results, often with Reciprocal Rank Fusion. Dense handles synonyms and paraphrases; BM25 handles exact terms like error codes and names. Together they improve recall.

Q22. What is Reciprocal Rank Fusion?

A rank-based way to merge result lists: each document gets the sum of 1/(k + rank) over all lists, with k usually 60. It avoids normalising incompatible scores and rewards documents that rank well in several lists.

Q23. What is MMR?

Maximal Marginal Relevance selects documents that are relevant to the query but not redundant with already-selected ones, using a λ parameter to balance relevance and diversity.

Q24. How do you choose top-k?

Balance recall against noise and token cost. A common pattern is to retrieve 20–50 candidates, re-rank, and keep the top 3–5. Tune using recall@k and answer quality; consider a similarity threshold to allow 'no answer'.

Q25. What is re-ranking and where does it sit?

A second stage after retrieval where a more accurate model (cross-encoder, Cohere Rerank, or LLM) re-scores the candidate chunks. It boosts precision so the best chunks reach the prompt.

Q26. What is metadata filtering? Pre vs post filtering?

Restricting search by attributes like department, date or user permissions. Pre-filtering applies the filter during the search (correct k results); post-filtering filters after retrieval and may leave fewer than k results.

Q27. Explain HyDE.

Hypothetical Document Embeddings: the LLM writes a hypothetical answer to the query, and we embed that answer to search, because answer-to-document similarity is stronger than question-to-document similarity. The final answer still uses only the real retrieved documents.

Q28. What is the 'lost in the middle' problem?

LLMs use information at the start and end of long contexts better than information in the middle. Mitigate by sending fewer, re-ranked chunks and placing the most relevant at the beginning or end.

6.9.4 Advanced RAG types

Q29. Naive vs Advanced vs Modular RAG?

Naive is a fixed retrieve-then-generate chain. Advanced adds pre-retrieval (query rewriting, routing) and post-retrieval (re-ranking, compression) optimisations. Modular breaks RAG into swappable modules with routing, loops and branches, orchestrated by tools like LangGraph.

Q30. Explain Corrective RAG.

After retrieval, an evaluator grades document relevance. If documents are relevant, they are refined and used; if irrelevant, the query is rewritten and a web search supplies new knowledge; if ambiguous, both are combined. Then the answer is generated. It makes RAG robust to bad retrieval.

Q31. Explain Self-RAG.

The model decides when to retrieve and critiques itself using reflection tokens: is the passage relevant, is the answer supported, is it useful. In LangGraph it is implemented with graders: document grader, hallucination grader and answer grader, looping back to rewrite or regenerate if checks fail.

Q32. Difference between CRAG and Self-RAG?

CRAG corrects the retrieved knowledge, mainly with a relevance grader and web-search fallback. Self-RAG reflects on the whole process — whether retrieval is needed and whether the generation is grounded and useful — and can loop on generation.

Q33. What is Adaptive RAG?

A router classifies query complexity and chooses the strategy: answer directly, single-step retrieval, or multi-step retrieval / web search. It saves cost on easy questions and improves quality on hard ones.

Q34. What is Agentic RAG?

Retrieval becomes a tool used by an LLM agent in a ReAct loop. The agent plans, chooses which tools or indexes to call, evaluates observations, retries with new queries, and stops when it has enough. It is more capable but less predictable and costlier, so it needs step limits and guardrails.

Q35. What is Graph RAG and when is it better than vector RAG?

It extracts entities and relationships into a knowledge graph and retrieves by traversing it, optionally with community summaries (Microsoft GraphRAG). It is better for multi-hop relationship questions and global questions like 'what are the main themes across all documents', where top-k similar chunks are not enough.

Q36. What is multi-hop RAG?

Answering questions that require chaining facts: retrieve, extract an intermediate answer, form a new query, retrieve again, until enough evidence is collected. Examples: IRCoT, FLARE.

Q37. How do you handle tables and images in RAG?

Use layout-aware parsers; convert tables to Markdown or summarise them; caption images with a vision model and embed the captions while returning the original image (multi-vector retrieval); or use multimodal embeddings / page-image retrieval like ColPali, and a multimodal LLM for generation.

Q38. What is the parent-document retriever?

Index small child chunks for precise matching, but return the larger parent chunk to the LLM so it has enough context. Children store their parent ID; parents live in a docstore.

Q39. How does conversational RAG handle follow-up questions?

It uses an LLM to condense the chat history and follow-up into a standalone question before retrieval, then generates the answer using retrieved context plus recent history.

6.9.5 Evaluation

Q40. How do you evaluate a RAG system?

Separately evaluate retrieval (hit rate, recall@k, precision@k, MRR, nDCG on a labelled set) and generation (faithfulness, answer relevancy, context precision and recall, correctness) using tools like RAGAS or DeepEval with LLM-as-a-judge, plus latency, cost and user feedback.

Q41. What is faithfulness?

The share of claims in the answer that can be supported by the retrieved context. Low faithfulness means hallucination even if the answer sounds right.

Q42. Context precision vs context recall?

Context precision: are the retrieved chunks relevant, and are relevant ones ranked at the top (low noise)? Context recall: does the retrieved context contain everything needed for the ground-truth answer (nothing missing)?

Q43. How do you build an evaluation dataset without labels?

Generate synthetic question–answer–context triples from your documents with an LLM (RAGAS testset generator), have humans review a sample, and add real user questions and failures from production over time.

Q44. What are the limitations of LLM-as-a-judge?

Bias (position, verbosity, self-preference), inconsistency, and cost. Mitigate with clear rubrics, low temperature, structured outputs, and calibration against human labels.

6.9.6 Production & Scenarios

Q45. Your RAG chatbot is hallucinating. How do you debug it?

First check retrieval: log retrieved chunks for failing queries. If the right chunk is missing, fix chunking, use hybrid search, query rewriting or re-ranking. If the right chunk is present but the answer is wrong, tighten the prompt ('only from context, else say I don't know'), lower temperature, reduce noisy context, and add a groundedness check. Measure with faithfulness and context recall before and after.

Q46. Users search with exact error codes and get poor results. Fix?

Dense embeddings handle rare tokens poorly. Add BM25 for hybrid search, store error codes as metadata for filtering, and possibly re-rank.

Q47. Retrieval is good but latency is high. What do you do?

Stream the response; cache embeddings, retrieval results and frequent answers (semantic cache); run retrievers in parallel; reduce k and context size; use a smaller re-ranker or grader model; tune HNSW efSearch; skip retrieval for simple queries (adaptive routing).

Q48. How do you keep the index up to date?

Incremental ingestion: detect new, changed and deleted documents with hashes or timestamps, re-chunk and re-embed only those, upsert by stable IDs, delete stale vectors, and store version and date metadata.

Q49. How do you enforce document-level security?

Store ACLs (user or group IDs) in chunk metadata and apply a metadata filter based on the logged-in user at retrieval time, so unauthorised chunks never reach the LLM.

Q50. How do you protect RAG from prompt injection?

Treat retrieved text as untrusted data: delimit it clearly, instruct the model not to follow instructions inside the context, sanitise ingested content, restrict which sources are indexed, limit tool permissions, and add output guardrails.

Q51. A question needs data from both PDFs and a SQL database. How would you design it?

Use a router or agent: classify the question, send numeric and aggregate parts to a Text-to-SQL tool (read-only), send policy and text parts to the vector retriever, then combine both results in the final generation step. LangGraph works well for this.

Q52. How would you reduce cost of a RAG system?

Use smaller and cheaper models for routing, grading and rewriting; reduce chunk count and size via re-ranking and compression; use prompt caching and semantic caching; batch embeddings; use open-source embeddings.

Q53. What would you log for observability?

User query, rewritten query, retrieved chunk IDs and scores, re-ranked order, final prompt token count, model, answer, latency per step, cost, and user feedback — with tracing via LangSmith or Langfuse.

Q54. Design a RAG system for 10 million internal documents.

Distributed ingestion with layout-aware parsing; structure-aware chunking with rich metadata including ACLs; a scalable vector DB with HNSW or IVF-PQ and sharding; hybrid search plus a cross-encoder re-ranker; query rewriting and routing by department; permission filters; a grounded prompt with citations; caching and streaming; RAGAS-based evaluation in CI; observability and a feedback loop.

6.9.7 Quick one-liners for rapid-fire rounds

Term	One-liner
Chunk	The unit of text that is embedded, retrieved and fed to the LLM.
Top-k	Number of most similar chunks returned by the retriever.
Embedding	Vector that captures meaning; similar text → nearby vectors.
BM25	Keyword ranking using term frequency, inverse document frequency and length normalisation.
Hybrid search	BM25 + vector search fused (often with RRF).
Re-ranker	Cross-encoder that re-scores candidates for higher precision.
HyDE	Search using the embedding of a hypothetical answer.
MMR	Relevance + diversity selection.
CRAG	Grade retrieved docs; fall back to web search if bad.
Self-RAG	Model reflects: retrieve? relevant? supported? useful?
Agentic RAG	Agent uses retrieval as a tool in a ReAct loop.
GraphRAG	Knowledge graph + community summaries for relational and global questions.
Faithfulness	Is the answer supported by the context?
Lost in the middle	LLMs underuse information placed in the middle of long contexts.

6.10 RAG Cheat Sheet

Problem	Go-to technique
Vague / short query	Query rewriting, Multi-query, HyDE
Exact terms / IDs missed	Hybrid search (BM25 + dense)
Right doc retrieved but ranked low	Bigger k + cross-encoder rerank
Chunks lack context	Parent-document, sentence-window, contextual retrieval
Irrelevant docs retrieved	Re-ranking, thresholds, CRAG grader
Answer not supported by docs	Strict prompt, Self-RAG hallucination grader, faithfulness eval
Follow-up questions	Conversational RAG (condense question)
Needs multiple steps	Query decomposition, multi-hop, agentic RAG
Relationships / global themes	GraphRAG
Numbers from tables	Text-to-SQL / structured RAG
Images and diagrams	Multimodal RAG
Mixed simple and complex traffic	Adaptive RAG (router)
Knowledge base incomplete	CRAG with web-search fallback

Interview tip: Golden rule for interviews and real projects: **measure first, then optimise**. Start with naive RAG, build an evaluation set, find whether retrieval or generation is failing, then add only the technique that fixes that failure.

CHAPTER 7

Agentic AI: Building Autonomous AI Agents

Agent basics • Architectures (ReAct, Plan-and-Execute, Reflection, ReWOO) • LangChain fundamentals • LangGraph architecture • Building agents • Planning & reasoning • Multi-agent systems • Human-in-the-loop • Memory & state • Tools • Error handling • Streaming • Evaluation • Production design • Research & support agents

7.1 Agent Basics

What is an AI agent?

An **AI agent** is a system where an LLM decides the **control flow**: it perceives a goal and context, **reasons/plans**, chooses and calls **tools** (APIs, search, code, databases), observes results, keeps **memory**, and loops until the goal is achieved. A chain is a fixed recipe; an agent chooses its own next step.

$$\text{Agent} = \text{LLM (brain)} + \text{Tools (hands)} + \text{Memory} + \text{Planning} + \text{Control loop}$$

Level of autonomy	Description	Example
LLM call	single prompt → response	summarise an email
Chain / workflow	fixed sequence of LLM + code steps	RAG pipeline
Router	LLM picks one branch	classify ticket → billing or tech flow
Tool-calling agent	LLM loops, choosing tools until done	research assistant
Multi-agent system	several specialised agents collaborate	planner + researcher + writer + reviewer
Autonomous agent	long-horizon goals, self-directed sub-tasks	coding agent fixing an issue end-to-end

Interview tip: Anthropic's guidance (Building Effective Agents) and practitioner experience agree: start with the **simplest** thing (single call → workflow) and add agentic autonomy only when the task truly needs flexible, multi-step decisions — agents cost more latency, tokens and predictability.

Function / tool calling — how agents act

1. You describe tools to the model with a name, description and JSON-schema parameters.
2. The model returns a structured **tool call** (name + arguments) instead of text when it needs a tool.
3. Your code executes the function and returns the result as a **tool message**.
4. The model continues reasoning with the observation, calls more tools, or writes the final answer.

tool_calling.py

```
from langchain_core.tools import tool
@tool
def get_order_status(order_id: str) -> str:
    """Return the shipping status for an order ID like 'ORD123'."""     # the docstring IS the tool description
    return orders_db.get(order_id, "not found")

llm_with_tools = llm.bind_tools([get_order_status])
msg = llm_with_tools.invoke("Where is my order ORD123?")
print(msg.tool_calls)   # [{'name': 'get_order_status', 'args': {'order_id': 'ORD123'}, 'id': '...'}]
```

Agent architectures

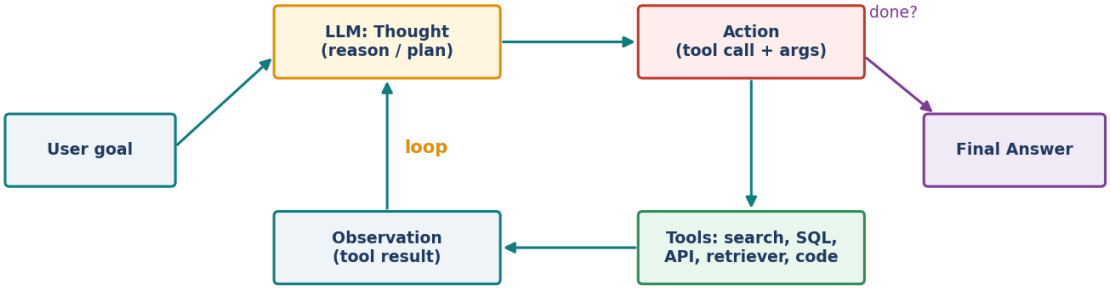


Figure: ReAct loop: Thought → Action → Observation, repeated until the final answer.

Architecture	How it works	Strengths / weaknesses
ReAct (Reason + Act)	interleave a reasoning step with a tool action and its observation in a loop	adaptive, simple, the default / can loop, many LLM calls
Plan-and-Execute	planner LLM writes a multi-step plan; executor runs steps; re-planner updates the plan	better for long tasks, cheaper executors / plans can go stale
ReWOO	plan all tool calls upfront with variable placeholders, execute, then solve	fewer LLM calls / less adaptive
Reflection / Reflexion	generate → critique (self or critic agent) → revise; store lessons in memory	higher quality / extra calls
LLM Compiler	plan a DAG of tool calls and run independent ones in parallel	low latency
Tree-of-Thoughts / LATS	explore multiple reasoning/action branches with evaluation and backtracking (search)	hard problems / expensive
Router / Orchestrator-workers	classify and dispatch; or a lead agent spawns worker LLMs dynamically	scalable, clear responsibilities
Evaluator-optimizer	one LLM produces, another evaluates and feeds back in a loop	clear quality criteria (translation, code)

Use cases

- Customer support agents (look up orders, refunds, create tickets, escalate).
- Research agents (web search, read, synthesise reports with citations).
- Data-analysis agents (Text-to-SQL, Python execution, charts).
- Coding agents (read repo, edit files, run tests).
- IT-ops agents (diagnose incidents from logs, suggest/run runbook steps with approval).
- Workflow automation (email triage, scheduling, CRM updates).

7.2 LangChain & LangGraph

LangChain fundamentals

Component	Purpose	Example
Chat models	unified interface over providers	ChatOpenAI, ChatGoogleGenerativeAI, ChatAnthropic, ChatOllama
Messages	SystemMessage, HumanMessage, AIMessage, ToolMessage	conversation state
Prompt templates	parameterised prompts	ChatPromptTemplate.from_messages([...]), MessagesPlaceholder
Output parsers / structured output	parse text into types	StrOutputParser, JsonOutputParser, llm.with_structured_output(PydanticModel)
LCEL (Runnables)	compose with the | operator; every runnable supports invoke / batch / stream / ainvoke	prompt | llm | parser
RunnablePassthrough / Parallel / Lambda	data routing inside chains	{'context': retriever, 'question': RunnablePassthrough()}
Document loaders, splitters, embeddings, vector stores, retrievers	RAG building blocks	PyPDFLoader, RecursiveCharacterTextSplitter, Chroma, FAISS
Tools	functions the LLM can call	@tool, StructuredTool, community tools (Tavily, Wikipedia)
Callbacks / tracing	observability	LangSmith

lcel_basics.py

```
from pydantic import BaseModel, Field
from typing import Literal
from langchain_core.prompts import ChatPromptTemplate

class Ticket(BaseModel):
    category: Literal["billing", "technical", "account", "other"]
    priority: Literal["P1", "P2", "P3"]
    summary: str = Field(description="one-line summary")

prompt = ChatPromptTemplate.from_messages([("system", "Classify the support ticket."), ("human", "{ticket}")])
chain = prompt | llm.with_structured_output(Ticket)            # LCEL composition
print(chain.invoke({"ticket": "Payment debited twice, need refund urgently"}))
# Ticket(category='billing', priority='P1', summary='Duplicate payment, refund requested')
for chunk in (prompt | llm).stream({"ticket": "..."}): print(chunk.content, end="")   # streaming
results = chain.batch([{"ticket": t} for t in tickets], config={"max_concurrency": 5})
```

Why LangGraph?

LCEL chains are DAGs (no loops). Agents need **cycles** (retry, reflect, call tools repeatedly), **branching**, **persistent state**, **human approval pauses** and **fault tolerance**. LangGraph models an application as a **state machine graph**.

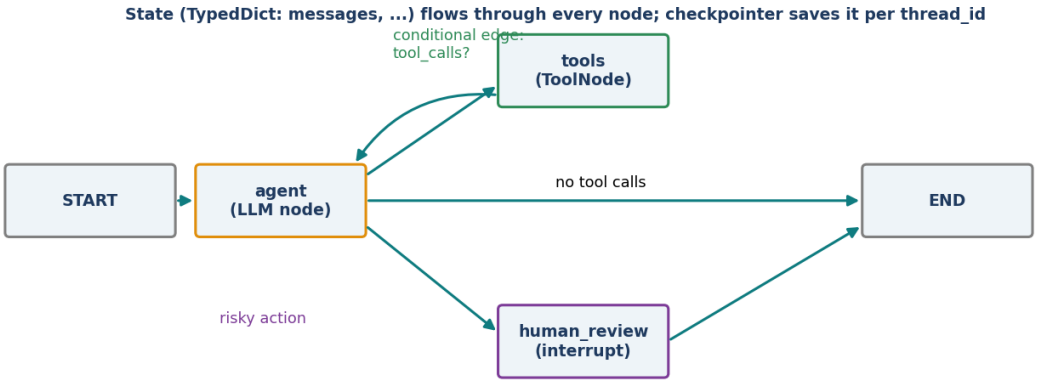


Figure: A LangGraph agent with a tool loop and a human-review interrupt.

Concept	Meaning
State	a TypedDict / Pydantic model shared by all nodes (e.g. messages, plan, docs, retries)
Reducers	how updates merge into state: default overwrite; Annotated[list, add_messages] appends messages; operator.add concatenates lists
Nodes	Python functions (state) → partial state update; an LLM call, a tool, any code
Edges	fixed transitions add_edge(a, b); START and END are special nodes
Conditional edges	add_conditional_edges(node, router_fn, mapping) — the router reads state and returns the next node name
Compile	graph.compile(checkpointer=..., interrupt_before=[...]) → runnable app (invoke/stream)
Checkpointer	saves state after every step per thread_id (MemorySaver, SqliteSaver, PostgresSaver) → memory, resume, time-travel
Interrupts	pause for human input (interrupt() / interrupt_before); resume with Command(resume=...)
Command / Send	Command(goto=..., update=...) for dynamic routing/handoffs; Send for map-reduce fan-out to parallel nodes
Subgraphs	a compiled graph used as a node — modular multi-agent systems
Streaming modes	values, updates, messages (tokens), custom

7.3 Building Agents

Your first agent with LangGraph (from scratch)

first_agent.py

```
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.checkpoint.memory import MemorySaver
from langchain_core.tools import tool
from langchain_google_genai import ChatGoogleGenerativeAI

@tool
def calculator(expression: str) -> str:
    """Evaluate a basic arithmetic expression, e.g. '23*7+1'."""
    import numexpr; return str(numexpr.evaluate(expression))
@tool
def search_kb(query: str) -> str:
    """Search the company knowledge base for policies and procedures."""
    return "\n".join(d.page_content for d in retriever.invoke(query))

tools = [calculator, search_kb]
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0).bind_tools(tools)

class State(TypedDict): messages: Annotated[list, add_messages]
def agent(state: State): return {"messages": [llm.invoke(state["messages"])]}

g = StateGraph(State)
g.add_node("agent", agent); g.add_node("tools", ToolNode(tools))
g.add_edge(START, "agent")
g.add_conditional_edges("agent", tools_condition)        # → "tools" if tool_calls else END
g.add_edge("tools", "agent")                              # loop back after tool execution
app = g.compile(checkpointer=MemorySaver())

cfg = {"configurable": {"thread_id": "1"}, "recursion_limit": 15}   # step limit = safety
out = app.invoke({"messages": [("user", "What is the travel allowance per day, and what is it for 12 days?")]}, cf
print(out["messages"][-1].content)
```

Shortcut: *langgraph.prebuilt.create_react_agent(model, tools, checkpointer=...)* builds the same graph. Build it once by hand to understand the loop — interviewers often ask how it works internally.

Agent planning & reasoning

- **Task decomposition**: break a goal into sub-tasks (CoT prompting, Plan-and-Execute, HuggingGPT-style).
- **Self-reflection**: critique intermediate results; retry with a corrected approach.
- **Re-planning**: update the plan after each observation (plans become stale when tools return surprises).
- **Reasoning models** (long internal chain-of-thought) improve planning on complex tasks but cost more latency.
- **Grounding the plan**: give the planner the list of tools and their limitations; validate plan steps before executing.

plan_and_execute.py

```
class PlanState(TypedDict):
    input: str; plan: list[str]; past_steps: Annotated[list, operator.add]; response: str
class Plan(BaseModel): steps: list[str]
class Act(BaseModel): response: str | None = None; plan: list[str] | None = None   # finish OR new plan

def planner(s):  return {"plan": planner_llm.with_structured_output(Plan).invoke(s["input"]).steps}
def executor(s):
    step = s["plan"][0]; result = react_agent.invoke({"messages": [("user", step)]})["messages"][-1].content
    return {"past_steps": [(step, result)]}
def replanner(s):
    out = replanner_llm.with_structured_output(Act).invoke(f"Goal: {s['input']}\nDone: {s['past_steps']}\nPlan: {s
    return {"response": out.response} if out.response else {"plan": out.plan}
g = StateGraph(PlanState)
for n, f in [("planner", planner), ("executor", executor), ("replanner", replanner)]: g.add_node(n, f)
g.add_edge(START, "planner"); g.add_edge("planner", "executor"); g.add_edge("executor", "replanner")
g.add_conditional_edges("replanner", lambda s: END if s.get("response") else "executor")
```

Multi-agent systems

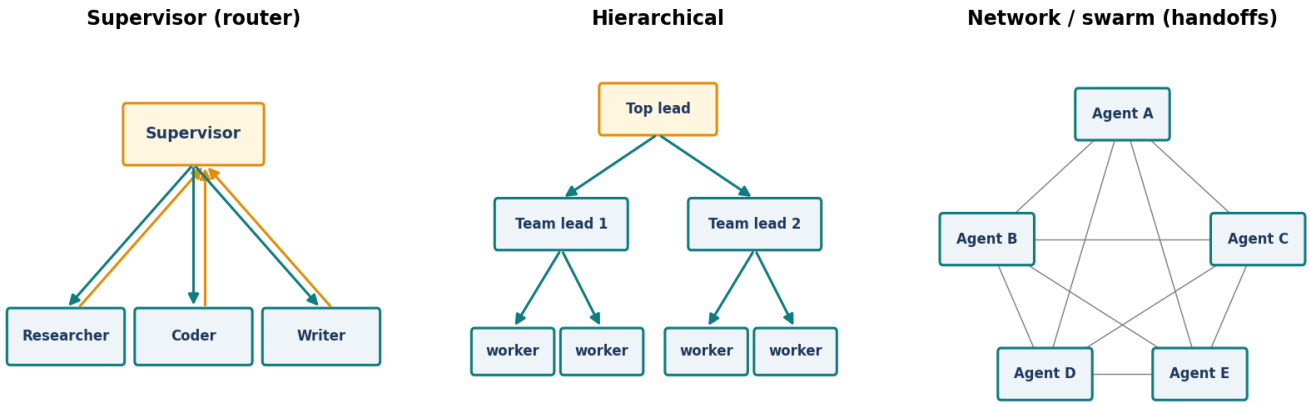


Figure: Common multi-agent topologies.

Pattern	How	When
Supervisor	a router LLM chooses which specialist agent acts next; agents report back	clear specialisations; most common
Hierarchical	supervisors of supervisors (teams)	large complex workflows
Network / swarm	agents hand off directly to each other (Command(goto=...))	conversational handoffs (sales → support)
Sequential pipeline	agent A → B → C	content pipelines (research → write → edit)
Debate / critic	agents argue or review each other	quality, reducing errors
Map-reduce	fan out sub-tasks with Send, aggregate	parallel research over many sources

Why multiple agents: smaller focused prompts and tool sets per agent (LLMs choose better among 3 tools than 30), separation of concerns, independent testing, parallelism. Costs: more tokens, coordination failures, harder debugging. Frameworks: LangGraph, CrewAI (role-based crews), AutoGen / AG2 (conversational agents), OpenAI Agents SDK, Google ADK, Semantic Kernel.

supervisor_multi_agent.py

```
from langgraph.types import Command
from typing import Literal
class Route(BaseModel): next: Literal["researcher", "coder", "FINISH"]

def supervisor(state) -> Command[Literal["researcher", "coder", "__end__"]]:
    r = sup_llm.with_structured_output(Route).invoke([("system", "Route to the best worker or FINISH.")] + state["
    return Command(goto=END if r.next == "FINISH" else r.next)
def researcher(state) -> Command[Literal["supervisor"]]:
    res = research_agent.invoke(state)
    return Command(update={"messages": [AIMessage(res["messages"][-1].content, name="researcher")]}, goto="supervi
# coder node analogous; add nodes, add_edge(START, "supervisor"), compile()
```

Human-in-the-loop (HITL)

Agents should not take irreversible or high-risk actions (refunds, deleting data, sending emails, production changes) without approval. Patterns: **approve/reject** a tool call, **edit** the tool arguments or draft, **provide input** when the agent is uncertain, **review final output**. LangGraph implements HITL with interrupts + checkpointer, so the graph can wait indefinitely and resume later.

human_in_the_loop.py

```
from langgraph.types import interrupt, Command
def issue_refund(state):
    decision = interrupt({"action": "refund", "amount": state["amount"], "order": state["order_id"]})  # pauses he
    if decision["approved"]: payments.refund(state["order_id"], state["amount"]); return {"status": "refunded"}
    return {"status": "rejected by human"}

app = g.compile(checkpointer=SqliteSaver.from_conn_string("ckpt.db"))
app.invoke(inputs, cfg)                                    # runs until interrupt → returns pending payload
app.invoke(Command(resume={"approved": True}), cfg)        # human approves → resumes from the checkpoint
```

Memory & state management

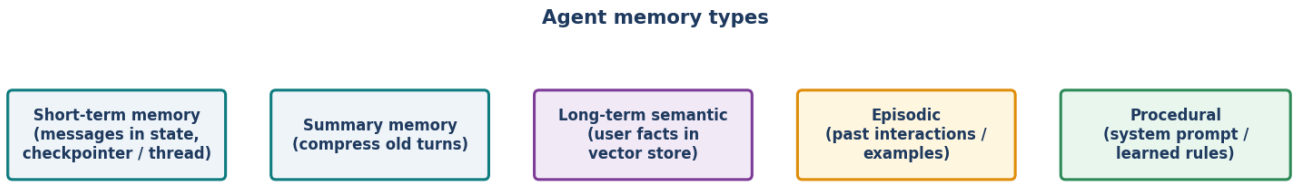


Figure: Types of agent memory.

Memory	Implementation	Notes
Short-term (thread)	messages in graph state + checkpointer keyed by thread_id	trim/summarise when it approaches the context limit
Conversation summary	summarise older turns into a running summary in state	keeps tokens bounded
Long-term (cross-thread)	LangGraph Store / vector DB keyed by user_id namespace	user preferences, facts; retrieve relevant memories each turn
Episodic	store successful past trajectories as few-shot examples	agents improve over time
Procedural	system prompt / rules updated from feedback	self-improving instructions

Interview tip: State design tips: keep state minimal and typed; store IDs not huge blobs; use reducers deliberately; include counters (retries, steps) to prevent infinite loops; make nodes idempotent so resuming after a crash is safe.

7.4 Advanced Agent Topics

Tool creation & integration

- Clear names and docstrings — the LLM chooses tools by their descriptions. Describe when to use and when NOT to use.
- Typed, minimal arguments with Pydantic schemas and validation; return concise, structured results (not 10 MB of HTML).
- Make tools **idempotent** and safe; separate read tools from write tools; least privilege credentials.
- Return errors as informative strings so the LLM can recover ('order not found; ask the user to re-check the ID').
- **Model Context Protocol (MCP)**: an open standard for exposing tools, resources and prompts from servers to any MCP-compatible client/agent — 'USB-C for AI tools'; LangChain/LangGraph can load MCP tools via adapters.
- Common tools: web search (Tavily, Bing/Google APIs), retrievers, SQL databases, Python REPL (sandboxed!), REST APIs, email/calendar, file systems.

Error handling & retry logic

Failure	Handling
Transient API errors / rate limits (429, 5xx)	retries with exponential backoff + jitter (tenacity, .with_retry()); timeouts
Model / provider outage	fallback model: llm.with_fallbacks([backup_llm])
Invalid tool arguments / malformed JSON	schema validation; send the validation error back to the LLM to self-correct (bounded retries)
Tool exceptions	catch inside the tool or ToolNode(handle_tool_errors=True) → error message as observation
Infinite loops	recursion_limit, max iterations counter in state, loop detection
Bad outputs	graders/validators node → retry or route to human
Crash mid-run	checkpointer allows resume from the last successful step

retries_and_fallbacks.py

```
from tenacity import retry, stop_after_attempt, wait_exponential_jitter
@retry(stop=stop_after_attempt(4), wait=wait_exponential_jitter(initial=1, max=20))
def call_crm(payload): return requests.post(CRM_URL, json=payload, timeout=10).raise_for_status().json()

robust_llm = primary_llm.with_retry(stop_after_attempt=3).with_fallbacks([backup_llm])
```

Streaming & real-time responses

Users perceive latency as time-to-first-token. Stream LLM tokens and intermediate agent steps ('Searching knowledge base…', 'Calling order API…').

streaming.py

```
# LangGraph: stream node updates and LLM tokens
for mode, chunk in app.stream(inputs, cfg, stream_mode=["updates", "messages"]):
    if mode == "messages": token, meta = chunk; print(token.content, end="", flush=True)
    else: print("\n[step]", list(chunk.keys()))

# FastAPI Server-Sent Events endpoint
from fastapi.responses import StreamingResponse
@app_api.post("/chat")
async def chat(req: ChatRequest):
    async def gen():
        async for token, meta in graph.astream({"messages": [("user", req.message)]}, cfg_for(req.user), stream_mo
            if token.content: yield f"data: {token.content}\n\n"
    return StreamingResponse(gen(), media_type="text/event-stream")
```

Evaluation & testing of agents

Level	What to check	How
Unit	tools, parsers, routers deterministic logic	pytest with mocked LLMs
Component	router accuracy, tool-selection accuracy, argument correctness	labelled datasets of (input → expected tool + args)
Trajectory	did the agent take a sensible path? number of steps, redundant calls	compare against reference trajectories; LLM-judge
Final response	task success, correctness, groundedness, tone, safety	golden set + LLM-as-judge + human review
Operational	latency p50/p95, cost per task, error rate, loop rate	tracing (LangSmith, Langfuse), dashboards
Safety / red-team	prompt injection, data exfiltration, harmful actions	adversarial test suites

Production-ready agent design

- Prefer deterministic workflows with LLM decisions only where needed; bounded autonomy.
- Typed state, structured outputs, validated tool inputs.
- Guardrails: input moderation, prompt-injection defences (treat tool/RAG outputs as untrusted data), output filters, PII redaction.
- Least-privilege tools, approval gates for write/irreversible actions, audit logs of every tool call.
- Durable execution: persistent checkpointer (Postgres), idempotent nodes, retries, timeouts.
- Observability: trace every step with inputs/outputs, tokens, latency and cost.
- Cost control: small models for routing/grading, caching, step limits, concise context.
- Deployment: FastAPI/LangGraph Platform, containerised, horizontally scaled, async; versioned prompts and graphs; evaluation in CI.

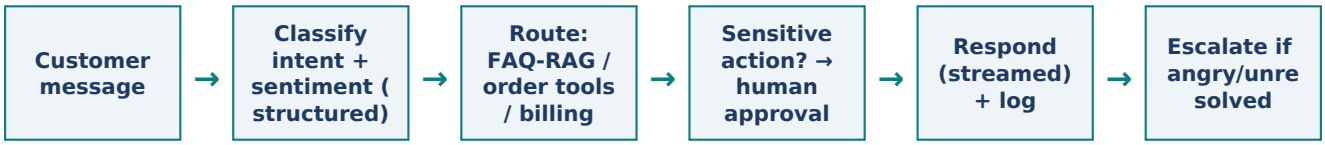
Hands-on 1: Research agent



research_agent.py

```
class ResearchState(TypedDict):
    topic: str; queries: list[str]; notes: Annotated[list, operator.add]; report: str; revisions: int
def plan(s): return {"queries": planner.with_structured_output(Queries).invoke(s["topic"]).items}
def fan_out(s): return [Send("search", {"q": q}) for q in s["queries"]]            # parallel map
def search(s): hits = tavily.invoke(s["q"]); return {"notes": [summarise(s["q"], hits)]}
def write(s): return {"report": writer.invoke({"topic": s["topic"], "notes": s["notes"]}).content, "revisions": s.
def review(s): ok = reviewer.with_structured_output(Verdict).invoke(s["report"]).ok; return END if ok or s["revisi
g = StateGraph(ResearchState)
for n, f in [("plan", plan), ("search", search), ("write", write)]: g.add_node(n, f)
g.add_edge(START, "plan"); g.add_conditional_edges("plan", fan_out, ["search"]); g.add_edge("search", "write")
g.add_conditional_edges("write", review)
```

Hands-on 2: Customer support agent



Component	Implementation
Intent router	with_structured_output(Intent) → conditional edges
Knowledge	RAG over FAQs and policies with citations
Tools	get_order_status, initiate_return, create_ticket (write tools gated by HITL)
Memory	thread checkpointer + long-term customer profile store
Guardrails	PII masking, tone check, policy compliance, no promises outside policy
Metrics	resolution rate, escalation rate, CSAT, handle time, cost per conversation

7.5 Interview Questions — Agentic AI

Q1. What is the difference between a chain and an agent?

A chain follows a predetermined sequence of steps; an agent uses an LLM to decide dynamically which action or tool to take next, looping until the goal is met.

Q2. Explain the ReAct pattern.

The model alternates Thought (reasoning about what to do), Action (tool call) and Observation (tool result) in a loop, grounding its reasoning in real tool outputs until it produces a final answer.

Q3. How does tool/function calling work?

Tools are described with names, descriptions and JSON schemas; the model outputs a structured call with arguments; the application executes it and returns the result as a tool message for the model to continue.

Q4. Why LangGraph over plain LangChain chains?

LangGraph supports cycles, conditional branching, explicit typed state, persistence via checkpointers, human-in-the-loop interrupts, streaming and multi-agent subgraphs — what real agents need.

Q5. What are reducers in LangGraph?

Functions that define how a node's update merges into a state key — for example add_messages appends messages instead of overwriting, and operator.add concatenates lists from parallel nodes.

Q6. What is a checkpointer?

A persistence layer that saves graph state after each step per thread_id, enabling conversation memory, resuming after failures or interrupts, and time-travel debugging.

Q7. How do you implement human-in-the-loop?

Compile with a checkpointer and call interrupt() (or interrupt_before a node) before risky actions; the graph pauses and persists; a human approves or edits and the graph resumes with Command(resume=...).

Q8. ReAct vs Plan-and-Execute?

ReAct decides one step at a time — adaptive but many large-model calls; Plan-and-Execute creates an upfront plan executed step by step with re-planning — better for long tasks and allows cheaper executor models.

Q9. When would you use multiple agents?

When the task spans distinct skills or tool sets, so specialised agents with focused prompts perform better; patterns include supervisor routing, hierarchical teams and handoffs. Avoid it when a single agent suffices.

Q10. How do you prevent an agent from looping forever?

Set recursion/step limits, track retry counters in state, detect repeated identical tool calls, and add termination criteria and fallbacks to a human.

Q11. How do you handle tool failures?

Validate inputs, catch exceptions and return informative error messages as observations, retry transient errors with backoff, use fallback tools/models, and escalate after bounded attempts.

Q12. Short-term vs long-term memory in agents?

Short-term memory is the current thread's messages/state persisted by the checkpointer; long-term memory stores facts across threads (e.g. user preferences) in a store or vector DB and is retrieved when relevant.

Q13. How do you evaluate an agent?

Test tools and routers deterministically, measure tool-selection and argument accuracy, evaluate trajectories and final answers against golden datasets with LLM-as-judge and humans, and monitor latency, cost and failure rates in production.

Q14. What is prompt injection in agents and how do you defend?

Malicious instructions hidden in user input or tool/RAG content that hijack the agent. Defend with least-privilege tools, approval gates, treating external content as data, input/output filtering, and isolating sensitive actions.

Q15. Why stream intermediate steps?

It reduces perceived latency and builds trust by showing progress (searching, calling tools), and lets users interrupt early.

Q16. What is MCP?

The Model Context Protocol — an open standard for connecting LLM applications to external tools and data sources via servers, so tools can be reused across different agent frameworks and clients.

CHAPTER 8

MLOps & LLMOps: Deploy, Scale and Maintain AI Systems

Principles • Experiment tracking (MLflow) • Model registry & versioning • Deployment strategies • FastAPI • TF Serving • Docker • Kubernetes • Model optimisation • CI/CD • Testing • Monitoring & drift • Retraining • AWS / Azure / GCP • LLMOps • Cost • Security, governance, explainability

8.1 MLOps Fundamentals

MLOps = DevOps principles applied to ML: reliably taking models from notebooks to production and keeping them healthy. Unlike software, ML systems depend on **code + data + model**, and they can silently degrade as the world changes. Google's 'Hidden Technical Debt in ML Systems' paper famously showed the model is a small box surrounded by data pipelines, serving, monitoring and configuration.

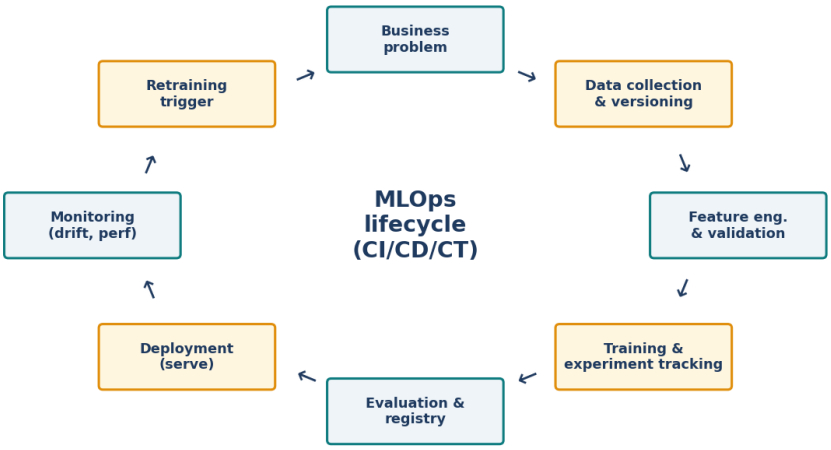


Figure: The MLOps lifecycle is a loop, not a line.

Principle	Meaning
Reproducibility	same code + data + config + environment → same model (version everything, fix seeds, pin dependencies)
Versioning	Git for code, DVC/lakeFS/Delta for data, model registry for models, prompt versioning for LLMs
Automation	pipelines for training, testing, deployment (CI/CD/CT)
Continuous Training (CT)	retrain automatically on schedule or when drift/performance triggers fire
Monitoring	data quality, drift, model performance, system health, cost
Collaboration & governance	lineage, approvals, audit trails, model cards

MLOps maturity (Google)	Characteristics
Level 0 — manual	notebooks, manual handoff, rare deployments, no monitoring

MLOps maturity (Google)	Characteristics
Level 1 — ML pipeline automation	automated training pipeline, continuous training, feature store, metadata tracking
Level 2 — CI/CD pipeline automation	automated build/test/deploy of the pipelines themselves; rapid reliable iteration

Experiment tracking with MLflow

Track every run's **parameters, metrics, artifacts (plots, model files), code version and environment** so you can compare runs and reproduce the best one. MLflow components: **Tracking**, **Models** (standard packaging 'flavors'), **Model Registry** (versions, aliases/stages, approvals), **Projects**; newer versions add LLM tracing and evaluation. Alternatives: Weights & Biases, Neptune, Comet, Azure ML/Vertex experiments.

mlflow_tracking.py

```
import mlflow, mlflow.sklearn
from sklearn.metrics import f1_score, roc_auc_score
mlflow.set_tracking_uri("http://localhost:5000"); mlflow.set_experiment("churn")
with mlflow.start_run(run_name="xgb_v3"):
    params = {"n_estimators": 400, "max_depth": 6, "learning_rate": 0.05}
    model = XGBClassifier(**params).fit(X_tr, y_tr)
    p = model.predict_proba(X_te)[:, 1]
    mlflow.log_params(params)
    mlflow.log_metrics({"f1": f1_score(y_te, p > .5), "auc": roc_auc_score(y_te, p)})
    mlflow.log_artifact("confusion_matrix.png")
    mlflow.sklearn.log_model(model, "model", registered_model_name="churn-classifier")   # → registry version N
# mlflow ui  → compare runs;   load later: mlflow.pyfunc.load_model("models:/churn-classifier@champion")
```

Model registry & versioning

- Each registered model has numbered **versions** with lineage to the run, data and code that produced it.
- Promote via aliases/stages: *staging* → *champion (production)* → *archived*, with approval gates and metadata (owner, metrics, model card).
- Serving loads 'models:/name@champion' so promotion = deployment without code changes; rollback = move the alias back.
- **Data versioning**: DVC tracks large data files with Git-like commands (dvc add, dvc push to S3/Azure Blob) so each model version links to an exact dataset.
- **Feature store** (Feast, Tecton, Databricks/Vertex/SageMaker feature stores): one definition of features for training (offline) and serving (online) → prevents **training–serving skew**.

Model deployment strategies

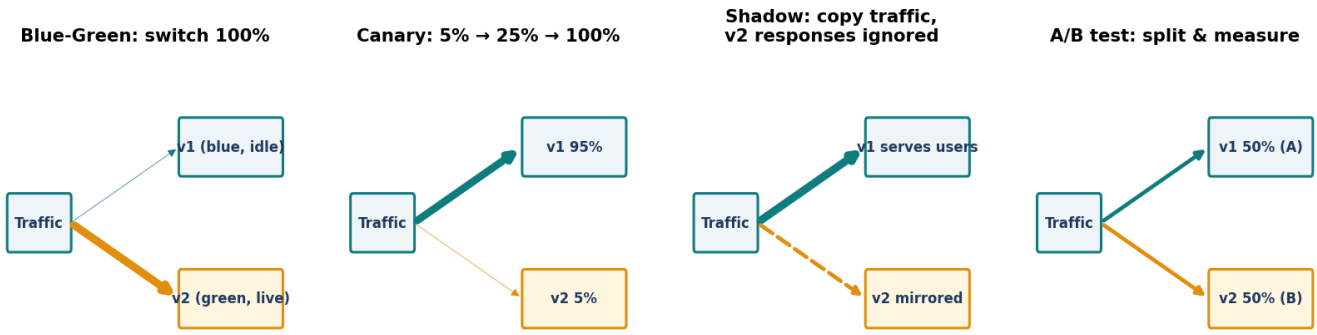


Figure: Blue-green, canary, shadow and A/B deployments.

Strategy	How	Pros	Cons
Recreate	stop v1, start v2	simple	downtime
Rolling	replace instances gradually	no downtime	mixed versions during rollout
Blue-green	two identical environments; switch the router	instant rollback	2× infrastructure
Canary	send small % of traffic to v2, increase if metrics OK	limits blast radius	needs good monitoring
Shadow (dark launch)	v2 receives a copy of live traffic; its outputs aren't used	zero user risk, real-data validation	extra compute; no user feedback
A/B test	split users randomly; compare business metrics statistically	measures real impact	needs traffic and time
Multi-armed bandit	adaptively shift traffic to the better model	less regret than A/B	more complex

Serving pattern	When
Online / real-time (REST/gRPC)	fraud check during payment, chatbots — low latency
Batch	nightly scoring of 4 lakh customers for a campaign — cheap, high throughput
Streaming	Kafka/Kinesis events scored continuously
Edge / on-device	mobile, IoT — TFLite, ONNX, Core ML; privacy, offline

8.2 Deployment & Infrastructure

Building ML APIs with FastAPI (production patterns)

serving_api.py

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, Depends, HTTPException
import mlflow, time, logging

models = {}
@asynccontextmanager
async def lifespan(app):                                    # load model once at startup
    models["churn"] = mlflow.pyfunc.load_model("models:/churn-classifier@champion"); yield; models.clear()

app = FastAPI(lifespan=lifespan)

@app.middleware("http")
async def timing(request, call_next):
    t = time.perf_counter(); resp = await call_next(request)
    resp.headers["X-Latency-ms"] = f"{(time.perf_counter() - t) * 1000:.1f}"; return resp

@app.post("/v1/predict", response_model=PredictionOut)
def predict(batch: list[CustomerIn], user=Depends(verify_api_key)):
    df = pd.DataFrame([c.model_dump() for c in batch])
    probs = models["churn"].predict(df)
    logging.info({"n": len(batch), "mean_p": float(probs.mean())})          # log for monitoring
    return PredictionOut(probabilities=probs.tolist(), model_version=MODEL_VERSION)
```

- Pydantic validation, versioned routes (/v1), /health and /ready probes, auth (API keys/OAuth/JWT), batching, async for I/O-bound calls, structured logging, request IDs.
- Run with multiple workers: *gunicorn -k uvicorn.workers.UvicornWorker -w 4 main:app*; CPU-heavy inference → worker processes or a dedicated model server.

- Test with FastAPI TestClient + pytest; load-test with Locust/k6.

TensorFlow Serving and other model servers

TF Serving serves SavedModel directories (model_name/1/, /2/ — version folders) over REST (:8501) and gRPC (:8500) with automatic version loading and request batching: *docker run -p 8501:8501 -v $PWD/models/churn:/models/churn -e MODEL_NAME=churn tensorflow/serving*; call POST /v1/models/churn:predict {"instances": [...]}. Others: **TorchServe**, **NVIDIA Triton** (multi-framework, dynamic batching, GPU), **BentoML**, **KServe/Seldon** on Kubernetes, **Ray Serve**; for LLMs **vLLM**, **TGI**, **Ollama** (local), **llama.cpp**.

Containerisation with Docker

A container packages code + dependencies + runtime so it runs identically on a laptop, CI and the cloud. **Image** = immutable template (built from a Dockerfile, layered and cached); **container** = running instance; **registry** = storage (Docker Hub, ECR, ACR, Artifact Registry).

Dockerfile

```
FROM python:3.11-slim AS base
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt     # cached layer: changes only when requirements change
COPY app/ ./app/
RUN useradd -m appuser && chown -R appuser /app
USER appuser                                             # don't run as root
EXPOSE 8000
HEALTHCHECK CMD curl -f http://localhost:8000/health || exit 1
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

docker commands

```
docker build -t churn-api:1.2.0 .
docker run -p 8000:8000 --env-file .env churn-api:1.2.0
docker compose up -d          # api + redis + vector DB + mlflow defined in docker-compose.yml
docker tag churn-api:1.2.0 myregistry.azurecr.io/churn-api:1.2.0 && docker push myregistry.azurecr.io/churn-api:1.
```

Best practices: slim base images, multi-stage builds, .dockerignore, pin versions, non-root user, no secrets in images (inject at runtime), one process per container, tag images with git SHA/semantic versions, scan for vulnerabilities (Trivy). GPU images: nvidia/cuda base + NVIDIA container toolkit.

Kubernetes for ML workloads

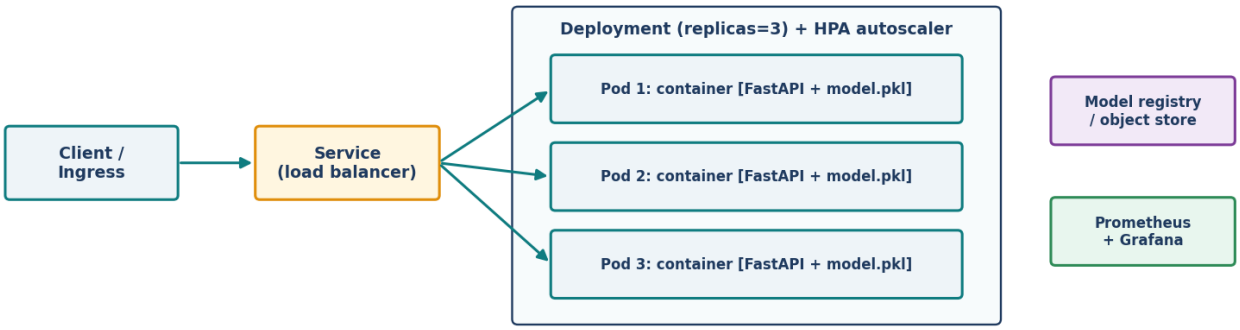


Figure: Model API on Kubernetes: Service load-balances across replicated Pods; HPA scales them.

K8s object	Role
Pod	smallest unit: one or more containers

K8s object	Role
Deployment	desired replicas, rolling updates, rollback
Service	stable network endpoint + load balancing across pods
Ingress / Gateway	HTTP routing, TLS, canary weights
ConfigMap / Secret	configuration and credentials injected as env/files
HPA	Horizontal Pod Autoscaler on CPU/GPU/custom metrics (requests per second, queue length)
Job / CronJob	batch scoring, scheduled retraining
PersistentVolume	model files, caches
Resource requests/limits	CPU/memory/GPU (nvidia.com/gpu: 1) per container

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: churn-api}
spec:
  replicas: 3
  selector: {matchLabels: {app: churn-api}}
  template:
    metadata: {labels: {app: churn-api}}
    spec:
      containers:
      - name: api
        image: myregistry.azurecr.io/churn-api:1.2.0
        ports: [{containerPort: 8000}]
        resources: {requests: {cpu: "500m", memory: "1Gi"}, limits: {cpu: "2", memory: "2Gi"}}
        readinessProbe: {httpGet: {path: /health, port: 8000}}
        envFrom: [{secretRef: {name: churn-secrets}}]
```

ML-specific tooling on Kubernetes: **Kubeflow** (pipelines, notebooks, training operators), **KServe** (serverless model serving, canary, scale-to-zero), **Argo Workflows**, **Ray** (distributed training/serving).

Model optimisation for production

Technique	Idea	Typical gain
Quantisation	FP32 → FP16/INT8/INT4 weights (and activations); post-training or quantisation-aware training	2–4× smaller, faster on supported hardware, small accuracy loss
Pruning	remove small/unimportant weights (unstructured) or whole channels/heads (structured)	smaller; structured pruning gives real speed-ups
Knowledge distillation	small student trained on teacher's soft labels: $L = \alpha CE(y, s) + (1-\alpha)T^2 \cdot KL(\text{teacher_T} \parallel \text{student_T})$	e.g. DistilBERT 40% smaller, 60% faster
Graph compilers / runtimes	ONNX Runtime, TensorRT, OpenVINO, TorchScript/torch.compile, XLA — operator fusion, kernel tuning	1.5–5×
Batching	dynamic/continuous batching amortises GPU cost	throughput ↑
Caching	cache frequent predictions / embeddings / LLM answers	latency and cost ↓
Hardware choice	CPU vs GPU vs TPU vs inference chips; right-size instances; spot instances for batch	cost ↓

8.3 CI/CD & Monitoring

CI/CD for ML

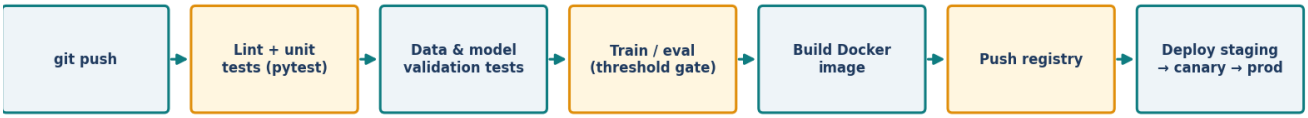


Figure: A typical ML CI/CD pipeline with a quality gate.

.github/workflows/ml-ci.yml

```
name: ml-ci
on: [push, pull_request]
jobs:
  test-train-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11"}
      - run: pip install -r requirements.txt
      - run: ruff check . && pytest -q tests/                    # code quality + unit tests
      - run: python pipelines/validate_data.py                   # schema / data checks
      - run: python pipelines/train.py && python pipelines/evaluate.py --min-f1 0.80   # quality gate
      - run: docker build -t $REGISTRY/churn-api:${{ github.sha }} .
      - run: docker push $REGISTRY/churn-api:${{ github.sha }}   # then CD: deploy to staging → canary
```

Automated testing for ML systems

Test type	Examples
Unit tests	feature functions, preprocessing, tokenisation, API schemas (pytest)
Data validation	schema, ranges, nulls, uniqueness, distribution checks (Great Expectations, Pandera, TFDV)
Model tests	minimum metric thresholds, no regression vs current champion, slice performance (per segment), invariance tests (changing a name shouldn't change sentiment), directional tests
Integration tests	end-to-end pipeline on a sample; API contract tests
Load / performance	latency p95 under expected RPS (Locust, k6)
LLM-specific	golden-set evals, LLM-as-judge regressions, prompt-injection and safety suites

Model monitoring & observability

Layer	Metrics
System	latency (p50/p95/p99), throughput, error rate, CPU/GPU/memory, saturation
Data	missing values, schema violations, feature distributions, input drift
Model	prediction distribution, confidence, accuracy/F1/AUC once labels arrive (delayed ground truth), calibration
Business	conversion, fraud loss, CSAT, revenue impact
LLM	tokens, cost, latency to first token, groundedness, user feedback, guardrail triggers

Tools: Prometheus + Grafana (system), Evidently AI / WhyLabs / Arize / Fiddler (ML), cloud monitors (SageMaker Model Monitor, Azure ML data drift, Vertex Model Monitoring), LangSmith/Langfuse/Phoenix (LLM traces), ELK/OpenTelemetry for logs/traces.

Data drift & model drift detection

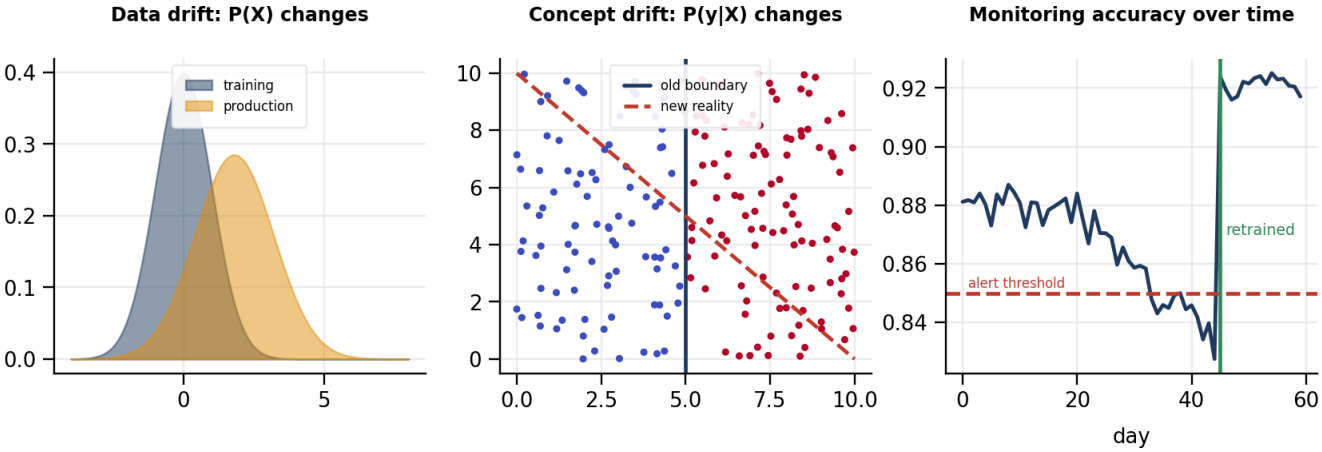


Figure: Data drift, concept drift, and monitoring-triggered retraining.

Drift type	Definition	Example
Data / covariate drift	P(X) changes	new customer segment, new device type, inflation shifts amounts
Concept drift	P(y|X) changes — same inputs now mean something different	fraudsters change tactics; COVID changed buying behaviour
Label / prior drift	P(y) changes	fraud rate doubles
Prediction drift	distribution of model outputs shifts	early warning when labels are delayed
Training–serving skew	features computed differently online vs offline	bug, not real-world change

$$\mathrm{PSI} = \sum_{i=1}^{B} (a_i - e_i) \ln \frac{a_i}{e_i} \qquad D_{KS} = \sup_x |F_{train}(x) - F_{prod}(x)|$$

PSI over B bins of expected (train) vs actual (prod) proportions: < 0.1 stable, 0.1–0.25 moderate, > 0.25 significant drift

Other tests: chi-square for categorical features, Jensen-Shannon / Wasserstein distance, embedding drift (distance between centroid of recent vs reference embeddings) for text/images. Alert on sustained drift in important features, then investigate before retraining blindly.

A/B testing & retraining strategies

- **A/B testing**: define a primary metric and guardrail metrics, compute sample size (power analysis), randomise at the user level, run for full business cycles, test significance (t-test / chi-square / Bayesian), beware peeking and novelty effects.
- **Retraining triggers**: scheduled (weekly), performance-based (metric below threshold), drift-based (PSI > 0.25), data-volume-based (N new labels).
- **Retraining modes**: full retrain on a sliding window, incremental/online learning, fine-tuning from the last checkpoint.
- Always validate the challenger against the champion on recent data before promotion (champion/challenger), then canary-deploy.

8.4 Cloud & LLMOps

Cloud ML platforms

Capability	AWS SageMaker	Azure Machine Learning	GCP Vertex AI
Notebooks / dev	SageMaker Studio	AML Studio, compute instances	Vertex Workbench / Colab Enterprise
Training	training jobs (managed, spot), HyperPod	command jobs, compute clusters	custom training jobs
AutoML	Autopilot / Canvas	Automated ML	Vertex AutoML
Pipelines	SageMaker Pipelines	AML pipelines / designer	Vertex Pipelines (Kubeflow)
Registry	Model Registry	Model registry (MLflow-native)	Model Registry
Serving	real-time / serverless / async / batch transform endpoints	managed online / batch endpoints	online / batch prediction endpoints
Monitoring	Model Monitor, Clarify (bias/explainability)	data drift, Responsible AI dashboard	Model Monitoring, Explainable AI
GenAI / LLMs	Amazon Bedrock (Claude, Llama, Titan, Mistral…)	Azure OpenAI / Azure AI Foundry (GPT, Llama, Phi…)	Vertex AI Gemini, Model Garden
Search / RAG	Kendra, OpenSearch, Bedrock Knowledge Bases	Azure AI Search	Vertex AI Search, Vector Search

Service names evolve frequently; focus on the capability map. Interview tip: explain how you would build the same pipeline (data → train → register → deploy → monitor) on any one cloud.

LLMOps — operating LLM applications

MLOps concern	LLMOps equivalent
Model training	mostly prompt design, RAG, fine-tuning/PEFT of foundation models
Feature versioning	prompt versioning, retrieval config, chunking/embedding versions
Accuracy metrics	LLM evals: golden sets, LLM-as-judge, faithfulness, toxicity, human feedback
Serving one model	routing across models, fallbacks, caching, streaming, rate limits
Compute cost	token cost per request/user/feature
Data drift	query drift, new topics, knowledge base staleness, model version changes by the provider
Security	prompt injection, jailbreaks, data leakage, tool misuse



Cost optimisation for LLM apps

$$\text{cost per request} = \frac{T_{in} \cdot p_{in} + T_{out} \cdot p_{out}}{10^6}$$

T = tokens, p = price per million tokens; output tokens are usually priced higher

- **Model routing / cascades**: small fast model for easy queries, large model only when needed.
- **Prompt optimisation**: shorter system prompts, fewer few-shot examples, compress retrieved context, limit max_tokens.
- **Caching**: provider prompt caching for repeated prefixes; exact and semantic response caches; embedding caches.
- **Batch APIs** for offline jobs (often discounted); async concurrency within rate limits.
- **Self-hosting** open models with vLLM + quantisation when volume is high; **fine-tune a small model** to replace a large one for a narrow task.
- Budgets, per-user quotas and cost dashboards.

Security, governance & explainability

Area	Controls
Security	secrets management (Key Vault/Secrets Manager), network isolation/private endpoints, authN/authZ, encryption, OWASP Top 10 for LLM apps (prompt injection, insecure output handling, training data poisoning, model DoS, supply-chain, sensitive info disclosure, excessive agency…)
Privacy	PII detection/redaction (Presidio), data minimisation, retention, consent, regional data residency
Governance	model inventory, model cards & datasheets, approval workflows, lineage, audit logs, risk classification (EU AI Act), periodic reviews
Fairness	bias metrics across groups, mitigation (re-weighting, constraints, threshold adjustment)
Explainability	global: feature importance, partial dependence; local: SHAP, LIME, counterfactuals; DL: Grad-CAM, attention/attribution; LLMs: citations, reasoning traces, retrieval transparency

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\ (|F| - |S| - 1)!}{|F|!} [f(S \cup \{j\}) - f(S)]$$

SHAP value: feature j's average marginal contribution over all feature subsets (Shapley values from game theory)

LIME fits a simple interpretable model (linear) around one prediction using perturbed samples. **SHAP** has solid theoretical guarantees (efficiency: contributions sum to prediction − baseline) and fast TreeSHAP for tree models.

Best practices & design patterns

- Design for failure: timeouts, retries, circuit breakers, graceful degradation, fallbacks.
- Separate training and serving pipelines but share feature code (or a feature store).
- Infrastructure as Code (Terraform/Bicep), GitOps for deployments.
- Everything versioned: code, data, models, prompts, configs, containers.
- Shadow → canary → full rollout with automatic rollback on metric regressions.
- Observability from day one; define SLOs (e.g. p95 < 300 ms, 99.9% availability).
- Human-in-the-loop for high-risk decisions; document limitations in model cards.

8.5 Interview Questions — MLOps & LLMOps

Q1. What is MLOps and why is it different from DevOps?

MLOps applies DevOps automation to ML, but must also version data and models, handle experiments, continuous training and monitor statistical behaviour like drift, because ML systems can fail silently even when code is unchanged.

Q2. How do you ensure reproducibility?

Version code (Git), data (DVC/snapshots), models (registry), configuration and environment (Docker, pinned requirements), fix random seeds, and log everything with an experiment tracker like MLflow.

Q3. What does a model registry give you?

A central catalogue of model versions with lineage, metrics, stages/aliases and approvals, so deployment and rollback become promoting or reverting a version.

Q4. Blue-green vs canary vs shadow deployment?

Blue-green switches all traffic between two identical environments (instant rollback). Canary gradually shifts a small percentage to the new model while watching metrics. Shadow mirrors traffic to the new model without using its outputs, for zero-risk validation.

Q5. What is training–serving skew?

A mismatch between how features are computed in training versus production, causing performance drops; prevented with shared feature code or a feature store and by logging serving features.

Q6. Data drift vs concept drift?

Data drift is a change in the input distribution P(X); concept drift is a change in the relationship P(y|X). Concept drift needs labels to confirm and usually requires retraining.

Q7. How do you detect drift?

Compare production feature distributions with the training reference using PSI, KS test, chi-square, JS/Wasserstein distance or embedding distances, monitor prediction distributions, and track performance once labels arrive.

Q8. When do you retrain a model?

On a schedule, when performance drops below a threshold, when significant drift is detected, or when enough new labelled data accumulates — then validate challenger vs champion before rollout.

Q9. Why Docker for ML?

It packages code, dependencies and runtime into an immutable image so the model runs identically across laptop, CI and cloud, enabling scaling and orchestration.

Q10. What does Kubernetes add?

Orchestration: scheduling containers across nodes, replicas and self-healing, load balancing, rolling updates, autoscaling (HPA), GPU scheduling, and config/secret management.

Q11. How would you reduce model latency?

Quantise, prune or distil the model, use optimised runtimes (ONNX Runtime, TensorRT), batch requests, cache results, choose appropriate hardware, and keep the model loaded in memory with enough replicas.

Q12. What tests do you write for an ML pipeline?

Unit tests for features and preprocessing, data validation tests, model quality gates and regression tests against the champion, slice and invariance tests, API contract and load tests.

Q13. What is LLMOps?

Operating LLM applications: prompt and configuration versioning, evaluation with golden sets and LLM-as-judge, tracing, cost and latency monitoring, guardrails and security, model routing and fallbacks, and feedback-driven improvement.

Q14. How do you control LLM costs?

Route to cheaper models where possible, shorten prompts and context, cap output tokens, use prompt and semantic caching, batch offline jobs, and self-host or fine-tune smaller models at high volume.

Q15. SHAP vs LIME?

Both explain individual predictions. LIME fits a local surrogate linear model on perturbed samples; SHAP uses Shapley values with consistency guarantees and exact fast algorithms for trees, giving additive feature contributions.

Q16. Describe an end-to-end MLOps pipeline you would build.

Data ingestion and validation → feature engineering → training with MLflow tracking → evaluation gate vs champion → registry → Docker image via CI → canary deploy on Kubernetes/managed endpoint → monitoring of system, drift and business metrics → automated retraining triggers.

CHAPTER 9

Capstone Projects & Assessments

End-to-end ML/DL project • Integrating RAG and agents • Real-world use cases • System design walkthroughs • Mock client interview • Project evaluation rubric

9.1 Capstone 1: End-to-End ML/DL Project

Example: bank customer propensity model — rank 4 lakh customers and select the 1 lakh most likely to accept a product offer.

Phase	What you do	Deliverable
1. Problem framing	business goal (conversion with fixed call budget), ML task (binary classification + ranking), metrics (precision@top-25%, lift, AUC), constraints (explainability for compliance)	problem statement
2. Data	join CRM, transactions, past campaigns (SQL); define label window; prevent leakage (only features before campaign date)	clean dataset + data dictionary
3. EDA	distributions, missing values, class imbalance, correlations, segment analysis	EDA notebook with plots
4. Features	RFM features (recency, frequency, monetary), balances, product holdings, tenure, digital activity, ratios	feature pipeline (sklearn Pipeline)
5. Modelling	baseline logistic regression → Random Forest → XGBoost/LightGBM; stratified CV; Optuna tuning; MLflow tracking	model comparison table
6. Evaluation	ROC/PR curves, lift/gain chart, calibration, SHAP global + local explanations, fairness check	evaluation report
7. Deployment	batch scoring job (monthly) + FastAPI for real-time scoring; Docker; registry	deployed service
8. Monitoring	PSI drift on key features, conversion tracking, retraining trigger	dashboard

$$\text{Lift@}k = \frac{\text{precision in top } k\%}{\text{overall positive rate}}$$

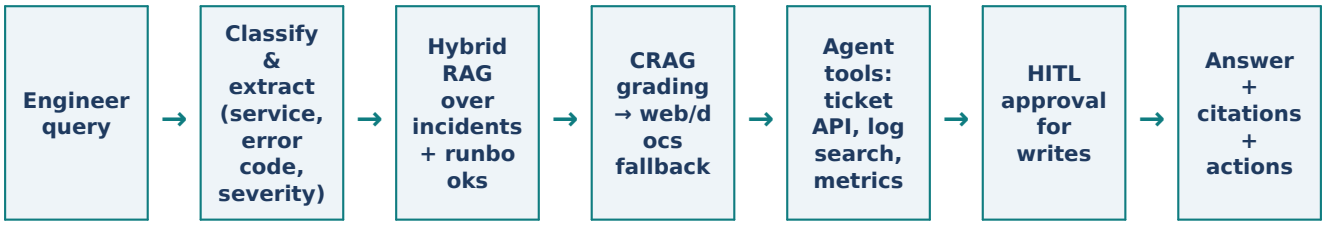
e.g. 20% conversion in the top decile vs 5% overall → lift = 4×

DL variant: visual defect detection



9.2 Capstone 2: Integrating RAG and Agents

Example: IT Incident Intelligence Assistant — engineers describe an incident; the assistant finds similar past incidents and runbooks, proposes a resolution, and can create/update tickets with approval.



Layer	Choice	Why
Orchestration	LangGraph (state, conditional edges, checkpointer, interrupt)	loops, approvals, memory
LLM	Gemini Flash for routing/grading, Pro-class model for final answer	cost/quality routing
Retrieval	Chroma/Qdrant + BM25 + RRF + cross-encoder rerank; metadata filters (service, date)	error codes need BM25
Structured output	Pydantic models for classification and tool args	reliable parsing
API	FastAPI with streaming SSE; auth	real-time UX
Evaluation	golden incidents set; RAGAS faithfulness/context recall; tool-call accuracy; pytest	measurable quality
Ops	Docker, LangSmith tracing, cost logging, feedback buttons	production readiness

suggested structure

```
incident_assistant/
├── main.py               # FastAPI app: /chat (stream), /feedback, /health
├── graph.py              # StateGraph wiring: classify → retrieve → grade → (rewrite|web) → agent ⇄ tools → approv
├── state.py              # TypedDict IncidentState (messages, incident, docs, plan, approvals, retries)
├── nodes.py              # node functions (each returns a partial state update)
├── tools.py              # @tool search_logs, get_metrics, create_ticket, update_ticket
├── retrieval.py          # hybrid retriever + reranker + filters
├── prompts.py            # versioned prompt templates
├── schemas.py            # Pydantic: IncidentClass, Grade, ToolArgs, Answer
└── tests/test_graph.py   # pytest with mocked LLM: routing, grading, tool-call paths
```

9.3 Real-World Use-Case Catalogue

Industry	Problem	Solution pattern
Banking	fraud detection	XGBoost on transaction features + graph features; real-time API; drift monitoring
Banking	KYC document processing	OCR + layout model + LLM extraction to JSON + human review
Insurance	claims triage	classification + RAG on policy wording + agent for missing info
Retail	product search & recommendations	hybrid search + embeddings + re-ranking; collaborative filtering
Telecom	churn prevention	propensity model + uplift modelling + next-best-action

Industry	Problem	Solution pattern
Healthcare	clinical note summarisation	domain LLM + RAG on guidelines + strict citations + clinician sign-off
Manufacturing	predictive maintenance	sensor time series → LSTM/GBM anomaly + forecasting
IT services	ticket routing & resolution	BERT/LLM classifier + RAG on past tickets + agent actions
HR	policy assistant	conversational RAG with ACLs
Marketing	content generation	LLM + brand RAG + diffusion images + approval workflow

9.4 System Design Walkthroughs (How to Answer)

Use a consistent framework in any design interview:

1. **Clarify requirements**: users, scale (QPS, data size), latency, accuracy target, constraints (privacy, cost, explainability).
2. **Define metrics**: offline (F1, recall@k, faithfulness) and online (CTR, resolution rate, cost).
3. **Data**: sources, labelling, features, freshness, privacy.
4. **Model / approach**: baseline first, then improvements; justify trade-offs (rules vs ML vs LLM; RAG vs fine-tune).
5. **Architecture**: draw the online and offline paths (ingestion, training, serving, storage, caches).
6. **Serving & scaling**: batch vs real-time, autoscaling, latency budget per component.
7. **Evaluation & monitoring**: golden sets, A/B tests, drift, feedback loops.
8. **Risks**: failure modes, security, responsible AI, fallbacks, human-in-the-loop.

Design 1: Customer-support GenAI assistant for 1M users

- Latency budget (p95 ≈ 3 s first token < 1 s): router (small model, 150 ms) → retrieval (hybrid + rerank, 300 ms) → generation streamed.
- Scale: stateless API pods behind a load balancer; Redis for sessions/semantic cache; managed vector DB; provider rate-limit management with queues.
- Quality: CRAG grading, citations, escalation to human when confidence low; weekly eval regression.
- Cost: cache FAQs, route simple intents to templates/small model, cap tokens.

Design 2: Real-time fraud detection

- Strict latency (< 50 ms): precomputed features in an online feature store (Redis), gradient-boosted model served in-process (ONNX), rules engine in parallel.
- Imbalance: cost-sensitive learning, threshold by business cost, PR-AUC.
- Feedback: chargeback labels arrive weeks later → monitor prediction drift and proxy metrics; retrain weekly; shadow-test new models.
- Explainability: reason codes from SHAP for analysts/regulators.

9.5 Mock Client / SME Interview

Practise answering aloud in 60–90 seconds: **context → approach → result (numbers) → learning** (STAR for project questions).

Project & behavioural questions

Q1. Walk me through a GenAI project you built.

Structure: business problem and users → architecture (e.g. LangGraph agent with hybrid RAG, Gemini, Chroma, FastAPI) → key decisions and trade-offs (why hybrid search, why HITL) → evaluation (golden set, faithfulness, tests passing) → result and next steps (caching, monitoring).

Q2. How would you explain RAG to a non-technical client?

It is like giving the AI an open-book exam: before answering, it looks up the company's own documents and answers using them, with references, so answers are current and verifiable.

Q3. The client wants 100% accurate answers from the chatbot. How do you respond?

Set realistic expectations: no probabilistic model is 100% accurate. Offer controls — grounding with citations, 'I don't know' behaviour, confidence thresholds with human handoff, evaluation reports, and monitoring — and agree measurable targets.

Q4. How do you choose between building on GPT, Gemini or an open-source model?

Evaluate on the client's own task data for quality, latency and cost; consider data residency and privacy (self-hosting may be required), context length, tool-calling reliability, licensing and vendor support; often use a router with more than one model.

Q5. How do you handle sensitive data in GenAI solutions?

Data classification, PII redaction before sending to models, private endpoints/enterprise agreements with no training on data, access control in retrieval, encryption, logging policies and audits.

Q6. Your model performs well offline but poorly in production. What do you check?

Training-serving skew, data drift, leakage in offline evaluation, different preprocessing, label definition changes, latency timeouts, and whether the offline test set represented real traffic.

Q7. Tell me about a time you debugged a failing pipeline.

Describe the symptom (failing tests, wrong outputs), how you isolated it (logs, tracing, smaller reproducible case), the root cause (e.g. schema mismatch, wrong state key), the fix and the test you added to prevent regression.

Q8. How do you keep up with the fast-changing AI field?

Follow official docs and release notes of key frameworks, papers via summaries, build small prototypes to validate new techniques, and evaluate before adopting in production.

Q9. What would you do in the first 30 days on a GenAI client project?

Understand business goals and users, audit data sources and access, define success metrics and a golden evaluation set, build a thin end-to-end prototype, and agree on governance and a roadmap.

Q10. Why should we hire you as an AI engineer?

Combine fundamentals (ML/DL maths), hands-on delivery (FastAPI, LangGraph, RAG, testing), production thinking (evaluation, monitoring, cost, security) and clear communication with stakeholders — supported by concrete project examples.

9.6 Incremental Project Evaluation Rubric

Criterion	Weight	Excellent looks like
Problem understanding	10%	clear objective, metric tied to business value
Data & features	15%	clean, leakage-free, documented
Modelling / architecture	20%	justified choices, baselines, trade-offs
Evaluation	15%	right metrics, golden sets, error analysis
Code quality & tests	15%	modular files, type hints, pytest passing, README
Deployment & MLOps	15%	API, Docker, tracking, monitoring plan
Communication	10%	concise demo, diagrams, limitations stated

CHAPTER 10

Rapid Revision: Formula Sheet, One-Liners & Roadmap Coverage

Key formulas • 60-second SME answers • Common confusions • Roadmap-to-chapter checklist

10.1 Formula Sheet

Bayes	$P(A\|B) = \frac{P(B\|A)P(A)}{P(B)}$
Cosine similarity	$\cos\theta = \frac{a \cdot b}{\|a\|\,\|b\|}$
Gradient descent	$\theta \leftarrow \theta - \eta \nabla_\theta L$
Linear / logistic regression	$\hat{y} = w^T x + b \qquad p = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$
Cross-entropy	$L = -\sum_k y_k \log \hat{y}_k \qquad \text{BCE} = -[y \log p + (1-y)\log(1-p)]$
Softmax with temperature	$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$
Gini / entropy	$G = 1 - \sum p_k^2 \qquad H = -\sum p_k \log_2 p_k$
Precision / recall / F1	$P = \frac{TP}{TP+FP} \quad R = \frac{TP}{TP+FN} \quad F_1 = \frac{2PR}{P+R}$
Bias-variance	$\text{Err} = \text{Bias}^2 + \text{Var} + \sigma^2$
Conv output size	$O = \frac{W - K + 2P}{S} + 1$
Attention	$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
TF-IDF	$\text{tf}(t,d) \cdot \log \frac{N}{1 + \text{df}(t)}$
LSTM cell	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad h_t = o_t \odot \tanh(C_t)$
VAE loss	$\mathcal{L} = \mathbb{E}[\log p(x\|z)] - D_{KL}(q(z\|x)\|p(z))$
GAN objective	$\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$

Diffusion	$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon, \quad L = \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$
LoRA	$W = W_0 + \frac{\alpha}{r}BA$
Adam	$\theta \leftarrow \theta - \eta\frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon}$
RRF	$\mathrm{RRF}(d) = \sum_r \frac{1}{k + \mathrm{rank}_r(d)}$
PSI	$\sum (a_i - e_i)\ln\frac{a_i}{e_i}$
Perplexity	$\exp\left(-\frac{1}{T}\sum_t \log p(x_t \mid x_{<t})\right)$

10.2 Common Confusions (say the difference in one line)

Pair	Difference
AI vs ML vs DL vs GenAI	AI = goal; ML = learn from data; DL = deep neural nets; GenAI = DL that creates content
Parameter vs hyper-parameter	learned from data (weights) vs set before training (learning rate, depth)
Validation vs test set	validation tunes choices; test is touched once for the final unbiased estimate
Epoch vs batch vs iteration	one full pass vs subset per update vs one update step
Bagging vs boosting	parallel, reduces variance vs sequential, reduces bias
L1 vs L2	sparse weights vs small weights
Batch norm vs layer norm	across the batch per feature vs across features per sample
Precision vs recall	how many predicted positives are right vs how many actual positives are found
ROC-AUC vs PR-AUC	ranking quality overall vs focus on the positive class under imbalance
Encoder vs decoder transformer	bidirectional understanding vs causal generation
Embedding vs token	dense meaning vector vs discrete text unit/ID
Fine-tuning vs RAG	changes behaviour/weights vs supplies knowledge at runtime
LoRA vs QLoRA	low-rank adapters vs adapters on a 4-bit quantised base model
Bi-encoder vs cross-encoder	separate fast embeddings for retrieval vs joint slow scoring for re-ranking
Chain vs agent	fixed steps vs LLM-chosen steps with tools in a loop
LangChain vs LangGraph	components and linear chains vs stateful graphs with cycles, persistence and HITL
Data drift vs concept drift	P(X) changes vs P(y|X) changes
Canary vs A/B	risk-limited rollout vs controlled experiment measuring impact
GAN vs VAE vs diffusion	adversarial sharp/unstable vs probabilistic blurry/stable vs iterative denoising best quality/slower
Semantic vs instance segmentation	class per pixel vs separate mask per object

Pair	Difference
Stemming vs lemmatisation	rule-based chopping vs dictionary base form

10.3 Sixty-Second SME Answers

Rapid-fire set

Q1. When should you NOT use ML?

When simple rules solve it reliably, data is insufficient or unrepresentative, errors are unacceptable without explainability, or the cost of building and maintaining ML outweighs the value.

Q2. When do you stop EDA?

When you understand data quality issues, distributions, target balance, key relationships and leakage risks well enough to make preprocessing and modelling decisions — then iterate via error analysis.

Q3. Why does PCA require scaling?

PCA maximises variance; features with larger units would dominate the components, so standardise first.

Q4. Why XGBoost for tabular data?

It captures non-linearities and interactions, handles missing values, is regularised, fast and typically top-performing on structured data with little preprocessing.

Q5. Why transformers replaced RNNs?

Parallel training, direct long-range connections via attention, and excellent scaling with data and compute.

Q6. What makes a good embedding model for RAG?

High retrieval scores on your domain data, suitable language support and max length, acceptable dimension/latency/cost, and the same model used for queries and documents.

Q7. How do you stop hallucinations?

Ground with retrieval and citations, instruct to answer only from context with an 'I don't know' option, low temperature, verification/groundedness checks, and continuous evaluation.

Q8. What is the role of the system prompt?

It sets persistent behaviour: role, rules, tone, safety boundaries and output format for the whole conversation.

Q9. What is token streaming good for?

It reduces perceived latency by showing tokens as they are generated; it doesn't change accuracy or total compute.

Q10. Which agent pattern relies heavily on tools?

ReAct — reasoning interleaved with tool actions and observations.

Q11. How do you make an LLM output reliable JSON?

Use structured output / function calling with a Pydantic or JSON schema, validate the result, and retry with the validation error on failure.

Q12. What is conversation state in an agent?

The per-session memory (messages and task variables) carried between turns, persisted per thread, enabling context-aware responses — not permanent global storage.

Q13. What is a fallback in an agent?

An alternative path when the primary fails — e.g. if a tool errors, answer with the LLM or another tool, or escalate to a human.

Q14. Why evaluate retrieval separately from generation?

To know whether failures come from missing context (fix retrieval) or from the model misusing context (fix prompting/generation).

10.4 Roadmap Coverage Checklist

Every topic in your training roadmap mapped to where it is taught in this handbook.

Roadmap module	Topics	Chapter / section
1.1 Prerequisites	Python, NumPy, Pandas, Matplotlib/Seaborn, SQL, statistics & probability, linear algebra, calculus	1.1–1.4
1.2 ML overview	ML/DL/GenAI, types of ML, use cases, end-to-end pipeline	1.5
1.3 Tools	Jupyter, scikit-learn, TF/PyTorch, Hugging Face, Git/GitHub, VS Code	1.6
2.1 Data preparation	collection/cleaning, feature engineering, split, missing values, scaling, PCA/t-SNE	2.1
2.2 Classical ML	linear/logistic regression, decision trees, Random Forest & ensembles, SVM, KNN, Naive Bayes, when to use which	2.2
2.3 Evaluation & tuning	accuracy, precision, recall, F1, ROC-AUC, CV, Grid/Random search, bias–variance, overfitting, improvement strategies	2.3
2.4 Persistence & serving	pickle/joblib, FastAPI ML service, multi-model comparison	2.4
3.1 NN basics	ANN, activations, losses, optimizers, backprop, Keras MNIST	3.1
3.2 CNN	convolution/pooling, LeNet, AlexNet, VGG, ResNet, EfficientNet (+Inception, DenseNet, MobileNet, ViT), transfer learning, R-CNN/YOLO/SSD, U-Net, Mask R-CNN, visualisation, CV project	3.2
3.3 RNN	RNN theory, vanishing gradient, LSTM, GRU, BiLSTM, seq2seq, attention, time-series forecasting	3.3
3.4 Generative models	GANs, advanced GAN variants, VAEs, comparison with modern GenAI	3.4
4.1 Classical NLP	preprocessing, TF-IDF, n-grams, Word2Vec, GloVe	4.1
4.2 Sequence models	RNN/LSTM/GRU, pre-transformer attention, transition to transformers	4.2 (+3.3)
4.3 Transformers	architecture deep dive, why it works, BERT pre-training, variants, optimisation	4.3–4.4
4.4 LLMs	applications, conversational AI, multilingual, landscape, OpenAI/HF API integration (+training, decoding, prompting, LoRA, quantisation)	4.5
4.5 NLP projects	text classification, summarisation, chatbots	4.6
5.1–5.4 Generative AI	GenAI intro, diffusion theory, Stable Diffusion, DALL-E, image generation, video, TTS, music, multimodal, code generation, content creation, ethics	Chapter 5
6.1–6.4 RAG	fundamentals, vector DBs & embeddings, loading & chunking, pipelines, evaluation, advanced patterns, multi-document & metadata filtering, conversational memory, Graph RAG, enterprise app, use cases	Chapter 6

Roadmap module	Topics	Chapter / section
7.1–7.4 Agentic AI	agent basics & architectures, LangChain, LangGraph, first agent, planning, multi-agent, HITL, memory/state, tools, error handling, streaming, evaluation, production design, research & support agents	Chapter 7
8.1–8.4 MLOps & LLMOps	principles, MLflow, registry, deployment strategies, FastAPI, TF Serving, Docker, Kubernetes, optimisation, CI/CD, testing, monitoring, drift, A/B & retraining, SageMaker/Azure ML/Vertex AI, LLMOps, cost, security, governance, explainability	Chapter 8
9 Capstone	end-to-end project, RAG + agents, use cases, mock client interview, evaluation	Chapter 9